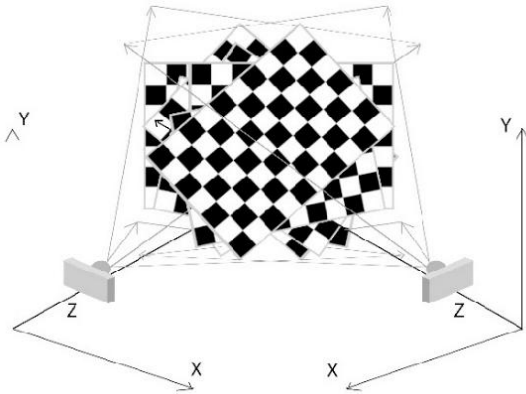
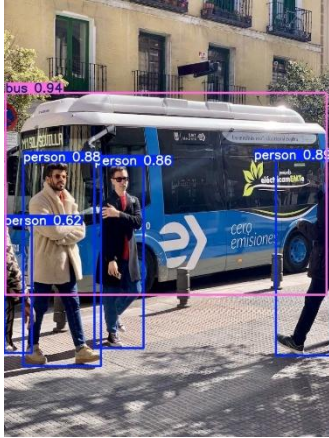


협동로봇 2

AI/음성/언어 기반 협동 로봇

- 과정명 : [두산로보틱스] 지능형 로보틱스 엔지니어
- 주제 : AI(Computer Vision)기반 협동 로봇 작업 어시스턴트 구현 프로젝트

Chapter	내 용	
00	목차, 수업목표, 수업계획, 프로젝트 개요	  
01	Realsense Depth Camera, Voice Processing 및 Tutorial 패키지 설치	
02	Realsense Depth Camera Topic 및 PointCloud with RQt and RViz2	 
03	Gripper 제어와 Modbus 통신	
04	Depth Camera Calibration(Checker Board and Eye-In-Hand Calibration)	
05	Object Detection(Ultralytics Settings, Datasets, Model Training, Inference)	
06	Object Detection → Pick and Place with Doosan Robot	  
07	Wakeup Word, Voice Processing and STT(Speech To Text)	
08	Pick and Place with Voice Processing(Wakeup word, Keyword Extraction, Voice Command)	

▶ 목표

- 프로젝트 수행방법 및 절차(문제 해결 능력)
- RGBD Camera(RealSense) Calibration, Topic, Service, Interface, Doosan 협동로봇 제어
- On-Robot RG2 Gripper 제어와 Modbus 통신 활용 Object Pick and Place
- Object Detection, 모델 학습, 모델 추론
- Voice Processing, Wakeup word, Keyword Extraction, Voice Command로 로봇 움직이기
- Docker & Container 활용한 독립적 환경을 통해 개발, 테스트, 배포, 구축 및 운영(코드 및 종속성을 표준 형식으로 패키징)

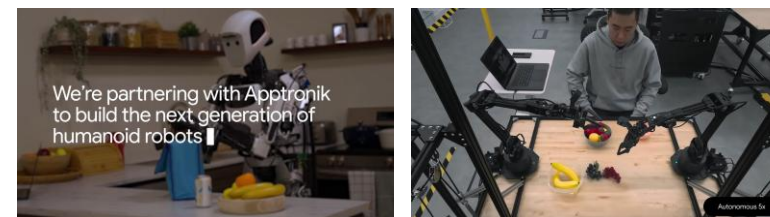
▶ 세부 목표

- 프로젝트 개요, 수행과제(문제) 정의 및 목표, 구성원 역할, 문제 해결 전략, 차별성, Lesson Learned, 개선 및 도전 과제
- 개발환경 구축(Ubuntu 22.04, ROS2 Humble) + Realsense ROS2 패키지 설치
- 음성 인식 및 추출 → RealSense Topic Subscribing 및 전처리 → Object Detection 및 추론 → Robot Control로 이어지는 pipeline 구축
- Pymodbus 기반 Onrobot RG2 그리퍼 제어
- Object Detection 모델 구축, 음성 인식 및 지능형 로봇 구축, 사람과 협업하는 반 자율 협동 로봇 구현
- 호스트와 컨테이너간 통신(Message 송수신) / 2대 이상의 PC에 2개 이상 컨테이너와 호스트간 통신 구축

차시	주제	세부 내용	Chapter
1일차	RGB Camera 로봇 통신환경 구축 - 1	Depth Camera 패키지 설치, Camera Topic, PointCloud with RQt and RViz2, Gripper제어와 Modbus	01, 02, 03
2일차	RGB Camera 로봇 통신환경 구축 - 2	Depth Camera Calibration(이론 및 개념, Checker Board & Eye-in-Hand Calibration, 3차원 좌표 추정)	04
3일차	Vision AI 모델 기반 로봇 서비스 구축 - 1	딥러닝 기반 인공지능 모델 기초(YOLO, Ultralytics Settings, Datasets, Model Training, Inference)	05
4일차	Vision AI 모델 기반 로봇 서비스 구축 - 2	Doosan ROS2 & DRL 기반 서비스 구축, Object Detection, Pick and Place	06
5일차	Vision AI 모델 기반 로봇 서비스 구축 - 3	Object Detection → Text 입력 기반 Pick and Place 구현	06
6일차	음성/언어 모델 기반 로봇 서비스 구축 - 1	Wakeup Word 모델 및 Voice Recognition API 사용법, STT(Speech To Text), Keyword Extraction	07
7일차	음성/언어 모델 기반 로봇 서비스 구축 - 2	LLM(Large Language Model) 소개 및 사용, LangChain 기반 Node 생성 및 ROS2 통신	08
8일차	AI기반 협동 로봇 작업 어시스턴트 구현 - 1	단위 모듈 통합 및 AI기반 협동 로봇 완성	-
9일차	AI기반 협동 로봇 작업 어시스턴트 구현 - 2	시나리오, 시스템 설계, 팀 프로젝트 개발 실습 및 적용	-
10일차	AI기반 협동 로봇 작업 어시스턴트 구현 - 3	팀 프로젝트 시연, 발표 및 피드백, 팀 프로젝트 산출물 정리	-

▶ 프로젝트 최종 결과물

- 사람의 음성명령 인식 및 문장을 분석하여 해당 물체를 인식하고 명령한 장소로 이동하는 로봇 제작



▶ 필요 Skill

- **Object Detection** : 사용자가 원하는 물체의 위치를 파악하는 데 사용
- **Wakeup word** : "지금부터 사용자가 명령을 내릴 것이다"라는 신호를 주기 위해 사용(ex : 시리야)
- **STT(Speech-to-Text)** : 사용자의 음성을 텍스트로 변환하는 데 사용
- **Lang-Chain** : 사용자의 명령어에서 핵심 키워드를 추출하는 데 사용(ex: 망치 가져와 → 키워드 : 망치)

▶ ROS2 패키지 및 기타 라이브러리 설치

- 아래 링크 접속해서 설치 진행
- <https://url.kr/cftiwt>

■ 링크 클릭 →

협동로봇2 패키지 설치 Tutorial

1. 개발 환경 구축(조별 노트북 또는 개인 노트북)

아래 2개 절차 중 1개 절차 선택 진행



A. Ubuntu 초기화 상태(MSI노트북)



B. 협동1 환경 유지

2. Camera Calibration, Voice 등 Base Code 다운로드

- 아래 2개 파일 다운로드 및 압축 풀기

1. 폴더 → 압축 풀고 `[home/Tutorial]` 로 이동

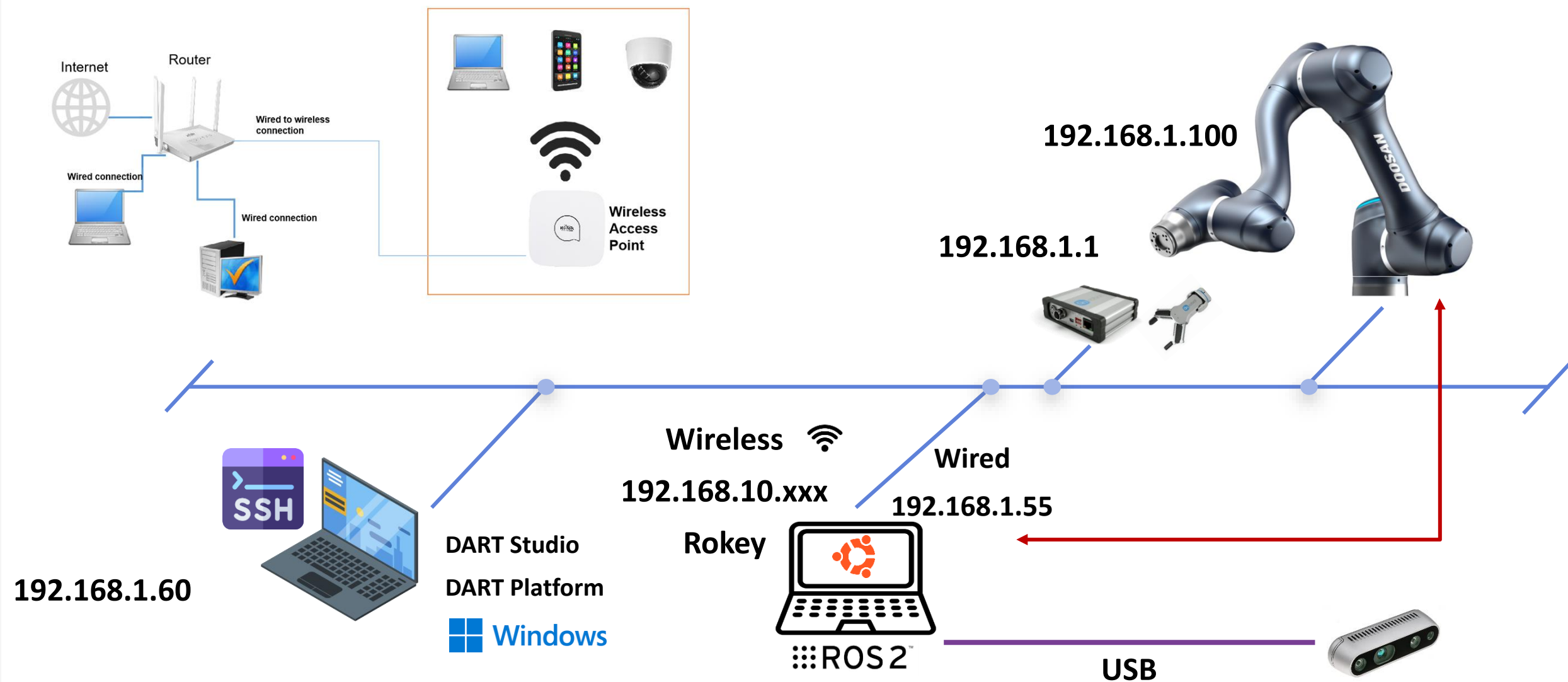
Tutorial_2026.zip 1.7 MiB

1. Camera Calibration, Voice Recording 등 단위 기능 별 Source Code
2. Python Code로 구현되어 있음(왜 ROS2 패키지가 아닌지 확인해보기)

2. 폴더 → 압축 풀고 `[home/cobot_ws/src]` 로 이동

rokey_cobot2.zip

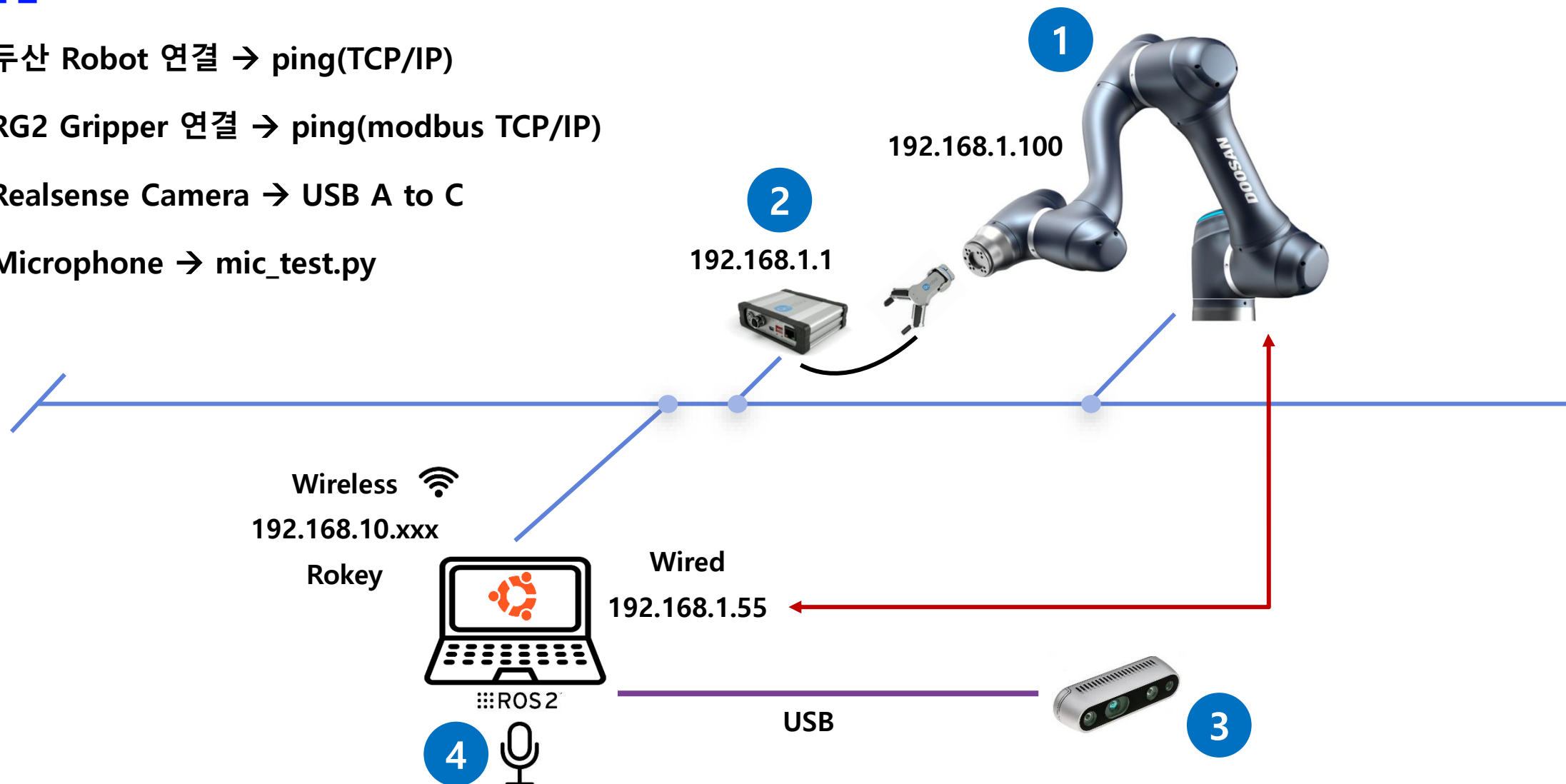
System Configuration



개별 Device 동작 확인

장비 점검

1. 두산 Robot 연결 → ping(TCP/IP)
2. RG2 Gripper 연결 → ping(modbus TCP/IP)
3. Realsense Camera → USB A to C
4. Microphone → mic_test.py



Doosan Robot & RG2 DIO(Digital Input Output)

RG2 Gripper Controller



DOOSAN Robot

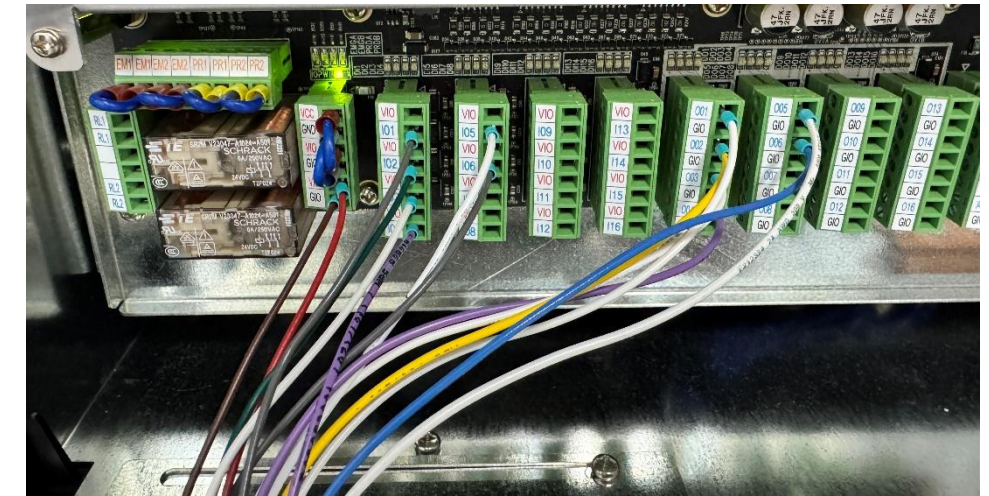


Grip →

IN	1	2	3	4	5	6	7	8
	1	0	-	-	-	-	-	-

Release →

IN	1	2	3	4	5	6	7	8
	0	1	-	-	-	-	-	-

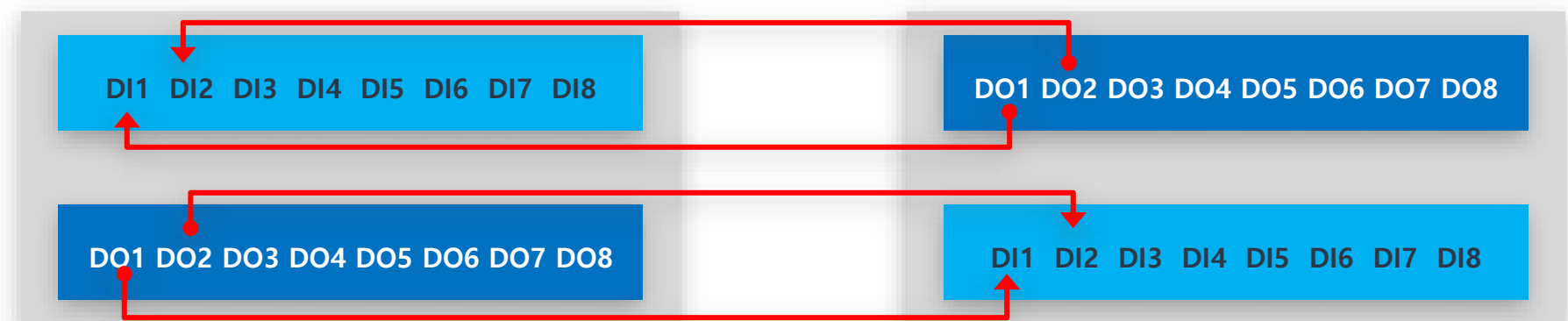


Grip →

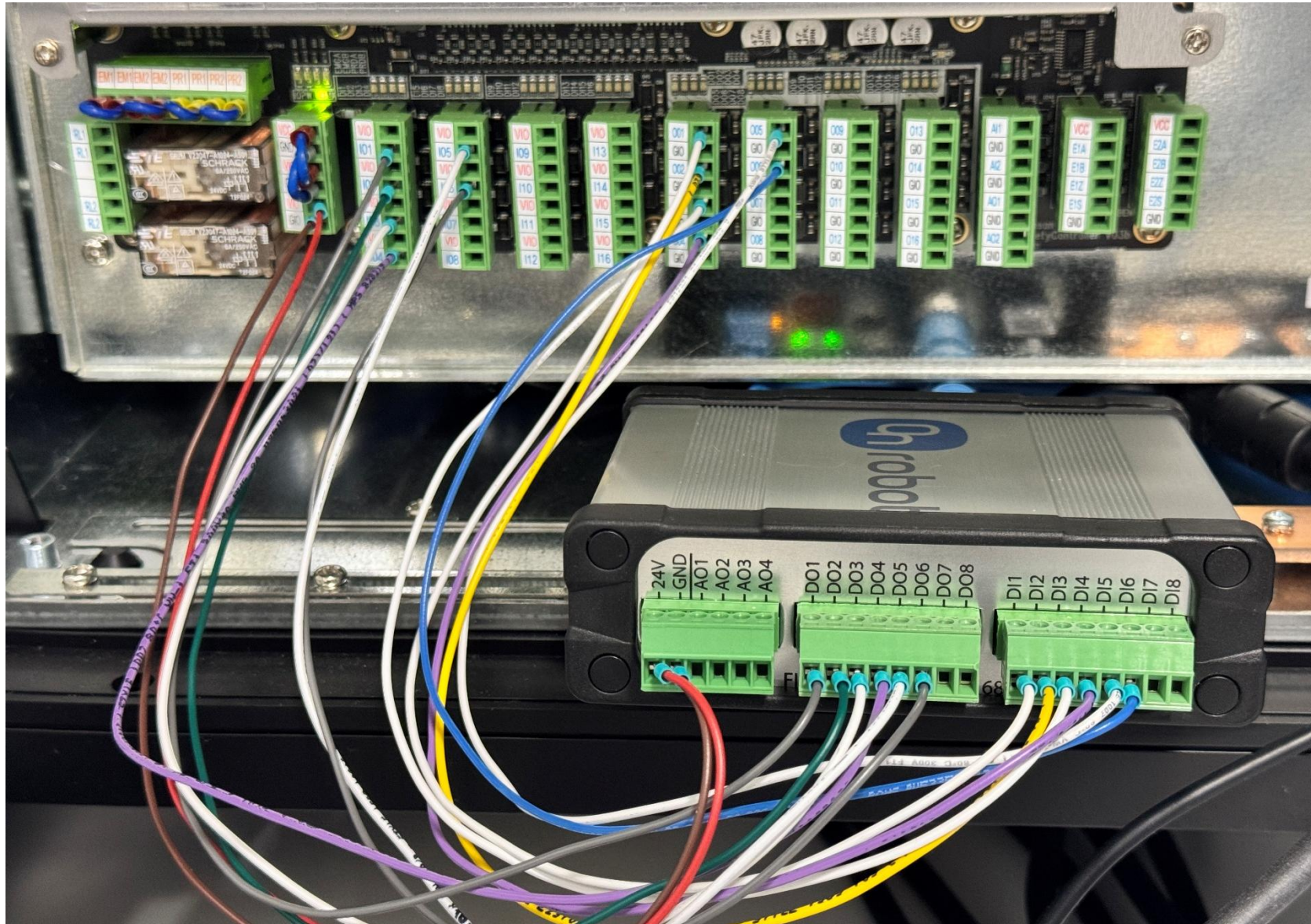
OUT	1	2	3	4	5	6	7	8
	1	0	-	-	-	-	-	-

Release →

OUT	1	2	3	4	5	6	7	8
	0	1	-	-	-	-	-	-



DIO(Digital Input Output) & AIO(Analog Input Output)



Grip →

✓ #0									
IN	1	2	3	4	5	6	7	8	
	1	0	-	-	-	-	-	-	

Release →

✓ #1									
IN	1	2	3	4	5	6	7	8	
	0	1	-	-	-	-	-	-	

Grip →

OUT	1	2	3	4	5	6	7	8	
	1	0	-	-	-	-	-	-	

Release →

OUT	1	2	3	4	5	6	7	8	
	0	1	-	-	-	-	-	-	

This page allows to browse/manage the OnRobot WebLogic™ programs. You can create new program and run it on the Editor tab. (To make your program run automatically on power-on, leave it running while powering the Compute Box off.)

Browser Program Editor

RG2_LEE

Program is running

NEW

SAVE

STOP

#0 SELECT DEVICE ▼ SELECT DEVICE ▼ IN 1 2 3 4 5 6 7 8 0 - - - - 1 - - AND (No device selected AND No device selected) → RG2-0 ▼ Width (110 mm, 40 N) OUT 1 2 3 4 5 6 7 8 - - - - - - - - ▼
#1 SELECT DEVICE ▼ SELECT DEVICE ▼ IN 1 2 3 4 5 6 7 8 0 - - - 1 - - - AND (No device selected AND No device selected) → RG2-0 ▼ Width (80 mm, 40 N) OUT 1 2 3 4 5 6 7 8 - - - - - - - - ▼
#2 SELECT DEVICE ▼ SELECT DEVICE ▼ IN 1 2 3 4 5 6 7 8 0 - - 1 - - - - AND (No device selected AND No device selected) → RG2-0 ▼ Width (60 mm, 40 N) OUT 1 2 3 4 5 6 7 8 - - - - - - - - ▼
#3 SELECT DEVICE ▼ SELECT DEVICE ▼ IN 1 2 3 4 5 6 7 8 0 - 1 - - - - - AND (No device selected AND No device selected) → RG2-0 ▼ Width (40 mm, 40 N) OUT 1 2 3 4 5 6 7 8 - - - - - - - - ▼
#4 SELECT DEVICE ▼ SELECT DEVICE ▼ IN 1 2 3 4 5 6 7 8 0 1 - - - - - - AND (No device selected AND No device selected) → RG2-0 ▼ Width (20 mm, 40 N) OUT 1 2 3 4 5 6 7 8 - - - - - - - - ▼
#5 SELECT DEVICE ▼ SELECT DEVICE ▼ IN 1 2 3 4 5 6 7 8 1 0 - - - - - - AND (No device selected AND No device selected) → RG2-0 ▼ Width (0 mm, 10 N) OUT 1 2 3 4 5 6 7 8 - - - - - - - - ▼

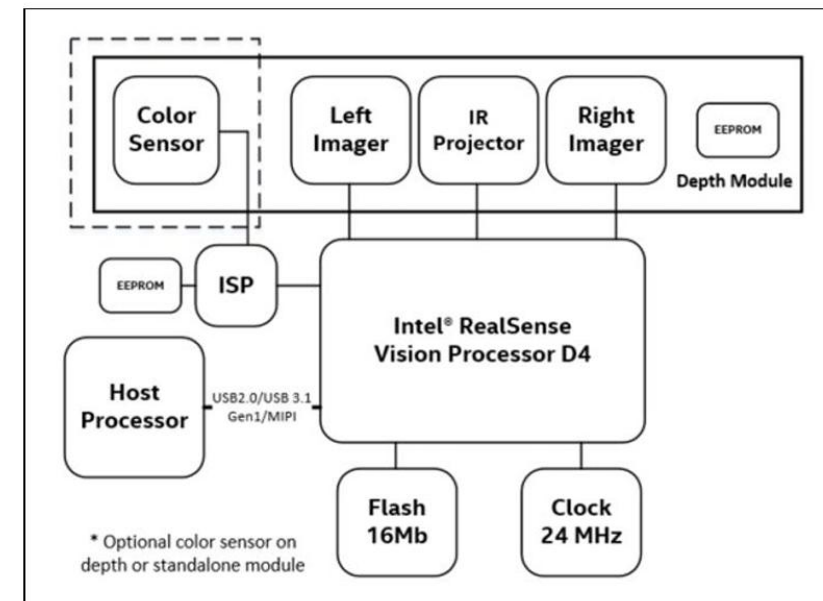
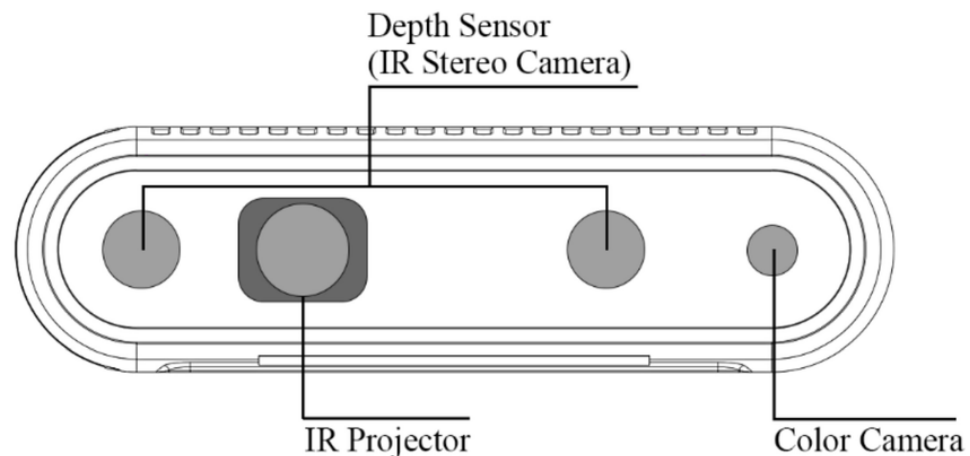
▶ Realsense

- Realsense는 컬러 이미지와 깊이(Depth)를 추출할 수 있는 RGB-D(rgb+depth) 센서로 로봇 비전, SLAM, 3D 매핑 등에 활용됨



▶ Realsense

- Realsense는 Depth Sensor, IR Projector, Color Camera를 가지고 있음
- Depth Sensor
 - ① 두 개의 적외선(IR) 카메라를 이용해 스테레오 방식으로 깊이(depth)를 계산
 - ② 두 카메라 간의 시차(disparity)를 이용해 거리 정보를 추정
 - ③ 빛의 변화에 강하게 영향을 받지 않으므로 실내, 야간에서도 안정적인 깊이 측정 가능

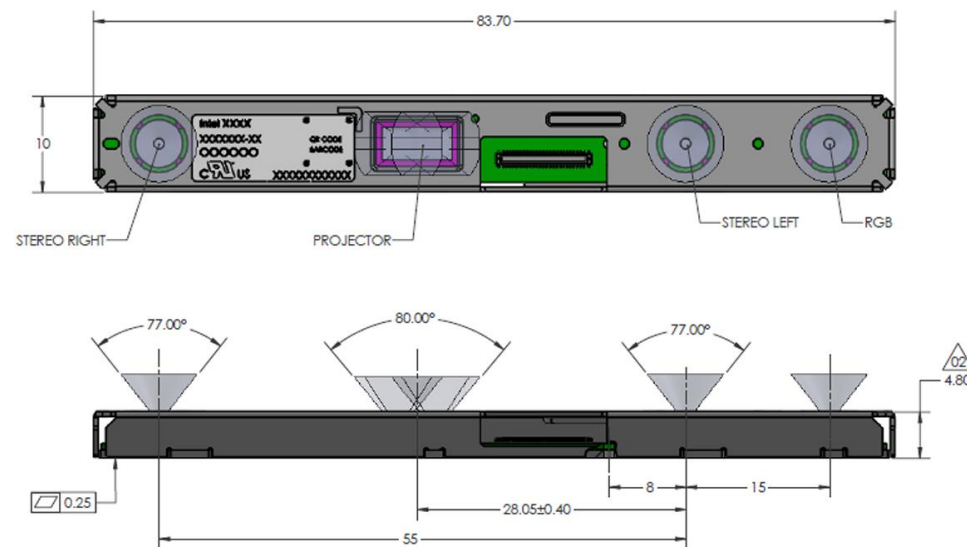
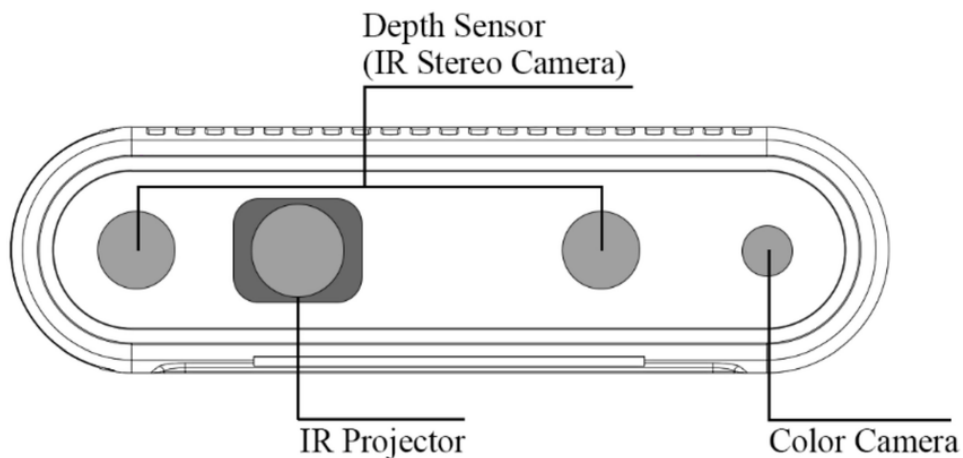


▶ Realsense

- Realsense는 Depth Sensor, IR Projector, Color Camera를 가지고 있음

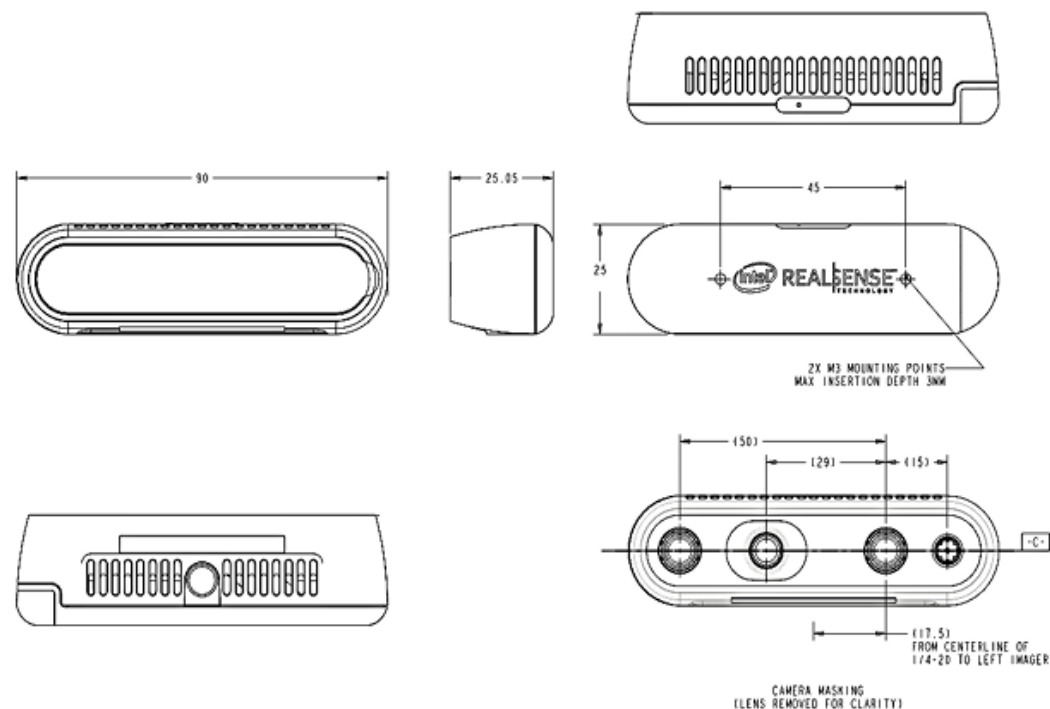
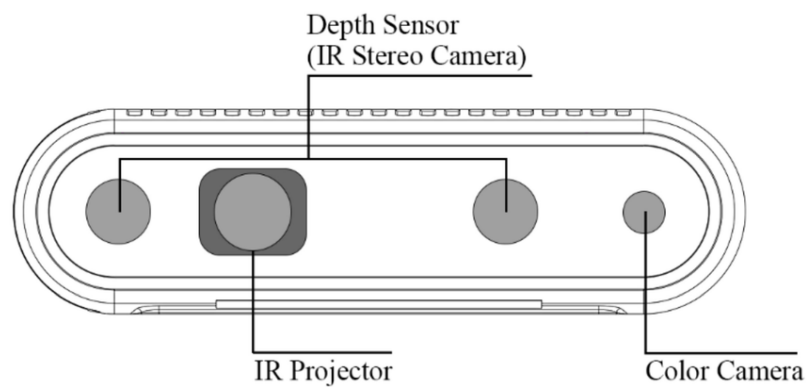
- IR Projector

- ① 보이지 않는 적외선(IR) 패턴을 주변 환경에 투사
- ② 표면에 명확한 texture(특징)가 없는 경우(예: 하얀 벽, 유리 등)에도 깊이 계산이 가능하도록 도움
- ③ 스테레오 매칭을 강화시켜 depth precision을 높이는데 기여



▶ Realsense

- Realsense는 Depth Sensor, IR Projector, Color Camera를 가지고 있음
- Color camera
 - ① 일반 RGB 컬러 이미지를 캡처
 - ② Depth 데이터와 함께 매핑(mapping)해서 사용하면 3D Reconstruction에도 활용 가능



Use environment:

Indoor/Outdoor

Ideal range:

.3 m to 3 m

Depth technology:

Stereoscopic

Depth Field of View (FOV):

87° × 58°

Image sensor technology:

Global Shutter

Minimum depth distance (Min-Z) at max resolution:

~28 cm

Depth output resolution:

Up to 1280 × 720

Depth Accuracy:

<2% at 2 m¹

Depth frame rate:

Up to 90 fps

Camera module:

Intel RealSense Module D430 + RGB Camera

Vision processor board:

Intel RealSense Vision Processor D4

RGB frame resolution:

1920 × 1080

RGB sensor FOV (H × V):

69° × 42°

Form factor:

Camera Peripheral

Connectors:

USB-C* 3.1 Gen 1*

RGB frame rate:

30 fps

RGB sensor resolution:

2 MP

Length × Depth × Height:

90 mm × 25 mm × 25 mm

Mounting mechanism:

- One 1/4-20 UNC thread mounting point.
- Two M3 thread mounting points.

RGB sensor technology:

Rolling Shutter

▶ **rqt를 이용하여 realsense topic 확인하기**

- 다음 명령어를 이용하여 realsense launch 파일 실행

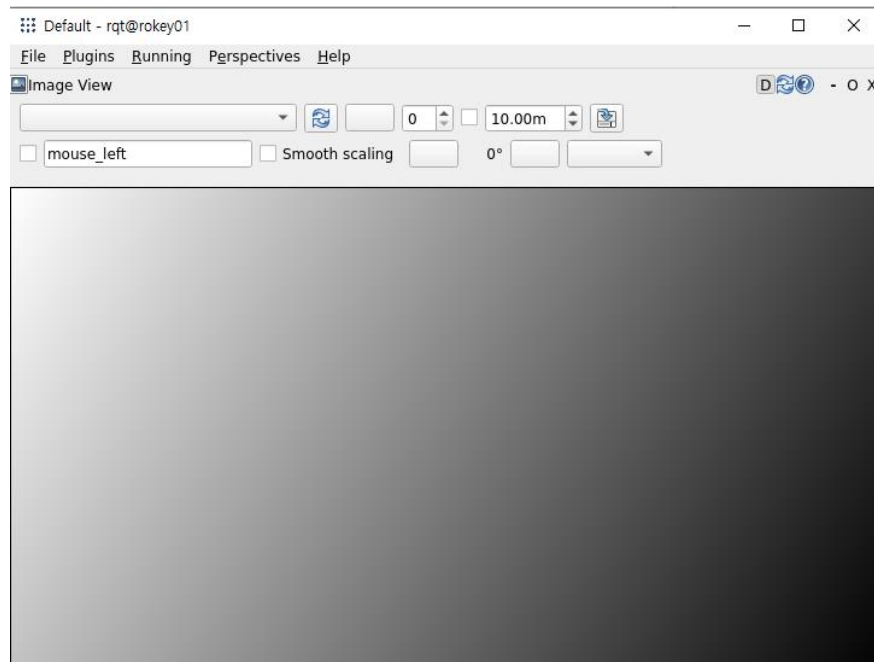
```
ros2 launch realsense2_camera rs_launch.py
```

▶ rqt를 이용하여 realsense topic 확인하기

- 새로운 터미널에서 rqt를 실행하여 발행되고 있는 토픽 확인

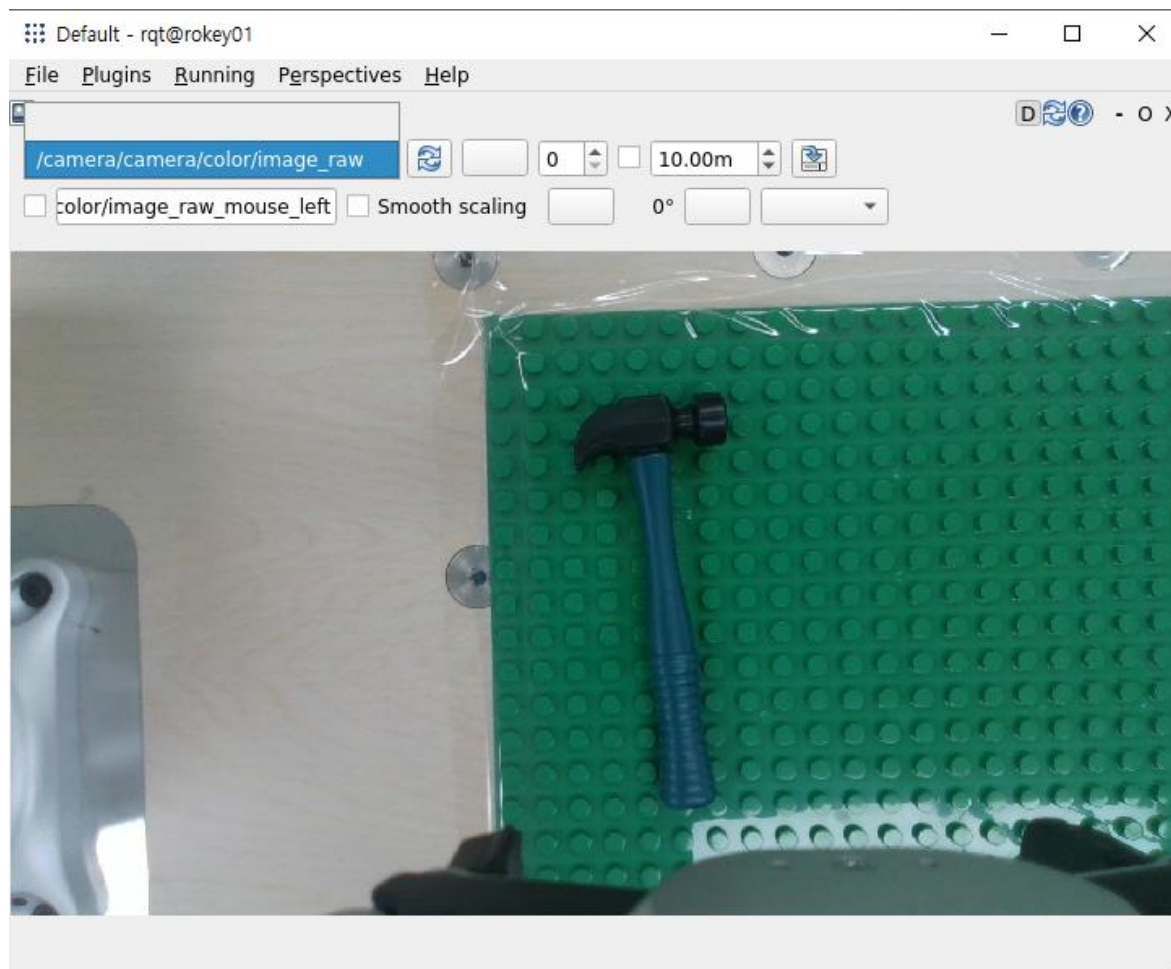
```
rqt
```

- 상단바에서 Plugins → Visualization → Image View를 선택



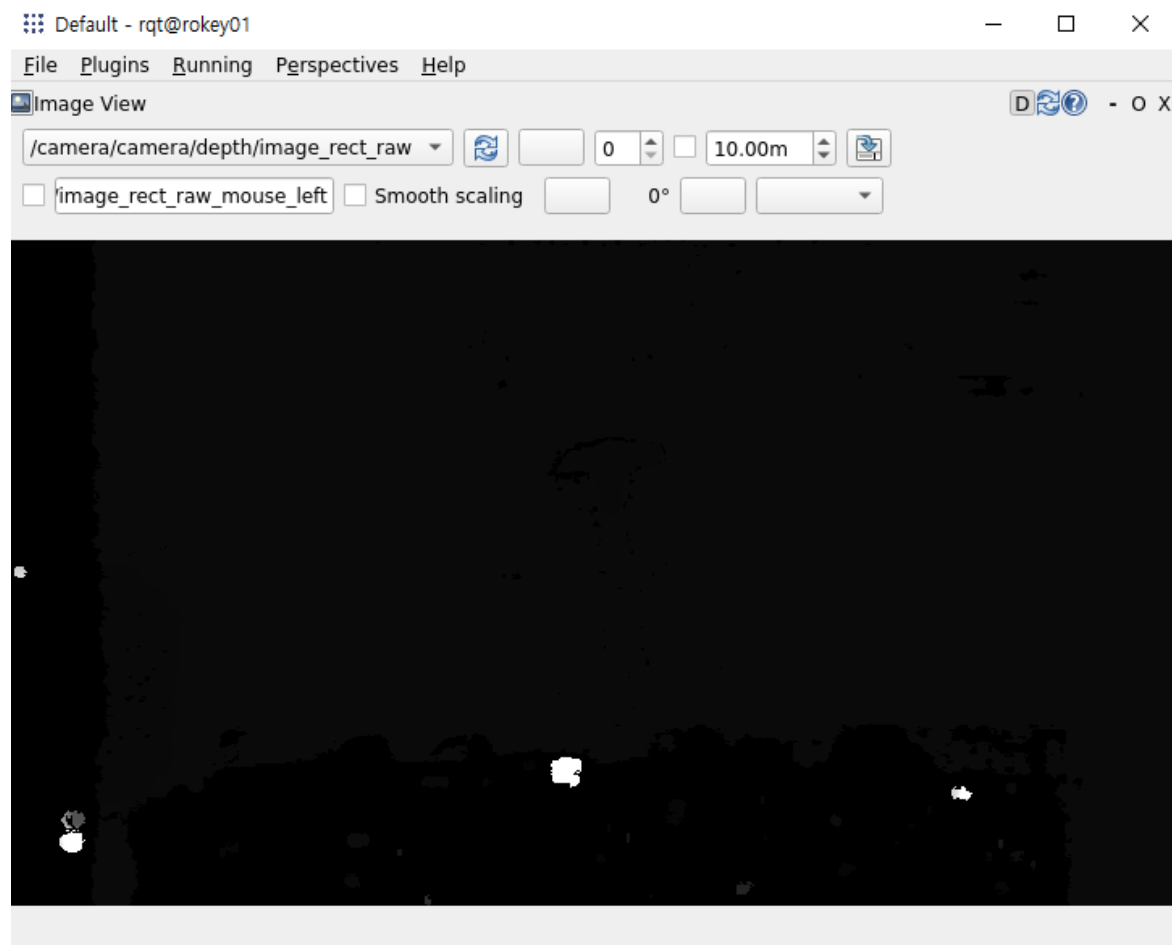
▶ rqt를 이용하여 realsense topic 확인하기

- /camera/camera/color/image_raw 토픽을 선택하여 color image 확인



▶ rqt를 이용하여 realsense topic 확인하기

- /camera/camera/depth/image_rect_raw 토픽을 선택하여 depth값 확인



Realsense-RViz2

▶ rviz2를 이용하여 realsense topic 확인하기

- rqt와 realsense launch파일 종료 후 새로운 launch 파일 실행

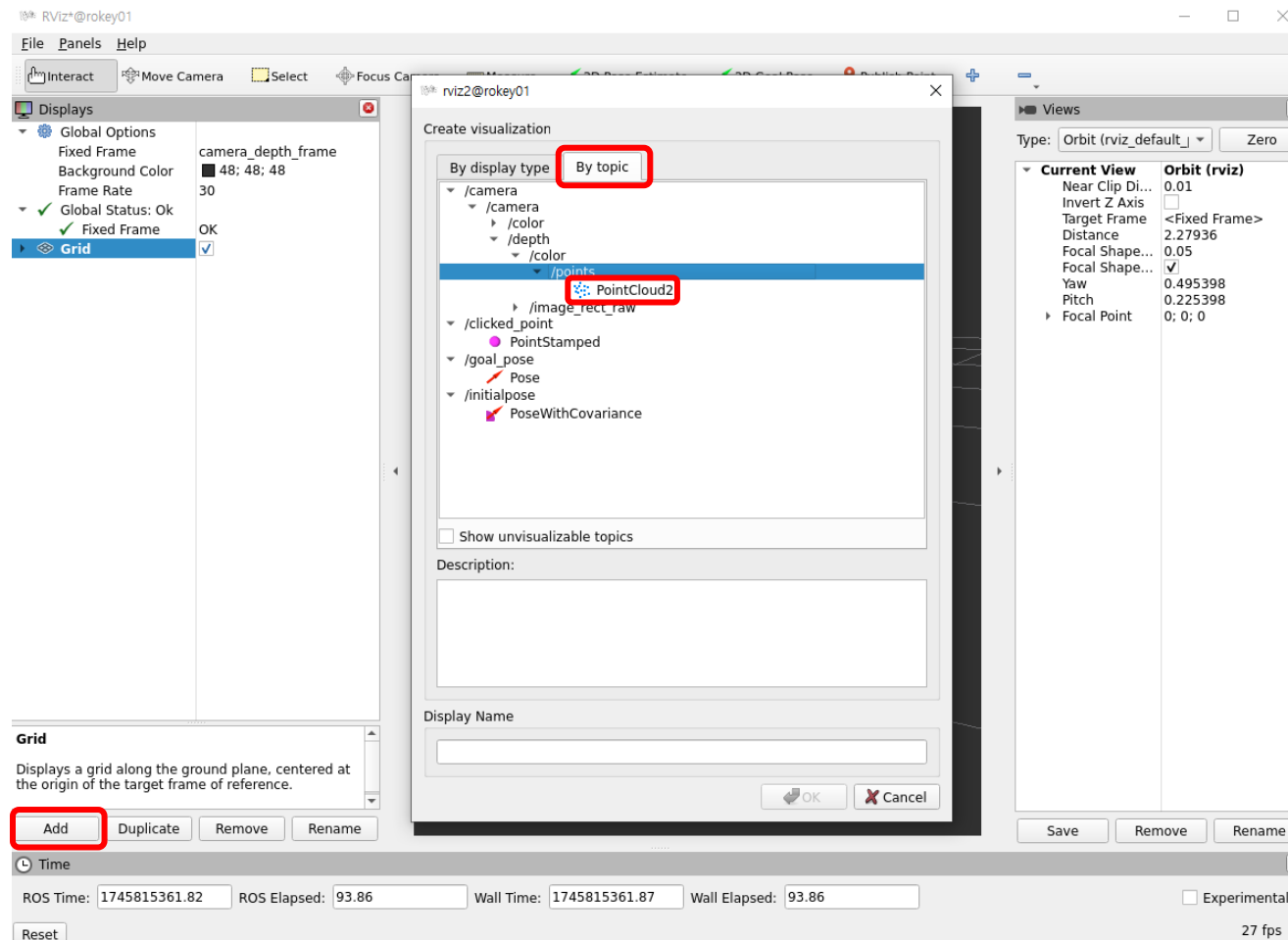
```
ros2 launch realsense2_camera rs_launch.py depth_module.depth_profile:=1280×720×30 pointcloud.enable:=true
```

- rviz2 실행

```
rviz2
```

▶ rviz2를 이용하여 realsense topic 확인하기

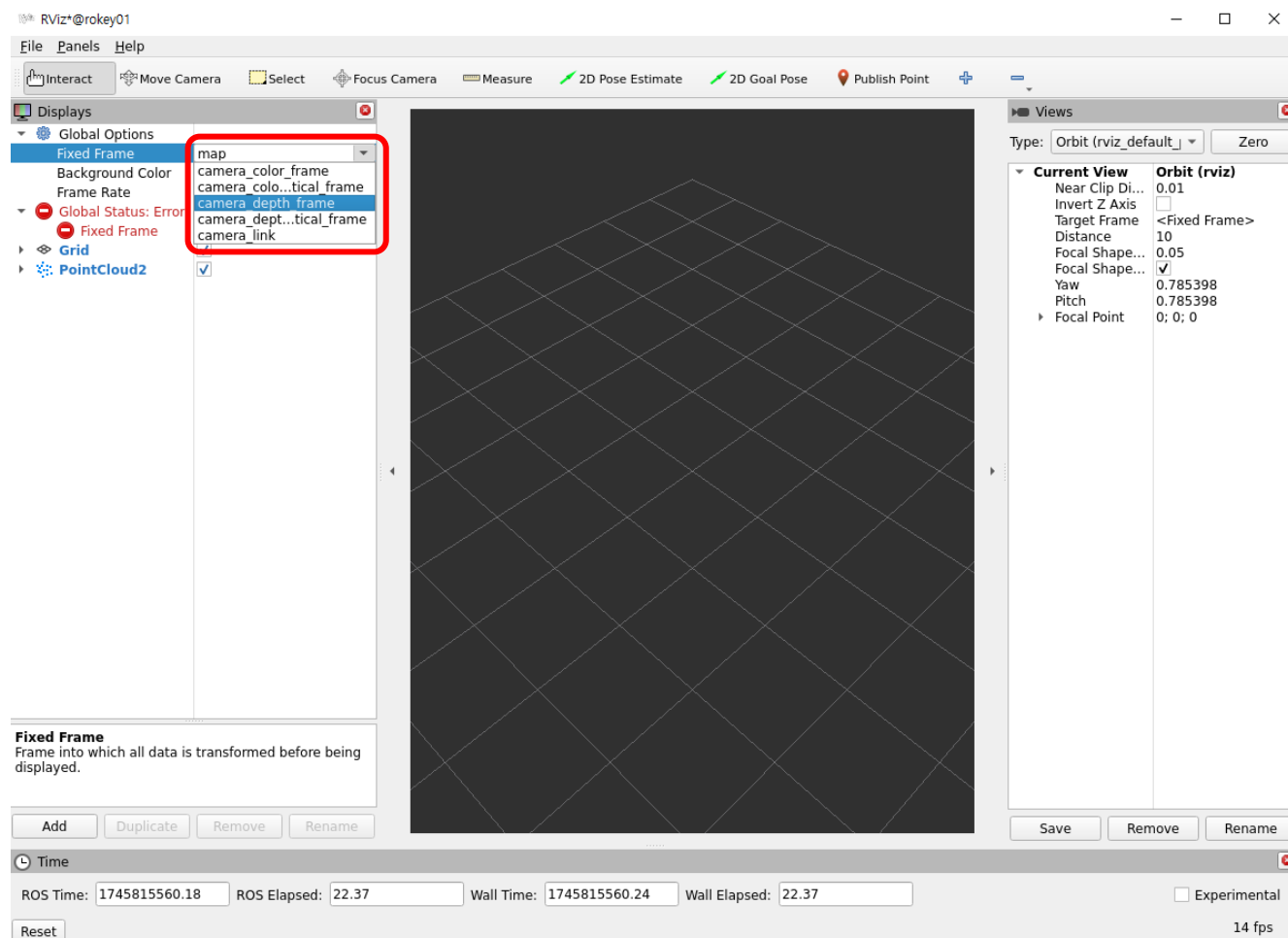
- rviz2에서 Add 버튼을 누른 후 By topic에서 camera→ camera→ depth → color → points → PointCloud2 선택



※ 오류 발생시 rviz2 재실행해보기

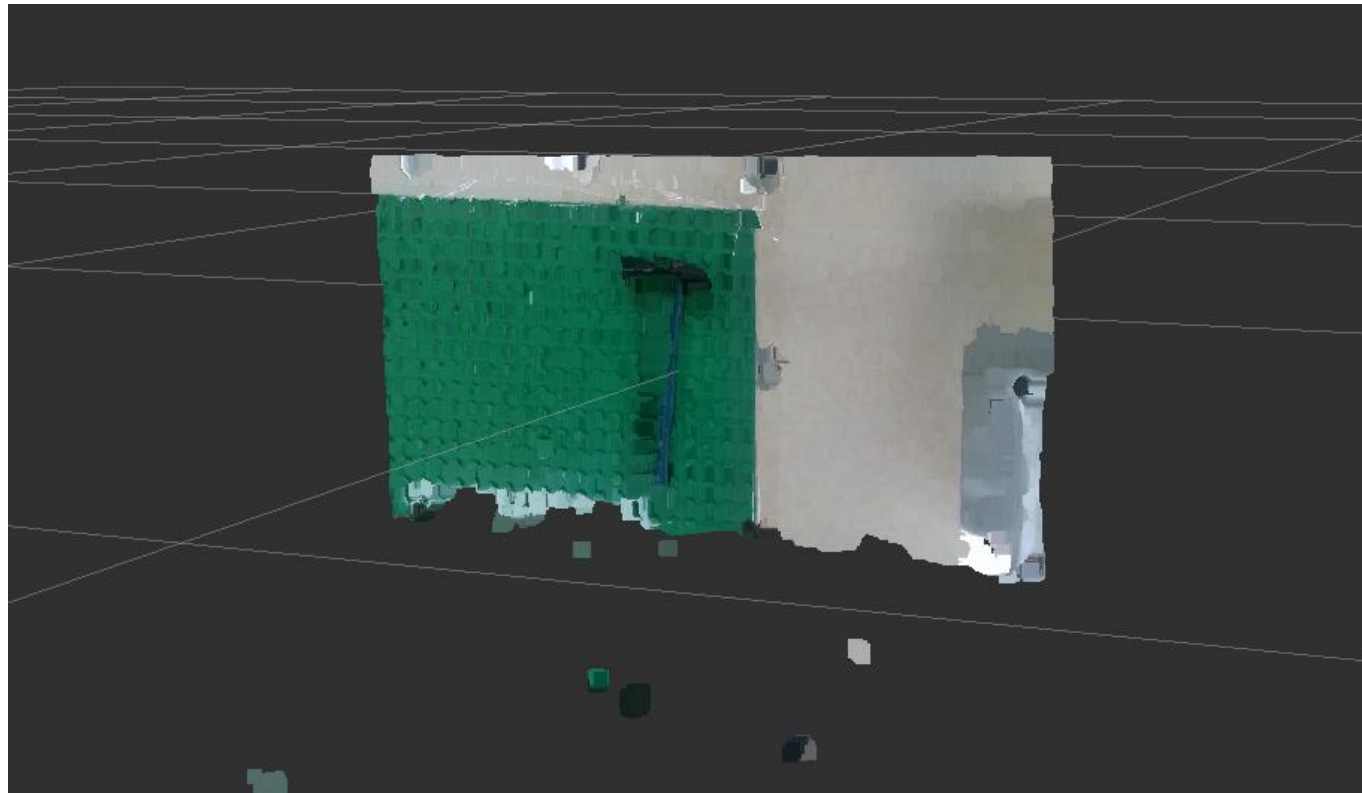
▶ rviz2를 이용하여 realsense topic 확인하기

- Fixed frame에서 camera_depth_frame 선택



▶ rviz2를 이용하여 realsense topic 확인하기

- Point cloud 확인 가능



The screenshot displays the Visual Studio Code (VS Code) interface. On the left, the Explorer sidebar shows a project structure with a red box highlighting the 'TUTORIAL' directory. This directory contains a subfolder 'Calibration_Tutorial' and several files: 'data_recording.py', 'eye2hand_calibration.py', 'handeye_calibration.py', 'modbus.ipynb' (selected), 'onrobot.py', 'realsense.py', 'T_gripper2camera.npy', and 'test.py'. Below this are other tutorial folders like 'DRL_Tutorial', 'OD_Tutorial', 'ROS_Tutorial', and 'VoiceProcessing'. The main editor area on the right shows the 'modbus.ipynb' Jupyter notebook. It contains two code cells. The first cell imports 'time' and 'RG' from 'onrobot', and defines constants for 'GRIPPER_NAME', 'TOOLCHARGER_IP', and 'TOOLCHARGER_PORT'. The second cell creates a 'gripper' object using these constants. The output of the second cell shows the gripper's status: 'Start opening gripper.', 'Start closing gripper.', and 'Start moving gripper.'. The bottom status bar indicates the user is 'rokey@rokey-Katana-17-B13VFK' in the '~/Tutorial' directory.

File Edit Selection View Go Run Terminal Help

EXPLORER

- TUTORIAL
 - Calibration_Tutorial
 - __pycache__
 - data
 - data_recording.py
 - eye2hand_calibration.py
 - handeye_calibration.py
 - modbus.ipynb
 - onrobot.py
 - realsense.py
 - T_gripper2camera.npy
 - test.py
 - DRL_Tutorial
 - force_control.png
 - movej.png
 - move1.png
 - movePeriodic.png
 - rg2.jpeg
 - rokey_study.ipynb
 - Fruits
 - OD_Tutorial
 - YOLO
 - YOLO_SIMPLE
 - modifyingsetting.ipynb
 - ultralytics.ipynb
 - ROS_Tutorial
 - Tools
 - VoiceProcessing
 - build
 - install
 - log
 - .env
 - class_embeddings.json
 - hello_rokey_8332_32.tflite
 - keyword_extraction.py
 - mic_test.py
 - MicController.py
 - STT.py
 - wakeup_word.py
 - yolo11n.pt

modbus.ipynb

Calibration_Tutorial > modbus.ipynb > import time

Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline

```
import time
from onrobot import RG

GRIPPER_NAME = "rg2"
TOOLCHARGER_IP = "192.168.1.1"
TOOLCHARGER_PORT = "502"

gripper = RG(GRIPPER_NAME, TOOLCHARGER_IP, TOOLCHARGER_PORT)
```

[1] ✓ 0.0s

```
gripper.open_gripper()
gripper.close_gripper()
gripper.move_gripper(500)
```

[2] ✓ 0.0s

... Start opening gripper.
Start closing gripper.
Start moving gripper.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER

rokey@rokey-Katana-17-B13VFK:~/Tutorial\$

RG2 Gripper

▶ 제품 설명

RG2 Gripper는 협동 로봇용으로 설계된 전기식 2-Finger Gripper로 다양한 작업 환경에서 유연성과 정밀성 제공하며 특히, 자동화된 생산 라인에서의 Pick & Place, 조립, 머신 텐딩 등의 작업에 많이 사용

▶ 제품 스펙

- Payload Force Fit : 2Kg
- Payload Form Fit : 5Kg
- Total stroke: 0~110mm
- Gripping force: 40N



▶ Modbus란?

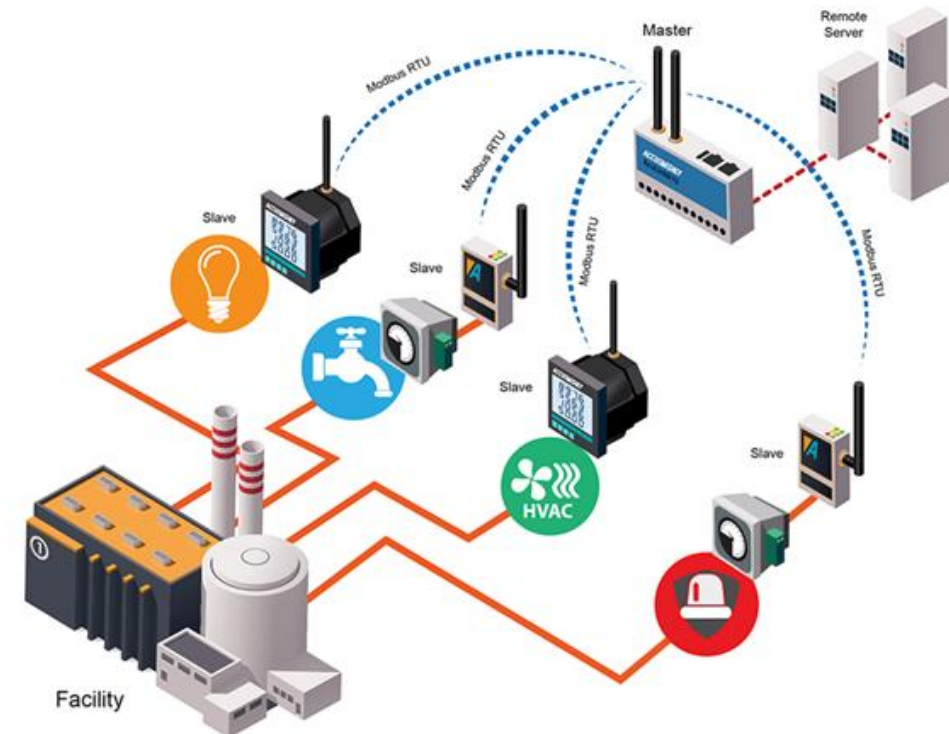
Modbus는 1979년, 지금은 Schneider Electric인 모디콘(Modicon)이라는 회사에서 만든 시리얼 통신 프로토콜이다.

제조공장 기계나 설비를 자동화하고 제어하는 목적으로 사용되는 프로그래머블 로직 컨트롤러(Programmable Logic Controller)들과의 통신에 사용할 목적으로 만들어졌다.

프로토콜이 단순하지만, 장비 제어와 모니터링에 필요한 기능들을 수행할 수 있기에 사실상의 표준 프로토콜의 지위를 얻게 되었고, 현재까지 산업용 전자 장치들을 서로 연결하는 목적으로 널리 사용된다.

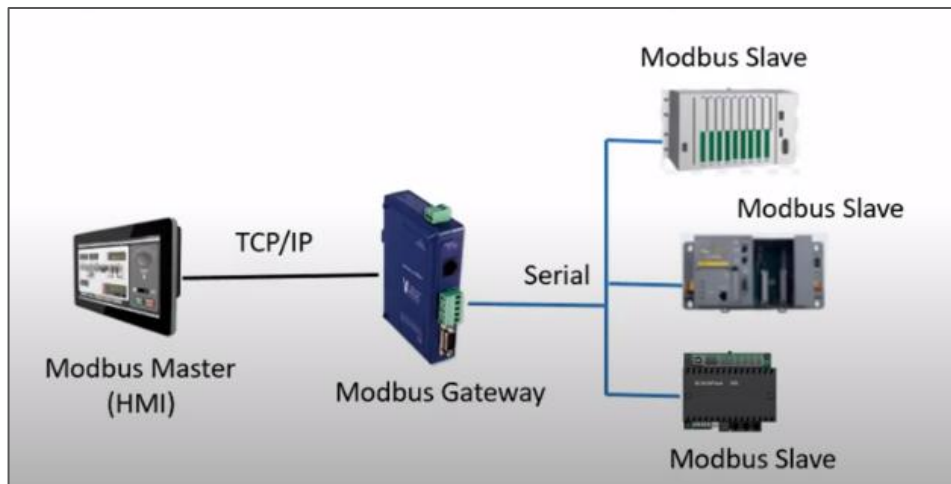
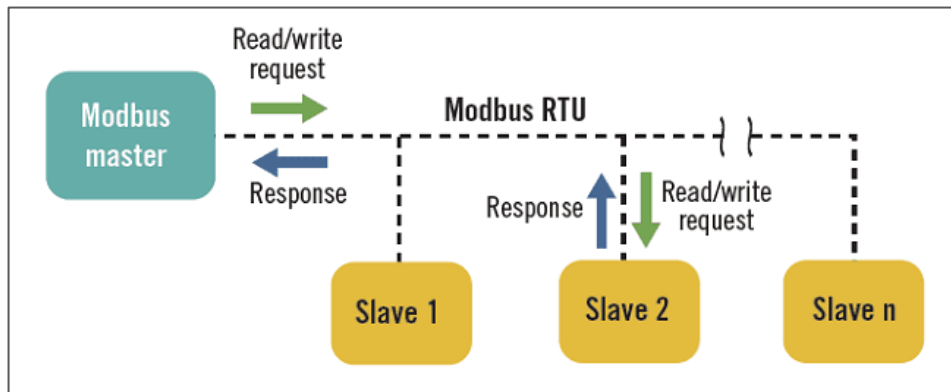
다음은 모드버스가 산업용으로 널리 사용되는 이유들이다.

- 산업용 통신 프로토콜로 개발됨
- Master와 Slave구조
- 프로토콜이 공개되어 있고 무료
- 설치와 유지보수가 용이
- 비트단위 또는 워드(16비트) 단위로 정보조작 용이



▶ Modbus 통신 Packet

- Modbus Master가 특정 Slave에게 요청을 보내면 해당 Slave만 응답하는 방식
- 패킷구조 : 주소(Slave) + 기능코드(읽기/쓰기) + 데이터(레지스터 주소 및 개수) + 오류 검증 코드(CRC)



Function Code 03	Register Type: Holding				Register Range: 40001 to 49999
Logical Address	40001	40002	40003	40004	40005
Read Address (Offset)	0000	0001	0002	0003	0004
When inputting 40005, Modbus interprets 40001+0004, reading the register at address 0004					

▶ Modbus 통신 Protocol

- Modbus RTU, Modbus ASCII, Modbus TCP/IP

항목	Modbus RTU	Modbus ASCII	Modbus TCP/IP
전송방식	Serial 통신(RS-232, RS-485)	Serial 통신	Ethernet(TCP/IP)
Frame 형식	Binary	ASCII Text	TCP/IP Packet
속도	빠름(압축된 이진 형태)	느림	매우 빠름(LAN기준)
응용분야	공장 자동화, 실시간 제어	테스트 환경, 디버깅, 초기 개발	원격 모니터링, SCADA
거리	수백 미터	←	거리제한 없음(네트워크에 따라)
전송속도	9600bps ~ 115200bps	←	1Gbps(네트워크에 따라 다름)
배선	별도 시리얼 케이블	←	기존 Ethernet 인프라 활용

▶ Modbus 예제

- Tutorial 폴더안에서 vscode 열기
- Modbus_Tutorial 폴더에 onrobot.py 파일을 생성 후 다음 링크에서 코드를 복사하여 붙여넣기
- <https://github.com/takuya-ki/onrobot-rg/blob/main/src/onrobot.py>



onrobot-rg / src / onrobot.py

takuya-ki fix a bug

Code Blame 185 lines (165 loc) · 7.3 KB Code 55% faster with GitHub Copilot

```
1  #!/usr/bin/env python3
2
3  from pymodbus.client.sync import ModbusTcpClient as ModbusClient
4
```

```
6  class RG():
7
8  def __init__(self, gripper, ip, port):
9      self.client = ModbusClient(
10         ip,
11         port=port,
12         stopbits=1,
13         bytesize=8,
14         parity='E',
15         baudrate=115200,
16         timeout=1)
17
18     if gripper not in ['rg2', 'rg6']:
19         print("Please specify either rg2 or rg6.")
20         return
21
22     self.gripper = gripper # RG2/6
23     if self.gripper == 'rg2':
24         self.max_width = 1100
25         self.max_force = 400
26     elif self.gripper == 'rg6':
27         self.max_width = 1600
28         self.max_force = 1200
29     self.open_connection()
```

RG2 Gripper-Modbus 예제

▶ Modbus 예제

- modbus.ipynb 파일 생성
- 다음 코드 작성 후 실행(shift + enter)

```
#그리퍼 모듈 불러오기
import time
from onrobot import RG
#그리퍼 관련 설정
GRIPPER_NAME = "rg2"
TOOLCHARGER_IP = "192.168.1.1"
TOOLCHARGER_PORT = "502"
#그리퍼 객체 생성
gripper = RG(GRIPPER_NAME, TOOLCHARGER_IP, TOOLCHARGER_PORT)
```

※ 오류 발생시 다음 항목 확인

- 노트북에 IP가 제대로 할당되어 있는지 확인
- pymodbus version확인 후 3.x version인 경우 uninstall후 pip3 install pymodbus==2.5.8로 설치해서 작동되는지 확인

RG2 Gripper-Modbus 예제

▶ Modbus 예제

- 다음 코드 작성 후 실행(shift + enter)하여 그리퍼가 열리는지 확인

```
#그리퍼 열기  
gripper.open_gripper()
```

- 다음 코드 작성 후 실행(shift + enter)하여 그리퍼가 닫히는지 확인

```
#그리퍼 닫기  
gripper.close_gripper()
```

- 다음 코드 작성 후 실행(shift + enter)하여 그리퍼가 움직이는지 확인(단위:1/10mm)

```
gripper.move_gripper(30)
```

RG2 Gripper-Modbus 예제

▶ Modbus 예제

- 다음 코드 작성 후 실행(shift + enter)하여 그리퍼의 상태 확인

```
gripper.get_status()
```

비트 번호	상태 플래그	설명
0	Busy	1: 동작 중, 0: 대기 중 (새 명령 가능)
1	Grip Detected	1: 물체 감지됨, 0: 감지되지 않음
2	S1 Pushed	1: 안전 스위치 1 눌림
3	S1 Triggered	1: 안전 회로 1 활성화됨 (재부팅 필요)
4	S2 Pushed	1: 안전 스위치 2 눌림
5	S2 Triggered	1: 안전 회로 2 활성화됨 (재부팅 필요)
6	Safety Error	1: 전원 인가 시 안전 스위치 눌러 있음

RG2 Gripper-Modbus 예제

▶ Modbus 예제

- 다음 코드 작성 후 실행(shift + enter)하여 그리퍼의 상태 확인

```
#그리퍼 열기
gripper.open_gripper()
#그리퍼 닫기
gripper.close_gripper()
#그리퍼의 상태를 체크하고 동작 중이면 wait(동작이 끝날 때까지 기다리기)
while gripper.get_status()[0]:
    time.sleep(0.5)
```

▶ Camera Calibration이란?

Camera Calibration은 카메라의 내·외부 파라미터를 추정하는 것이다.

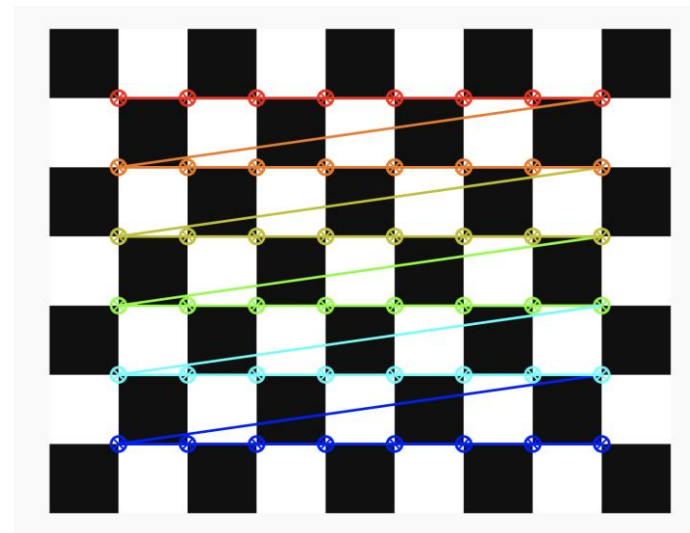
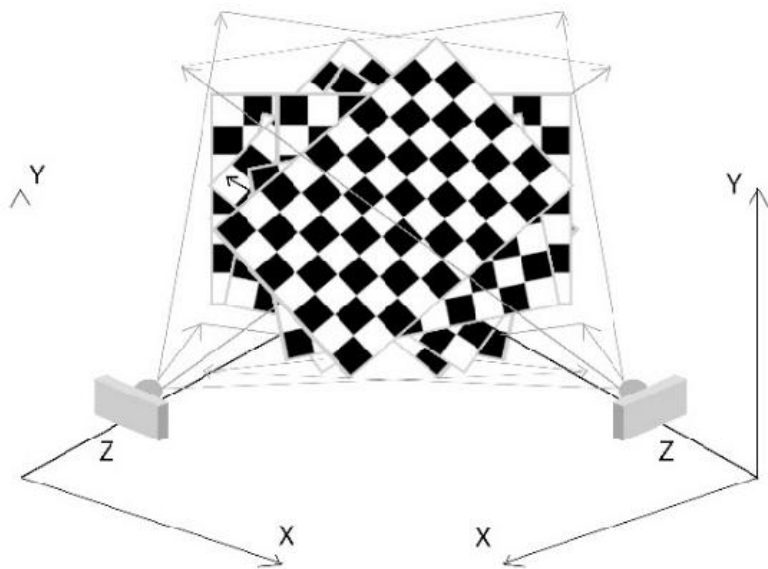
내부 파라미터는 카메라의 초점 거리, 주점, 렌즈 왜곡 계수 등이 있으며 카메라에 의한 왜곡을 보정할 수 있다.

외부 파라미터는 3D 공간 좌표를 정확히 매핑할 때 사용할 수 있다.

내부 파라미터는 체커보드를 이용하여 추정할 수 있지만 보통 카메라의 제조사가 제공하는 경우가 많다.

외부 파라미터는 카메라를 로봇 팔에 부착하느냐 로봇 팔과 독립적인 곳에 위치하는지에 따라,

Eye-in-Hand Calibration과 Eye-to-Hand Calibration으로 나뉜다.

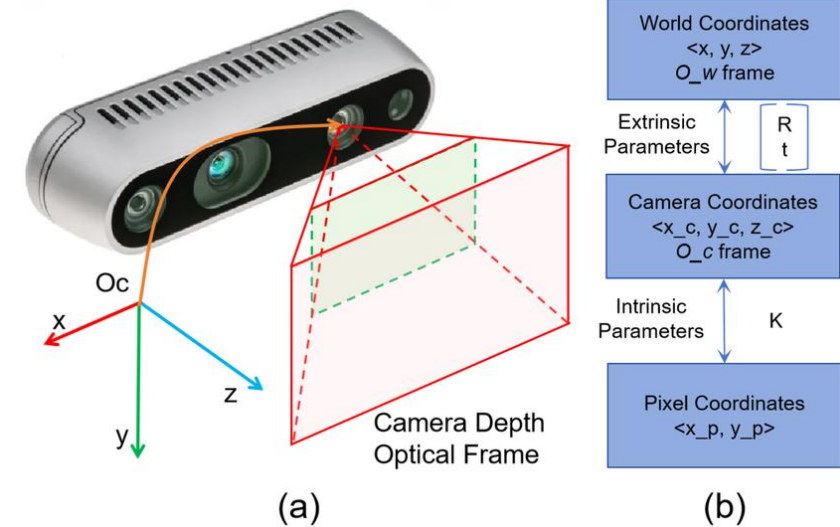


▶ Camera Calibration – 외부 파라미터

- 우리가 알아야 할 외부 파라미터는 로봇 좌표계 기준으로 카메라가 어디에 위치해 있는지!
- 이를 위해서는 다음과 같은 chain-rule을 사용하게 된다.

$$T_{base}^{gripper} \cdot T_{gripper}^{camera} \cdot T_{camera}^{target} = T_{base}^{target}$$

- | | | |
|---|------------------------|--|
| { | $T_{base}^{gripper}$ | Base → Gripper 변환, 알고 있는 정보 |
| | $T_{gripper}^{camera}$ | Gripper → Camera 변환, 알아야 하는 정보 |
| | T_{camera}^{target} | Camera → Target 변환, 알고 있는 정보(체커보드) |
| | T_{base}^{target} | Base → Target 변환, 모르지만 항상 고정되어 있는 정보(상수취급) |

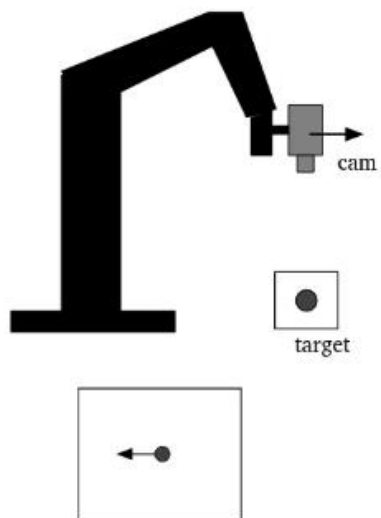


- 이때 Base → Target 변환이 항상 같다는 것을 이용하여 python코드로 방정식을 풀 수 있다.

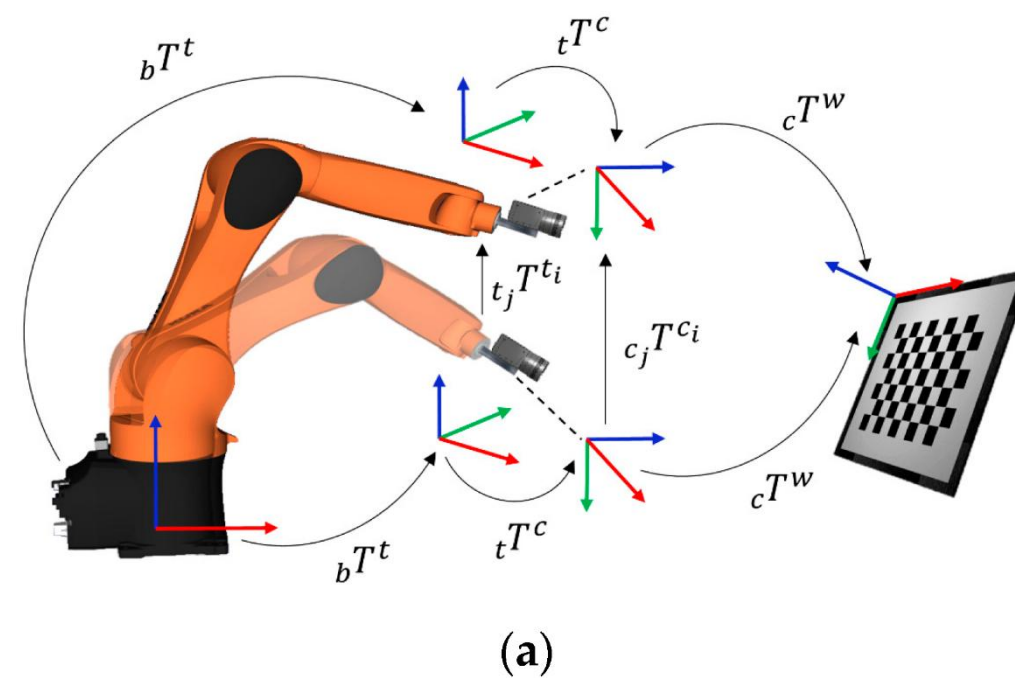
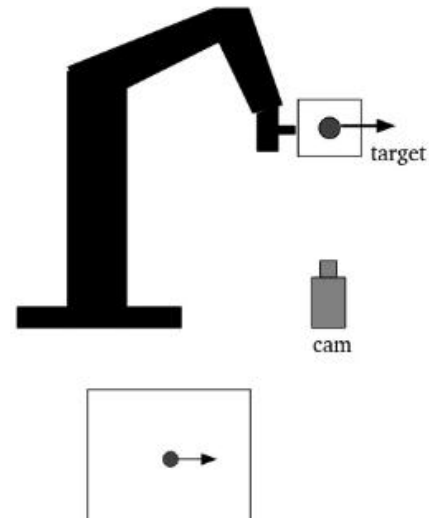
▶ Camera Calibration – data 수집

- 우리가 사용할 방법은 Eye-in-Hand 방식이다.

Eye-in-Hand system



Eye-to-Hand system



▶ Camera Calibration – data 수집

- 우리가 사용할 방법은 Eye-in-Hand 방식

Eye-in-Hand	Eye-to-Hand
로봇 팔에 카메라 부착	로봇 팔에 Checker Board 부착
Checker Board를 고정된 공간에 위치시킴	카메라를 고정된 공간에 위치시킴
로봇팔을 움직여가며 체커보드 촬영	
Checker Board를 촬영할 때 TCP도 같이 기록해야 함	

▶ Camera Calibration – data 수집

- 다음 링크에서 checker-board 파일을 다운로드하고 프린트한다.
- <https://raw.githubusercontent.com/MarkHedleyJones/markhedleyjones.github.io/master/media/calibration-checkerboard-collection/Checkerboard-A4-25mm-10x7.pdf>
- 해당 Checker Board를 두산 로봇 베이스(책상)에 고정
- 다음 명령어를 이용해 로봇 bring up 연결하기

```
cd cobot_ws
source install/setup.bash
ros2 launch dsr_bringup2 dsr_bringup2_rviz.launch.py mode:=real host:=192.168.1.100 port:=12345 model:=m0609
```

Camera Calibration

▶ Camera Calibration – data 수집

- 데이터 수집을 위해 로봇 모드를 직접 교시로 전환(새로운 터미널에서 진행)

```
cd cobot_ws  
source install/setup.bash  
ros2 service call /dsr01/system/set_robot_mode dsr_msgs2/srv/SetRobotMode "robot_mode: 0"
```

- 로봇이 수동모드로 바뀌었는지 확인(**BLUE Color LED**)

▶ Camera Calibration – data 수집

- 새로운 터미널에서 CalibrationTutorial 폴더에 있는 data_recording.py 파일 실행
- **DEVICE_NUMBER = 4(이 숫자를 바꿔서 RGB 이미지가 나오도록 수정)**
- data_recording.py는 이전에 받은 Tutorial폴더 안에 Calibration_Tutorial 폴더에 있음

```
source ~/cobot_ws/install/setup.bash
export PYTHONPATH=$PYTHONPATH:~/cobot_ws/install/dsr_common2/lib/dsr_common2/imp
python3 data_recording.py
```

- RGB카메라 영상이 표시되는 실행 화면에서 q를 누르면 현재 찍고 있는 사진과 좌표 값이 저장됨
- 직접 교시로 위치를 바꾸어 가며 데이터 수집(20장 이상 수집)
- **data_recording.py 파일 실행 시 다른 터미널에서 realsense node가 실행되어 있으면 동작안됨**
→ **Realsense node 종료하고 data_recording.py 다시 실행**
- **※ Checker Board 사진은 Color로 진행!!!**

▶ DEVICE_NUMBER 확인

```
rokey@rokey:~$  
rokey@rokey:~$ v4l2-ctl --list-devices  
Intel(R) RealSense(TM) Depth Ca (usb-0000:00:14.0-2):  
    /dev/video2  
    /dev/video3  
    /dev/video4  
    /dev/video5  
    /dev/video6  
    /dev/video7  
    /dev/media1  
    /dev/media2  
  
HD Webcam: HD Webcam (usb-0000:00:14.0-6):  
    /dev/video0  
    /dev/video1  
    /dev/media0  
  
rokey@rokey:~$
```

```
rokey@rokey:~$  
rokey@rokey:~$ v4l2-ctl -d /dev/video0 --get-fmt-video  
Format Video Capture:  
    Width/Height      : 1280/720  
    Pixel Format       : 'MJPG' (Motion-JPEG)  
    Field              : None  
    Bytes per Line     : 0  
    Size Image         : 1843200  
    Colorspace         : sRGB  
    Transfer Function  : Rec. 709  
    YCbCr/HSV Encoding: ITU-R 601  
    Quantization       : Default (maps to Full Range)  
    Flags              :  
  
rokey@rokey:~$
```

▶ Camera Calibration – data 수집

- 데이터 수집 완료 시 아래와 같은 디렉토리 구조가 되어야 함

```
#최종 예상 디렉토리
Calibration_Tutorial
├── data
│   ├── ...jpg
│   ├── ...jpg
│   ├── ...jpg
│   └── calibrate_data.json
├── handeye_calibration.py
├── data_recording.py
├── realsense.py
├── test.py
└── eye2hand_calibration.py3
```

▶ Camera Calibration – data 수집

- calibrate_data.json 파일 예시

```
calibrate_data.json
~/Desktop/data

1 {
2   "file_name": [
3     "00000001.jpg",
4     "00000002.jpg",
5     "...
6     "00000024.jpg",
7     "00000025.jpg"
8   ],
9   "pos": [
10    [
11      539.47,
12      -122.985,
13      408.414,
14      89.168,
15      -179.998,
16      89.148
17    ],
18    "...
19    [
20      285.861,
21      -108.523,
22      331.677,
23      0.017,
24      160.055,
25      0.0
26    ]
27  ],
28  "type": [
29    "translation",
30    "translation",
31    "translation",
32    "translation",
33    "...

```

▶ Camera Calibration – data 수집

- handeye_calibration.py 파일 확인하기(checkerboard_size와 square_size가 제대로 되어있는지 확인)
 - checkerboard_size = (7,10) 또는 (10,7), square_size = 25

```
import cv2
import numpy as np
import json
from scipy.spatial.transform import Rotation

# 1) 로봇 그리퍼의 절대 좌표 (x, y, z, rx, ry, rz)를 행렬로 변환하는 함수
def get_robot_pose_matrix(x, y, z, rx, ry, rz):...

# 2) 체커보드 코너 검출 (카메라+체커보드 변환 구하기)
def find_checkerboard_pose(...

def calibrate_camera_from_chessboard(...

# Main Function
if __name__ == "__main__":
    # 캘리브레이션 데이터 로드
    data = json.load(open("data/calibrate_data.json"))
    robot_poses = np.array(data["poses"])

    robot_poses[:, :3] = robot_poses[:, :3]
    image_paths = ["data/" + d for d in data["file_name"]]

    checkerboard_size = (8, 6) # 내부 코너 개수
    square_size = 25
    # 카메라 캘리브레이션 수행(내부 파라미터 왜곡 보정)
    camera_matrix, dist_coeffs, rvecs, tvecs = calibrate_camera_from_chessboard(
        image_paths, checkerboard_size, square_size
```

▶ Camera Calibration – data 수집

- handeye_calibration.py 파일 실행하기(실행 후 T_gripper2camera.npy 파일이 만들어졌는지 체크하기)

```
python3 handeye_calibration.py
```

- handeye_calibration.py 파일안의 cv2.calibrateHandEye함수 내에 method를 바꾸어 가며 실험해보기
 - 0, 1, 2, 3, 4 등의 숫자로 바꿀 수 있음

```
R_cam2gripper, t_cam2gripper = cv2.calibrateHandEye(  
    R_gripper2base_list,  
    t_gripper2base_list,  
    R_checker2camera_list,  
    t_checker2camera_list,  
    method=cv2.CALIB_HAND_EYE_PARK,  
)
```



▶ Camera Calibration – 테스트

- 로봇 연결

```
cd cobot_ws
source install/setup.bash
ros2 launch dsr_bringup2 dsr_bringup2_rviz.launch.py mode:=real host:=192.168.1.100 port:=12345 model:=m0609
```

- Realsense 런치 파일 실행(줄바꿈 없음)

```
ros2 launch realsense2_camera rs_align_depth_launch.py depth_module.depth_profile:=640×480×30
rgb_camera.color_profile:=640×480×30 initial_reset:=true align_depth.enable:=true
enable_rgbd:=true pointcloud.enable:=true
```

- test.py 파일 실행하기

```
export PYTHONPATH=$PYTHONPATH:~/cobot_ws/install/dsr_common2/lib/dsr_common2/imp
source ~/cobot_ws/install/setup.bash
python3 test.py
```

▶ Camera Calibration – 테스트

- TODO : 코드 분석하여 로봇 움직여 보기



※ 조별 Domain ID 설정

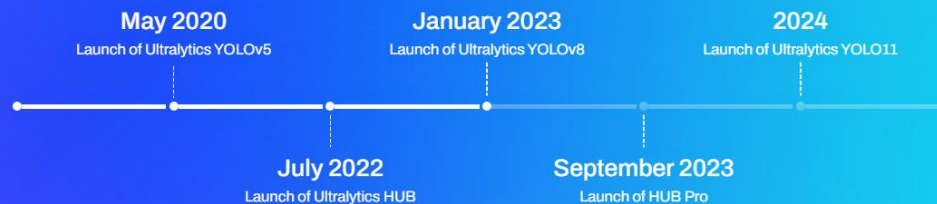
▶ Ultralytics란?

- Computer Vision 분야에서 강력한 딥러닝 기반 Object Detection 모델을 개발하고 배포하는 기업이 운영하는 웹사이트
- 주로 YOLO(You Only Look Once) 계열의 모델(YOLOv5, YOLOv8, YOLOv11)과 관련된 도구, 문서, 서비스 등을 제공
- 단순히 모델만 제공하는 것이 아니라, 모델 학습, 추론, 배포를 쉽게 할 수 있는 소프트웨어 플랫폼과 툴킷을 함께 제공한다.



Simpler. Smarter. Further.

The Ultralytics story: past present, and future



▶ Ultralytics Settings

- Inspecting Settings
- Ultralytics.ipynb 파일 생성하여 아래 코드문 작성 후 실행

```
from ultralytics import settings  
  
# View all settings  
print(settings)
```

▶ Ultralytics Settings

- settings_version : Ultralytics 설정 버전
- datasets_dir : Dataset이 저장된 경로
- weights_dir : Model weights이 저장된 경로
- runs_dir : 실험 실행 기록이 저장되는 경로
- Settings값에 대한 더 자세한 정보는 다음 링크 참조
- <https://docs.ultralytics.com/quickstart/#understanding-settings>



ultralytics

Ultralytics YOLO 문서

```
JSONDict("/home/rokey/.config/Ultralytics/settings.json"):
{
  "settings_version": "0.0.6",
  "datasets_dir": "/path/to/datasets",
  "weights_dir": "weights",
  "runs_dir": "runs",
  "uuid": "90216b1751b87aa8453cf4585822f79cc7acfd3e7f53f54c7debad2919f02f8b",
  "sync": true,
  "api_key": "",
  "openai_api_key": "",
  "clearml": true,
  "comet": true,
  "dvc": true,
  "hub": true,
  "mlflow": true,
  "neptune": true,
  "raytune": true,
  "tensorboard": false,
  "wandb": false,
  "vscode_msg": true,
  "openvino_msg": true
}
```

▶ Ultralytics Settings

- Modifying Settings (ipynb 파일 새로 생성 후 아래 코드 작성하여 실행)

```
from ultralytics import settings

settings.update({"datasets_dir": "/path/to/datasets"})

# Check if the setting was updated    ← 개인 작업환경에 맞게 경로 설정하기
print(settings["datasets_dir"])
```

- 실행 결과

```
/path/to/datasets
```

- Reset Settings

```
settings.reset()
```

▶ Ultralytics Predict

- 초기 세팅(echo 문의 경우 한 번만 수행하면 됨)

```
echo 'export PATH=$PATH:~/.local/bin' >> ~/.bashrc  
source ~/.bashrc
```

▶ Ultralytics Predict

- URL로 제공된 이미지 predict

```
yolo detect predict model=yolo11n.pt source='https://ultralytics.com/images/bus.jpg'  
xdg-open runs/detect/predict/bus.jpg
```



← /home/runs/detect/predict/bus.jpg 이미지 생성되었는지 확인

▶ Ultralytics Predict

- URL로 제공된 이미지 predict

```
yolo detect predict model=yolo11n.pt source='https://ultralytics.com/images/bus.jpg'  
xdg-open runs/detect/predict/bus.jpg
```

- model argument
 - Yolo11n.pt: 사전 학습된 YOLOv11 모델 사용
 - 커스텀으로 학습한 모델 pt 파일은 절대경로 기준으로 해당 디렉토리를 입력

▶ Ultralytics Predict

- 실시간 웹캠 predict

```
yolo detect predict model=yolo11n.pt source=0 show=True
```

- model
 - Yolo11n.pt: 사전 학습된 YOLOv11 모델 사용
 - 커스텀으로 학습한 모델 pt 파일은 절대경로 기준으로 해당 디렉토리를 입력
- source 번호에 따라 내장 cam, depth camera 또는 webcam이 설정됨

Fruits and Tools in Tutorial Directory

The screenshot shows the VS Code interface with the Tutorial directory open in the Explorer. The directory structure is as follows:

- TUTORIAL
 - Calibration_Tutorial
 - DRL_Tutorial
 - Fruits
 - test
 - train
 - train_20251126_170132
 - val_20251126_170132
 - valid
 - best.pt
 - data.yaml
 - Fruits.v1i.yolov11.zip
 - README.dataset.txt
 - README.roboflow.txt
 - yolo_predict_camera.py
 - yolo_predict_imgs.py
 - yolo_train.py
 - yolov8n.pt
 - OD_Tutorial
 - ROS_Tutorial
 - Tools
 - test
 - train
 - train_20251126_165946
 - train_20251126_180341
 - train_20251126_180438
 - val_20251126_180438
 - valid
 - best.pt
 - data.yaml
 - README.dataset.txt
 - README.roboflow.txt
 - tool.v2i.yolov11.zip
 - yolo_predict_camera.py
 - yolo_predict_imgs.py
 - yolo_train.py
 - yolov8n.pt
 - VoiceProcessing
 - yolo11n.pt

The main editor shows the `yolo_predict_camera.py` script. The script is a Python program that uses the `ultralytics` library to load a YOLO model and predict objects from a video feed. It includes comments in Korean explaining the steps.

```

1 from ultralytics import YOLO
2 import cv2
3 import os
4
5 project_folder = "Tools"
6 model_filename = "best.pt" # train 디렉토리에 있는 best.pt를 project_folder로 복사
7
8 # 모델 로드
9 # 자동으로 현재 폴더 기준 경로 설정
10 BASE_DIR = os.getcwd() # 현재 폴더 기준
11 OUTPUT_DIR = os.path.join(BASE_DIR, project_folder) # 원하는 상위 폴더 이름
12
13 model_path = os.path.join(OUTPUT_DIR, model_filename)
14 model = YOLO(model_path)
15
16 # 카메라 열기 (0: 기본 내장 카메라)
17 cap = cv2.VideoCapture(0)
18
19 if not cap.isOpened():
20     print("카메라를 열 수 없습니다.")
21     exit()
22
23 print("실시간 예측 시작 (종료: Q 키 누르기)")
24
25 while True:
26     ret, frame = cap.read()
27     if not ret:
28         break
  
```

The terminal shows the execution of the script. It runs the `train.py` script, which outputs performance metrics for the YOLO model. The output includes a table of results for various classes and a summary of the training process.

```

rokey@rokey-Katana-17-B13VFK:~/Tutorial$ /bin/python3 /home/rokey/Tutorial/Tools/yolo train.py
val: Fast image access (ping: 0.0±0.0 ms, read: 3585.1±1034.2 MB/s, size: 30.4 KB)
val: Scanning /home/rokey/Tutorial/Tools/valid/labels.cache... 9 images, 0 backgrounds, 0 corrupt: 100% 9/9 331.1Kit/s 0.0s
  
```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)
all	9	9	0.898	0.913	0.995	0.881
hammer	2	2	0.904	1	0.995	0.895
knife	2	2	0.909	1	0.995	0.895
pliers	2	2	0.798	1	0.995	0.92
screw driver	1	1	0.877	1	0.995	0.895
wrench	2	2	1	0.566	0.995	0.796

Speed: 0.3ms preprocess, 6.0ms inference, 0.0ms loss, 0.5ms postprocess per image
 Results saved to /home/rokey/Tutorial/Tools/train_20251126_180438
 Train 결과: /home/rokey/Tutorial/Tools/train_20251126_180438
 Val 결과: /home/rokey/Tutorial/Tools/val_20251126_180438

The terminal also shows the execution of the `yolo_predict_camera.py` script, which outputs the same performance metrics as the `train.py` script.

```

rokey@rokey-Katana-17-B13VFK:~/Tutorial$ /bin/python3 /home/rokey/Tutorial/Tools/yolo_predict_camera.py
실시간 예측 시작 (종료: Q 키 누르기)
^CTraceback (most recent call last):
  File "/home/rokey/Tutorial/Tools/yolo_predict_camera.py", line 26, in <module>
    ret, frame = cap.read()
KeyboardInterrupt
rokey@rokey-Katana-17-B13VFK:~/Tutorial$
  
```

Fruits and Tools Directory 구조

Fruits and Tools in Tutorial Directory

Visual Studio Code interface showing the tutorial directory structure and the training process.

EXPLORER

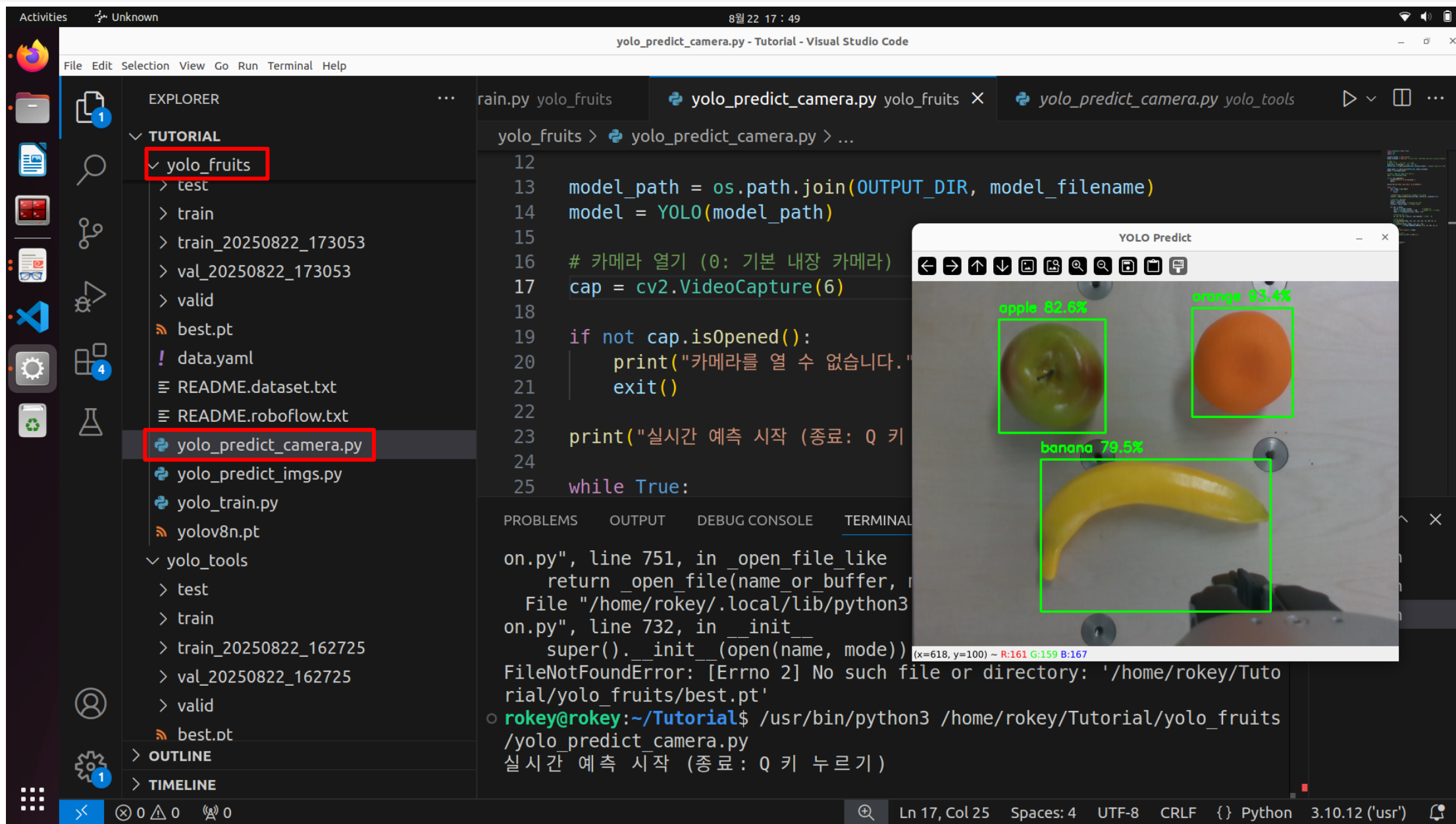
- TUTORIAL
 - yolo_fruits
 - test
 - train
 - train_20250822_173053
 - val_20250822_173053
 - valid
 - best.pt
 - ! data.yaml
 - README.dataset.txt
 - README.roboflow.txt
 - yolo_predict_camera.py
 - yolo_predict_imgs.py
 - yolo_train.py
 - yolov8n.pt
 - yolo_tools
 - test
 - train
 - train_20250822_162725
 - train_20250822_175125
 - val_20250822_162725
 - valid
 - best.pt
 - ! data.yaml
 - yolo_predict_camera.py
 - yolo_predict_imgs.py
 - yolo_train.py

TERMINAL

Training...

	all	697	1621	0.338	0.324	0.263	0.142
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
2/100	1.27G	1.58	2.177	1.456	21	640: 100%	307/307 [00:15<
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
	all	697	1621	0.426	0.485	0.445	0.242
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
3/100	1.28G	1.6	1.987	1.474	35	640: 100%	307/307 [00:15<
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%
	all	697	1621	0.551	0.522	0.518	0.284
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
4/100	1.3G	1.571	1.82	1.451	43	640: 89%	273/307 [00:13<

Fruits Model



Visual Studio Code interface showing the YOLO Predict application running on a camera feed. The Explorer sidebar shows the project structure with 'yolo_fruits' and 'yolo_predict_camera.py' highlighted. The main editor shows the Python code for the camera prediction. A terminal window shows the command to run the script. An inset window shows the camera feed with bounding boxes and confidence scores for an apple (82.8%), orange (93.4%), and banana (79.5%).

```
12
13 model_path = os.path.join(OUTPUT_DIR, model_filename)
14 model = YOLO(model_path)
15
16 # 카메라 열기 (0: 기본 내장 카메라)
17 cap = cv2.VideoCapture(6)
18
19 if not cap.isOpened():
20     print("카메라를 열 수 없습니다.")
21     exit()
22
23 print("실시간 예측 시작 (종료: Q 키 누르기)")
24
25 while True:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

on.py", line 751, in _open_file_like
return _open_file(name_or_buffer, r
File "/home/rokey/.local/lib/python3
on.py", line 732, in __init__
super().__init__(open(name, mode))
FileNotFoundError: [Errno 2] No such file or directory: '/home/rokey/Tuto
rial/yolo_fruits/best.pt'
rokey@rokey:~/Tutorial\$ /usr/bin/python3 /home/rokey/Tutorial/yolo_fruits
/yolo_predict_camera.py
실시간 예측 시작 (종료: Q 키 누르기)

YOLO Predict

apple 82.8% orange 93.4% banana 79.5%

(x=618, y=100) ~ R:161 G:159 B:167

Tools Model

The screenshot displays the Visual Studio Code interface for a YOLO Predict camera application. The Explorer sidebar on the left shows the project structure, with the 'yolo_tools' directory and the 'yolo_predict_camera.py' file highlighted. The main editor window shows the Python code for the camera application, which uses YOLO for object detection. The code includes comments in Korean and Python code for opening the camera, reading frames, and predicting objects. A preview window titled 'YOLO Predict' shows a video feed with bounding boxes and confidence scores for three objects: pliers (32.8%), screwdriver (44.1%), and hammer (81.0%). The bounding boxes are green, and the confidence scores are displayed in green text above each object. The preview window also shows the coordinates of the bounding boxes: (x=533, y=7) ~ R:173 G:175 B:179.

```
13 model_path = os.path.join(OUTPUT_DIR, model_filename)
14 model = YOLO(model_path)
15
16 # 카메라 열기 (0: 기본 내장 카메라)
17 cap = cv2.VideoCapture(6)
18
19 if not cap.isOpened():
20     print("카메라를 열 수 없습니다.")
21     exit()
22
23 print("실시간 예측 시작 (종료: Q 키 누르기)")
24
25 while True:
26     ret, frame = cap.read()
27     if not ret:
28         break
29
30     # 프레임 예측 (stream=False: 프레임
31     results = model.predict(source=f
32
33     # 예측 결과 가져오기
34     result = results[0]
35     boxes = result.boxes # bounding
36     classes = result.names # 클래스
37
38     for box in boxes:
39         cls_id = int(box.cls[0]) # 클래스 ID
40         conf = float(box.conf[0]) * 100 # 신뢰도 (0~1 → 0~100%)
```

YOLO Predict

pliers 32.8%

screwdriver 44.1%

hammer 81.0%

(x=533, y=7) ~ R:173 G:175 B:179

실시간 예측 시작 (종료: Q 키 누르기)

Python 3.10.12 ('usr')

▶ 모델 학습

- Ultralytics는 다음과 같은 간단한 코드로 모델 학습이 가능

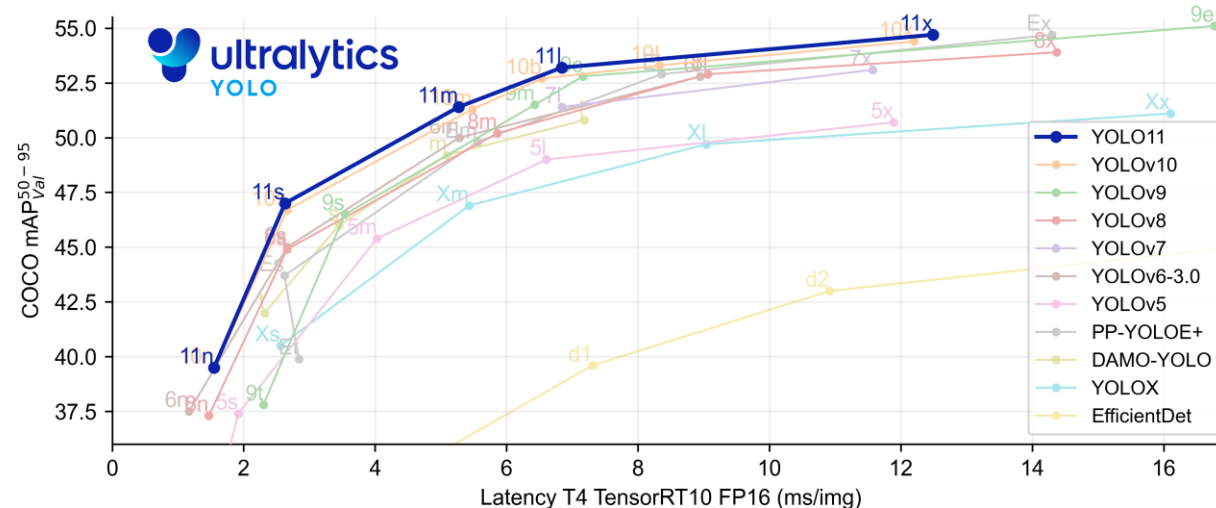
```
from ultralytics import YOLO

# Load a model
model = YOLO("yolo11n.pt") # load a pretrained model (recommended for training)

# Train the model
results = model.train(data="my_data.yaml", epochs=100, imgsz=640)
```

- model.train의 Hyperparameter
 - data: 데이터 집합 구성 파일의 경로
 - epochs: 총 에포크 수
 - imgsz: 학습을 위한 이미지 크기
 - 더 자세한 설명은 다음 링크 참조:
 - <https://docs.ultralytics.com/modes/train/#train-settings>

사용 가능한 모델 종류는 다음 링크 참조: <https://docs.ultralytics.com/models/>



▶ 모델 학습

```
from ultralytics import YOLO

# Load a model
model = YOLO("yolo11n.pt") # load a pretrained model (recommended for training)

# Train the model
results = model.train(data="my_data.yaml", epochs=100, imgsz=640)
```

- 모델(YOLO) 경로 "yolo11n.pt": Ultralytics의 사전학습된 YOLOv11n 모델을 불러옴
 - 그 외 다양한 크기의 YOLOv11 모델을 사용할 수 있음: <https://docs.ultralytics.com/models/yolo11/>
 - 또한, YOLOv12, RT-DETR 등의 다양한 모델 사용 가능: <https://docs.ultralytics.com/models/>
- models.train 함수
 - epochs, imgsz, batch는 적절히 수정
 - data의 경로는 개인 작업 환경에 맞게 경로 설정하기
 - data yaml 파일 안에서 train, val, test 데이터셋 경로 확인해야 함

Object Detection

▶ 모델 학습

- 다음과 같이 학습이 진행되는 것을 확인 가능

```

15      [-1, 4] 1      0 ultralytics.nn.modules.conv.Concat      [1]
16      -1 1      32096 ultralytics.nn.modules.block.C3k2      [256, 64, 1, False]
17      -1 1      36992 ultralytics.nn.modules.conv.Conv      [64, 64, 3, 2]
18      [-1, 13] 1      0 ultralytics.nn.modules.conv.Concat      [1]
19      -1 1      86720 ultralytics.nn.modules.block.C3k2      [192, 128, 1, False]
20      -1 1      147712 ultralytics.nn.modules.conv.Conv      [128, 128, 3, 2]
21      [-1, 10] 1      0 ultralytics.nn.modules.conv.Concat      [1]
...
Transferred 448/499 items from pretrained weights
TensorBoard: Start with 'tensorboard --logdir runs/detect/yolov11-custom3', view at http://localhost:6006/
Freezing layer 'model.23.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
/home/rokey/.local/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user\_install.html
from .autonotebook import tqdm as notebook_tqdm
AMP: checks passed ✓
train: Scanning /home/rokey/Tutorial/train/labels.... 35 images, 0 backgrounds, 0 corrupt: 100%|██████████| 35/35 [00:00<00:00, 3831.91it/s]
train: New cache created: /home/rokey/Tutorial/train/labels.cache

INFO:albumentations.check_version:A new version of Albumentations is available: 2.0.5 (you have 1.4.10). Upgrade using: pip install --upgrade albumentations
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01), CLAHE(p=0.01, clip_limit=(1, 4.0), tile_grid_size=(8, 8))
val: Scanning /home/rokey/Tutorial/valid/labels.... 10 images, 0 backgrounds, 0 corrupt: 100%|██████████| 10/10 [00:00<00:00, 2146.08it/s]
val: New cache created: /home/rokey/Tutorial/valid/labels.cache

Plotting labels to runs/detect/yolov11-custom3/labels.jpg...
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum' automatically...
optimizer: AdamW(lr=0.000909, momentum=0.9) with parameter groups 81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bias(decay=0.0)
TensorBoard: model graph visualization added ✓
Image sizes 640 train, 640 val
Using 8 dataloader workers
Logging results to runs/detect/yolov11-custom3
Starting training for 100 epochs...

  Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
  1/100   2.44G    1.716    3.895    1.335      21        640: 100%|██████████| 3/3 [00:00<00:00, 3.30it/s]
          Class    Images  Instances  Box(P)      R      mAP50  mAP50-95): 100%|██████████| 1/1 [00:00<00:00, 2.07it/s]
          all      10      41      0.0293    0.535    0.0334    0.013

  Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
  2/100   2.43G    1.575    3.908    1.238      29        640: 100%|██████████| 3/3 [00:00<00:00, 16.02it/s]
          Class    Images  Instances  Box(P)      R      mAP50  mAP50-95): 100%|██████████| 1/1 [00:00<00:00, 36.99it/s]
          all      10      41      0.0405    0.481    0.0445    0.0167

```

▶ 모델 학습

- 학습이 완료되면 runs/detect/my_model/weight 폴더에 학습이 완료된 모델이 저장됨(my_model)
 - best.pt: validation 성능이 제일 좋은 epoch의 모델 weight
 - last.pt: 마지막 epoch의 모델 weight
- TO-DO1: runs폴더에 저장되어 있는 학습 결과를 가지고 plot 그려보기

▶ 모델 추론

- 다음 코드를 이용해 학습된 모델 추론 (Tutorial/OD_Tutorial/YOLO 폴더 참조)
- /path/to/model_weight: 학습된 모델 경로
- sample_test.jpg: 테스트에 사용할 파일

```
from ultralytics import YOLO

class YoloEval:
    def __init__(self, model_path):
        self.model = YOLO(model_path)

    def eval(self, image_path, conf = 0.25, img_save = False):
        results = self.model.predict(source=image_path, conf=conf)
        for i, result in enumerate(results):
            if img_save:
                result.save(filename=f'result_{i}.jpg') # 결과 이미지 저장

if __name__ == "__main__":
    yolo_eval = YoloEval("path/to/model_weight/best.pt") ← pc 작업환경에 맞게 경로 설정하기
    yolo_eval.eval(["sample_test.jpg"], img_save=True)
```

▶ 모델 추론

- TODO2: 학습된 모델을 이용해 가지고 있는 물건(ex: fruit or tools)를 객체 탐지 해보기
- 웹캠을 사용한 inference 추천

▶ 모델 학습2

- 다음과 같이 class 형태로 객체 지향적으로 모델 학습 코드 제작 가능

```
from ultralytics import YOLO

class YoloTrain:
    def __init__(self, model_path):
        self.model = YOLO(model_path)

    def train(self, data_path, is_absolute_path=False):
        if not is_absolute_path:
            import os
            data_path = os.path.abspath(data_path)

        self.model.train(
            data=data_path,      # 데이터 경로
            name='yolo_custom',  # 실험 이름
            pretrained=True,     # 사전학습 모델 사용 여부
            cfg='custom_config.yaml' # 하이퍼파라미터 파일
        )

if __name__ == "__main__":
    yolo_train = YoloTrain("yolo11n.yaml")
    yolo_train.train("my_data/data.yaml")
```

▶ 모델 학습2

- cfg 인자를 이용해 모델 학습 config 파일 사용 가능

```
from ultralytics import YOLO

class YoloTrain:
    def __init__(self, model_path):
        self.model = YOLO(model_path)

    def train(self, data_path, is_absolute_path=False):
        if not is_absolute_path:
            import os
            data_path = os.path.abspath(data_path)

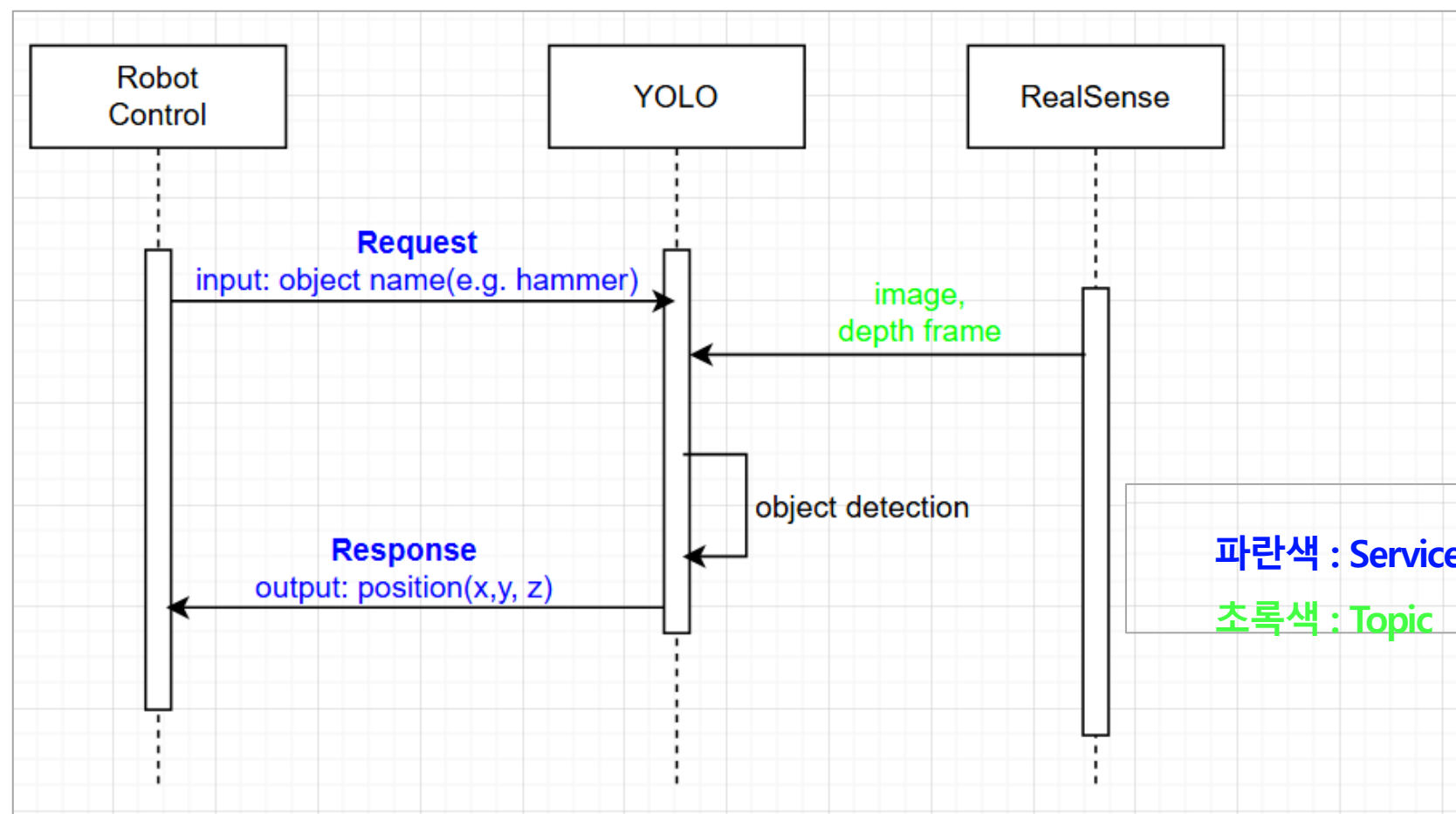
        self.model.train(
            data=data_path,      # 데이터 경로
            name='yolo_custom',  # 실험 이름
            pretrained=True,     # 사전학습 모델 사용 여부
            cfg='custom_config.yaml' # 하이퍼파라미터 파일
        )

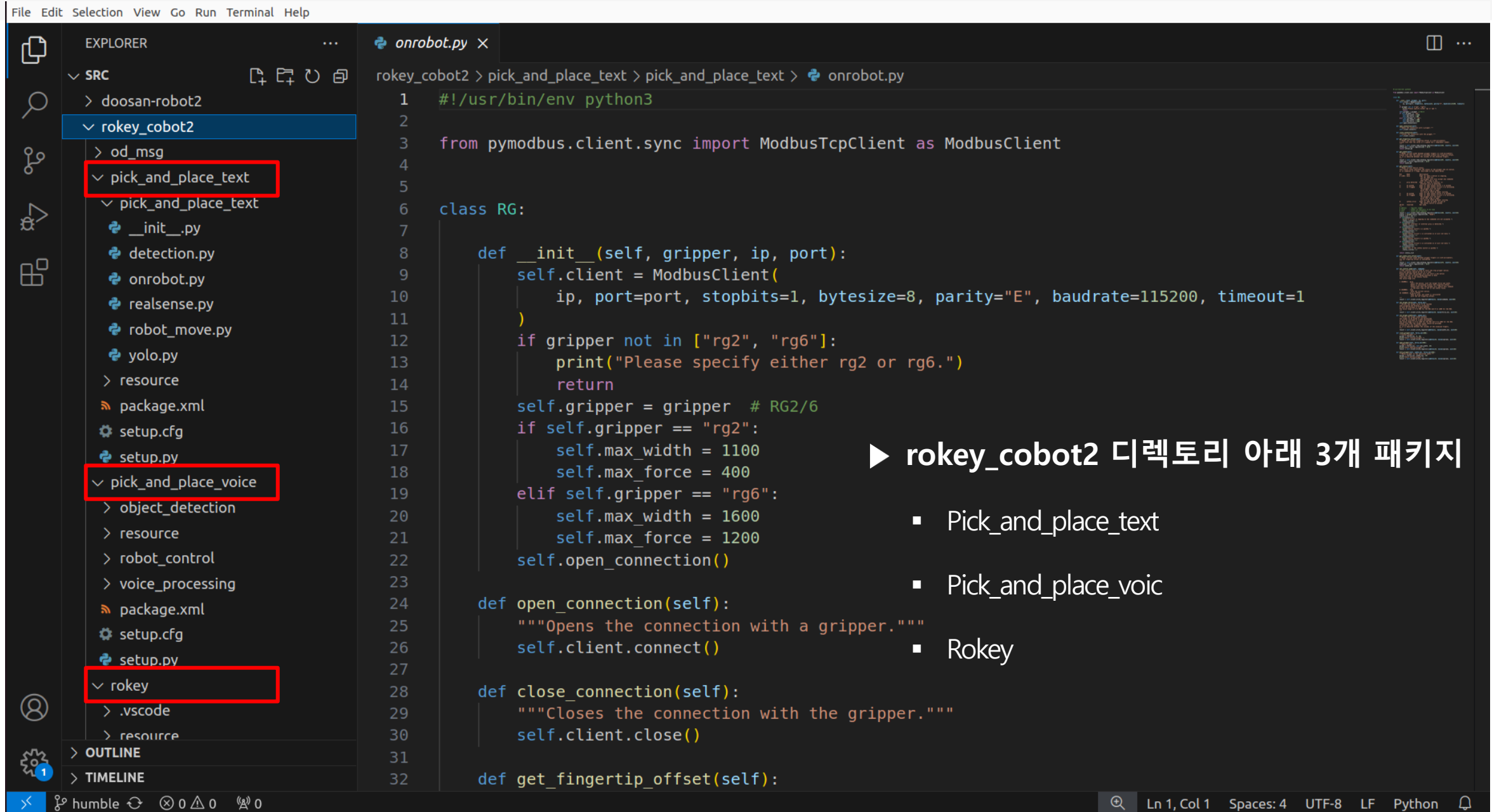
if __name__ == "__main__":
    yolo_train = YoloTrain("yolo11n.yaml")
    yolo_train.train("my_data/data.yaml")
```

```
# custom_config.yaml 예시
task: detect
mode: train
model: yolov8n.yaml
data: data.yaml
epochs: 100
imgsz: 640
batch: 16
```

► Pick & Place

- 사용자가 입력한 텍스트를 기반으로 지정한 물체를 Pick하고 이동시키는 작업





The screenshot shows the VS Code interface with the `rokey_cobot2` package structure in the Explorer view and the `onrobot.py` file open in the editor.

Package Structure (Explorer View):

- `rokey_cobot2`
 - `od_msg`
 - `pick_and_place_text` (highlighted)
 - `pick_and_place_text`
 - `__init__.py`
 - `detection.py`
 - `onrobot.py`
 - `realsense.py`
 - `robot_move.py`
 - `yolo.py`
 - `resource`
 - `package.xml`
 - `setup.cfg`
 - `setup.py`
 - `pick_and_place_voice` (highlighted)
 - `object_detection`
 - `resource`
 - `robot_control`
 - `voice_processing`
 - `package.xml`
 - `setup.cfg`
 - `setup.py`
 - `rokey` (highlighted)
 - `.vscode`
 - `resource`

Code Snippet (onrobot.py):

```
1  #!/usr/bin/env python3
2
3  from pymodbus.client.sync import ModbusTcpClient as ModbusClient
4
5
6  class RG:
7
8      def __init__(self, gripper, ip, port):
9          self.client = ModbusClient(
10              ip, port=port, stopbits=1, bytesize=8, parity="E", baudrate=115200, timeout=1
11          )
12          if gripper not in ["rg2", "rg6"]:
13              print("Please specify either rg2 or rg6.")
14              return
15          self.gripper = gripper # RG2/6
16          if self.gripper == "rg2":
17              self.max_width = 1100
18              self.max_force = 400
19          elif self.gripper == "rg6":
20              self.max_width = 1600
21              self.max_force = 1200
22          self.open_connection()
23
24      def open_connection(self):
25          """Opens the connection with a gripper."""
26          self.client.connect()
27
28      def close_connection(self):
29          """Closes the connection with the gripper."""
30          self.client.close()
31
32      def get_fingertip_offset(self):
```

rokey_cobot2 디렉토리 아래 3개 패키지

- Pick_and_place_text
- Pick_and_place_voic
- Rokey

▶ Service Message 파일 생성

- ~/cobot_ws/src/rokey_cobot2/od_msg/srv/SrvDepthPosition.srv 파일 확인

```
string target
---
float64[] depth_position
```

- ---을 기준으로 위는 요청(Request), 아래는 응답(Response)을 의미
- string target: string타입의 target이라는 변수를 request 메시지 전송
- float64 타입의 실수형 배열인 depth_position이라는 변수를 response 메시지 전송

▶ Service Message 파일 생성

- ~/cobot_ws/src/rokey_cobot2/od_msg/CMakeLists.txt 파일에 다음 스크립트가 포함되어 있는지 확인

```
rosidl_generate_interfaces(${PROJECT_NAME}  
    "srv/SrvDepthPosition.srv"  
)
```

- rosidl_generate_interfaces : build 시 *.msg, *.srv, *.action 파일들을 자동으로 코드로 변환해주는 명령어
- srv/SrvDepthPosition.srv : 생성하고자 하는 서비스 인터페이스 파일의 경로

▶ Object Detection

- ~/cobot_ws/src/rokey_cobot2/pick_and_place_text/resource에 아래 파일 옮기기
 - 이전에 학습한 모델 *.pt 파일
 - Calibration 결과로 나온 T_gripper2camera.npy
 - .env파일을 생성 후 OPENAI_API_KEY 입력하기(슬랙 채널에 별도 제공)

```
OPENAI_API_KEY=sk-proj-A....7HcA
```

Pick and Place

▶ Object Detection

- ~/cobot_ws/src/rokey_cobot2/pick_and_place_text/resource에 아래 파일 옮기기
 - class_name_tool.json 파일 생성 후 아래와 같이 사용하는 데이터셋에 알맞게 수정
 - json 파일 딕셔너리의 클래스 키 값은 0번부터 시작해야 함을 주의!!

```
{  
  "0": "drill",  
  "1": "hammer",  
  "2": "pliers",  
  "3": "screwdriver",  
  "4": "wrench"  
}
```

▶ Object Detection

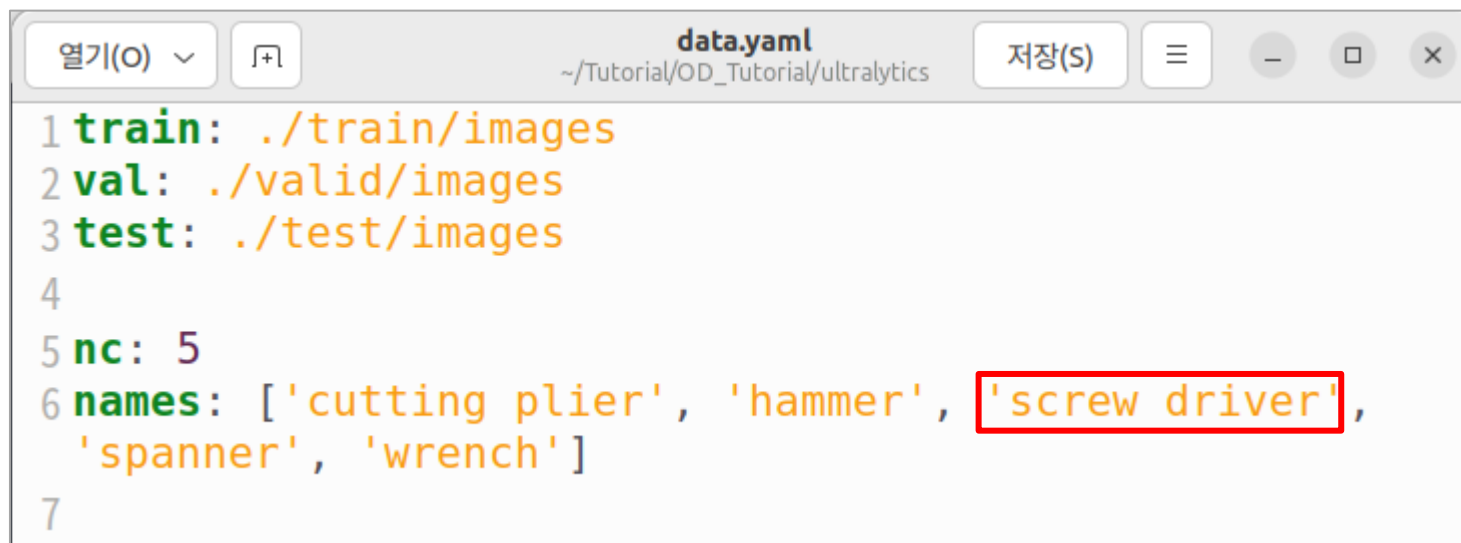
- ~/cobot_ws/src/rokey_cobot2/pick_and_place_text/resource에 아래 파일 옮기기
 - 번호 별 클래스 확인 방법: 데이터셋에서 한 이미지와 이에 해당되는 라벨 txt 파일을 열어서 클래스 번호 확인



1 3 0.509375 0.4765625 0.74140625 0.9125

▶ Object Detection

- ~/cobot_ws/src/rokey_cobot2/pick_and_place_text/resource에 아래 파일 옮기기
 - data.yaml 파일에서 names의 해당 인덱스 번호가 무엇인지 확인(data.yaml에 있는 클래스 번호는 1번부터 있음)
 - 전 슬라이드 예시에서는 3이므로 3은 'screw driver'



```
1 train: ./train/images
2 val: ./valid/images
3 test: ./test/images
4
5 nc: 5
6 names: ['cutting plier', 'hammer', 'screw driver',
7         'spanner', 'wrench']
```

▶ Object Detection

- ~/cobot_ws/src/rokey_cobot2/pick_and_place_text/pick_and_place_text/yolo.py에 모델 이름과 json 파일 이름 수정

```
PACKAGE_NAME = 'object_detection'
PACKAGE_PATH = get_package_share_directory(PACKAGE_NAME)

YOLO_MODEL_FILENAME = 'yolov8n_tools_0122.pt'
YOLO_CLASS_NAME_JSON = "class_name_tool.json"

YOLO_MODEL_PATH = os.path.join(PACKAGE_PATH, 'resource', YOLO_MODEL_FILENAME)
YOLO_JSON_PATH = os.path.join(PACKAGE_PATH, 'resource', YOLO_CLASS_NAME_JSON)
```

← 개인 환경에 맞게 수정

- YOLO_MODEL_FILENAME: resources 폴더에 옮긴 학습된 모델 pt 파일 이름으로 수정

▶ Object Detection - 실행

- 총 4개의 터미널을 열어 각각의 터미널에서 아래와 같은 명령어 실행
- Realsense 런치파일 실행(줄바꿈 없이)

```
ros2 launch realsense2_camera rs_align_depth_launch.py depth_module.depth_profile:=640×480×30  
rgb_camera.color_profile:=640×480×30 initial_reset:=true align_depth.enable:=true enable_rgbd:=true  
pointcloud.enable:=true
```

- 두산로봇 bringup

```
colcon build  
source install/setup.bash  
ros2 launch dsr_bringup2 dsr_bringup2_rviz.launch.py mode:=real host:=192.168.1.100 port:=12345 model:=m0609
```

▶ Object Detection - 실행

- 총 4개의 터미널을 열어 각각의 터미널에서 아래와 같은 명령어 실행
- detection 노드 실행 (cobot_ws 폴더에서 실행)

```
source ~/cobot_ws/install/setup.bash  
ros2 run pick_and_place_text detection
```

- robot_move 노드 실행 (cobot_ws 폴더에서 실행)

```
export PYTHONPATH=$PYTHONPATH:~/cobot_ws/install/dsr_common2/lib/dsr_common2/imp  
source ~/cobot_ws/install/setup.bash  
ros2 run pick_and_place_text robot_move
```

Pick and Place

▶ Object Detection - 실행

- Pick_and_place 노드를 실행한 터미널에서 집고 싶은 객체의 번호를 선택

```
=====
```

```
Available tools:
```

```
1 : drill  
2 : hammer  
3 : pliers  
4 : screwdriver  
5 : wrench
```

```
What do you want to bring?: 2
```

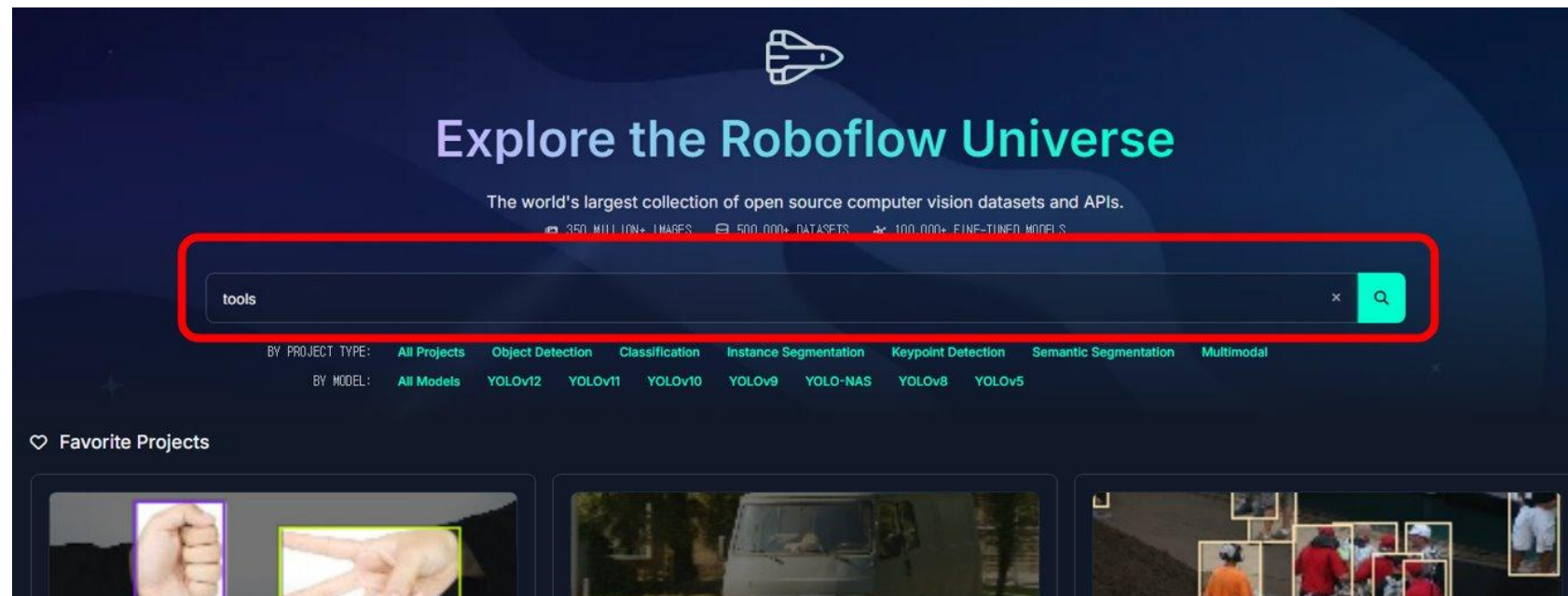
- Robot이 물체를 Pick and Place 하는지 확인(물체를 Realsense Camera의 가시범위 내에 있도록 놓자)

▶ 모델 학습 - 데이터셋 다운로드

- 데이터셋 다운로드(아래 코드에서 data에 들어갈 yaml 파일)

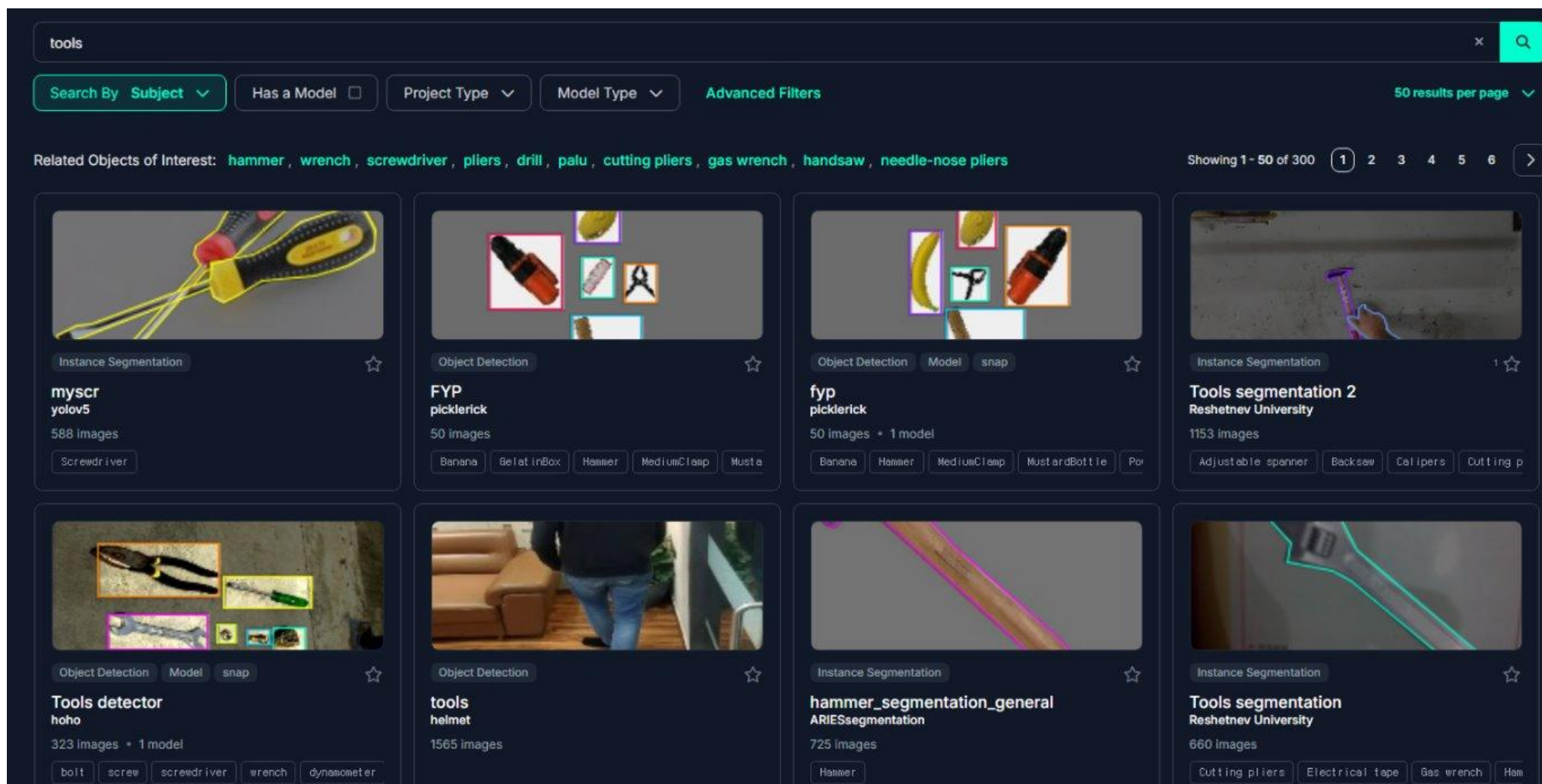
```
results = model.train(data="my_data.yaml", epochs=100, imgsz=640)
```

- 다음 링크 접속하여 원하는 dataset 검색(ex: tools, fruit)
- <https://universe.roboflow.com/>



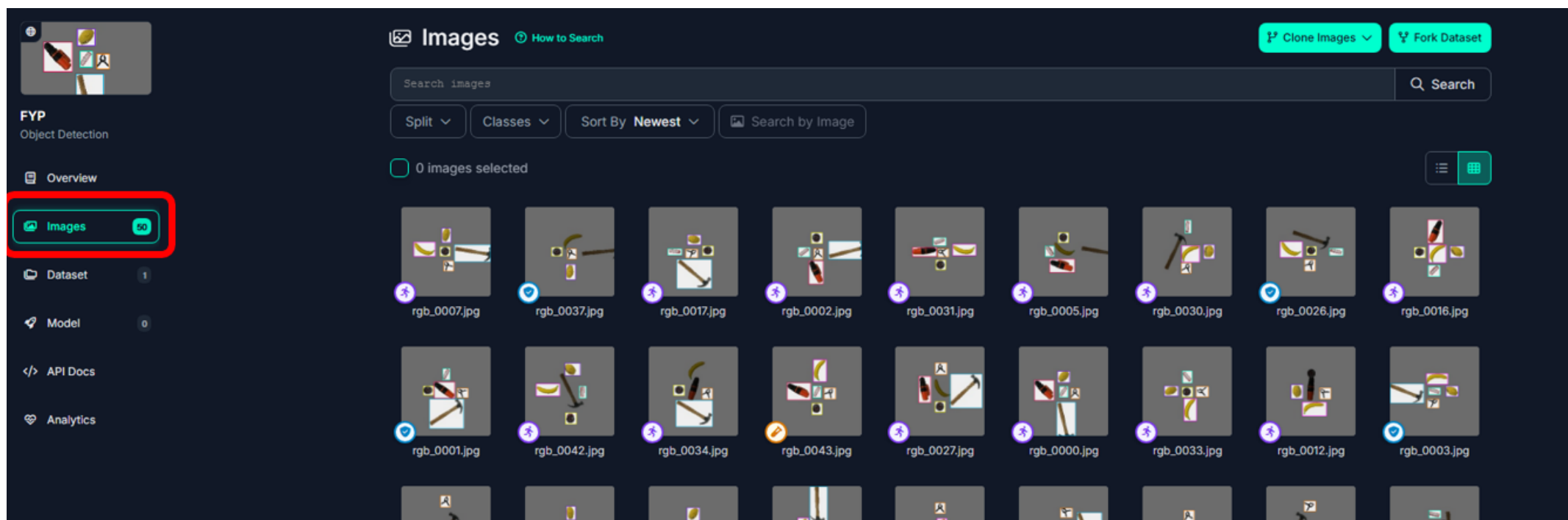
▶ 모델 학습 - 데이터셋 다운로드

- 적절한 데이터셋 선택(현재 가지고 있는 물품들과 최대한 비슷한 클래스를 가지고 있으면서 데이터의 양이 많은 것)



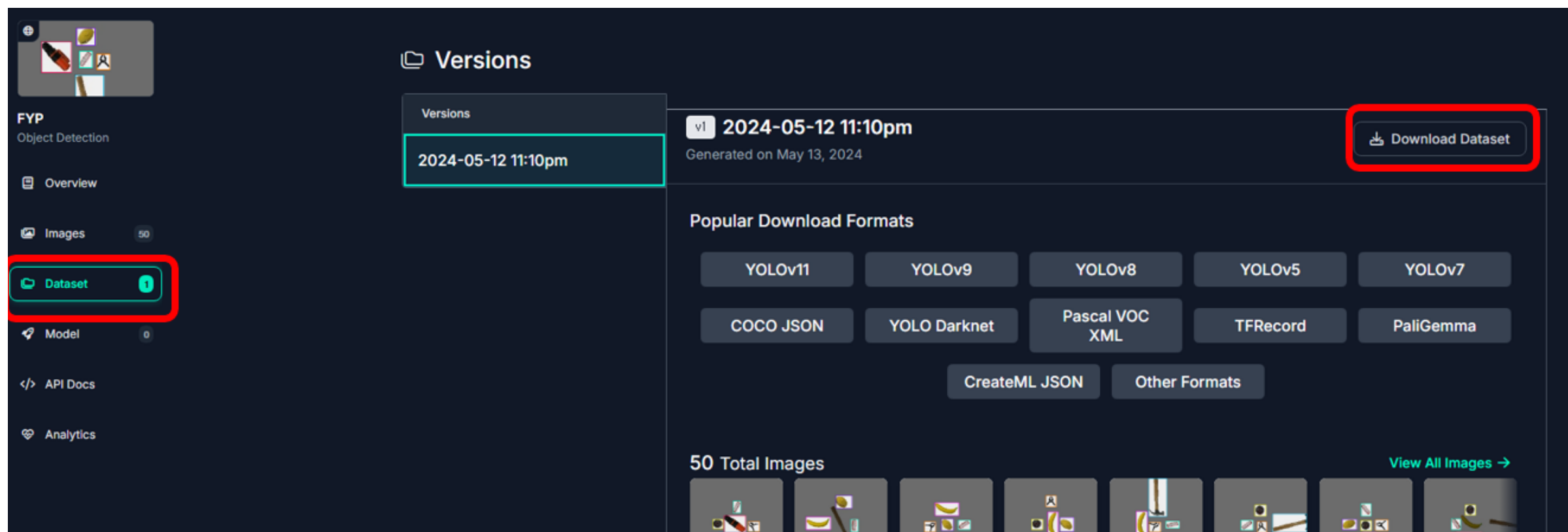
▶ 모델 학습 - 데이터셋 다운로드

- Side-bar에서 Images를 선택하여 샘플 데이터 확인



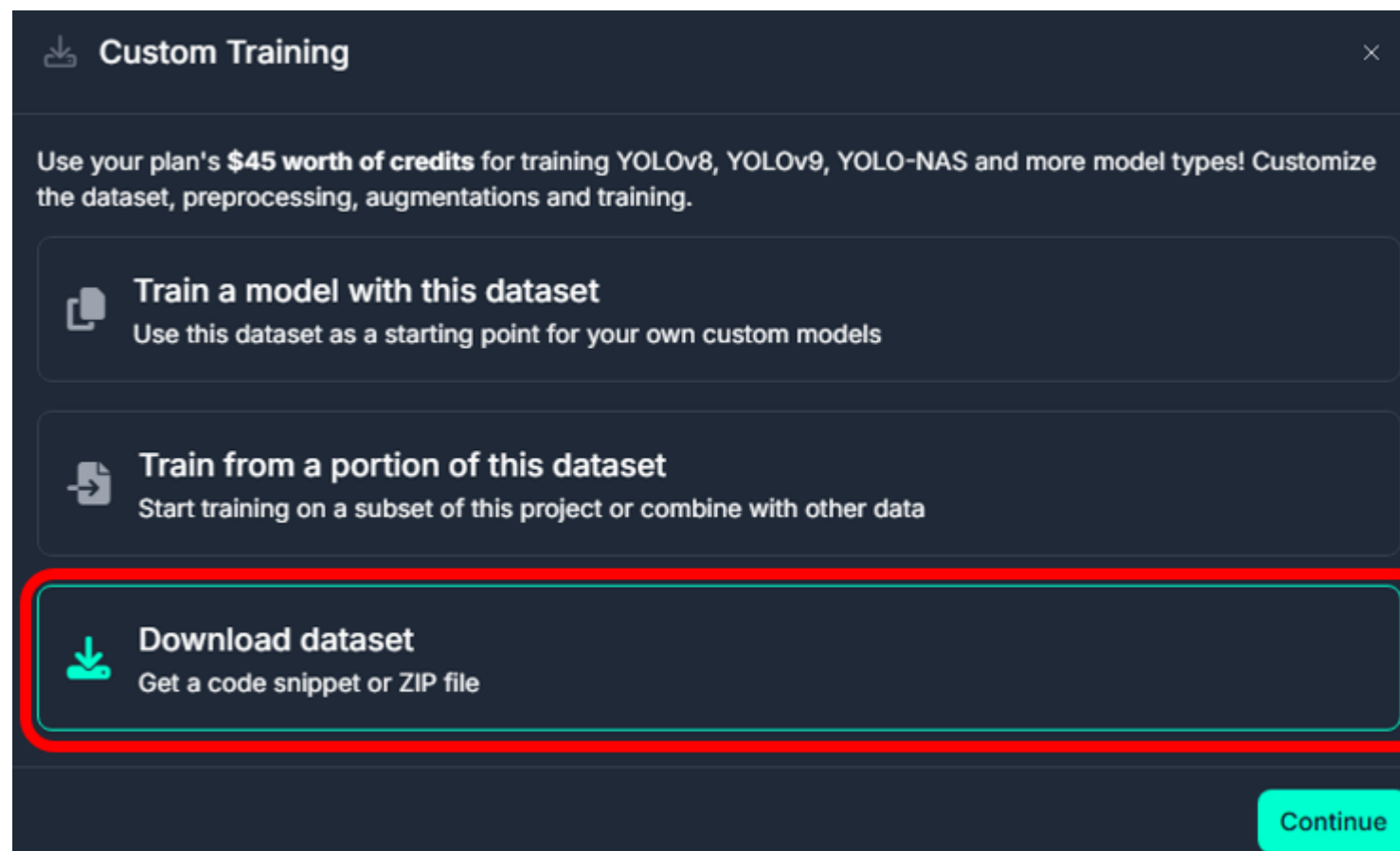
▶ 모델 학습 - 데이터셋 다운로드

- 적합한 데이터셋을 선택해서 side-bar에서 Dataset 선택 후 Download Dataset 아이콘을 눌러 데이터셋 다운로드



▶ 모델 학습 - 데이터셋 다운로드

- Download dataset을 선택 후 continue



▶ 모델 학습 - 데이터셋 다운로드

- format을 선택한 후 Show download code를 선택 후 continue

Download

Format

YOLOv11

TXT annotations and YAML config used with YOLOv11.

Download Options

☐ Download zip to computer

☐ Downloads all images, annotations, and classes.

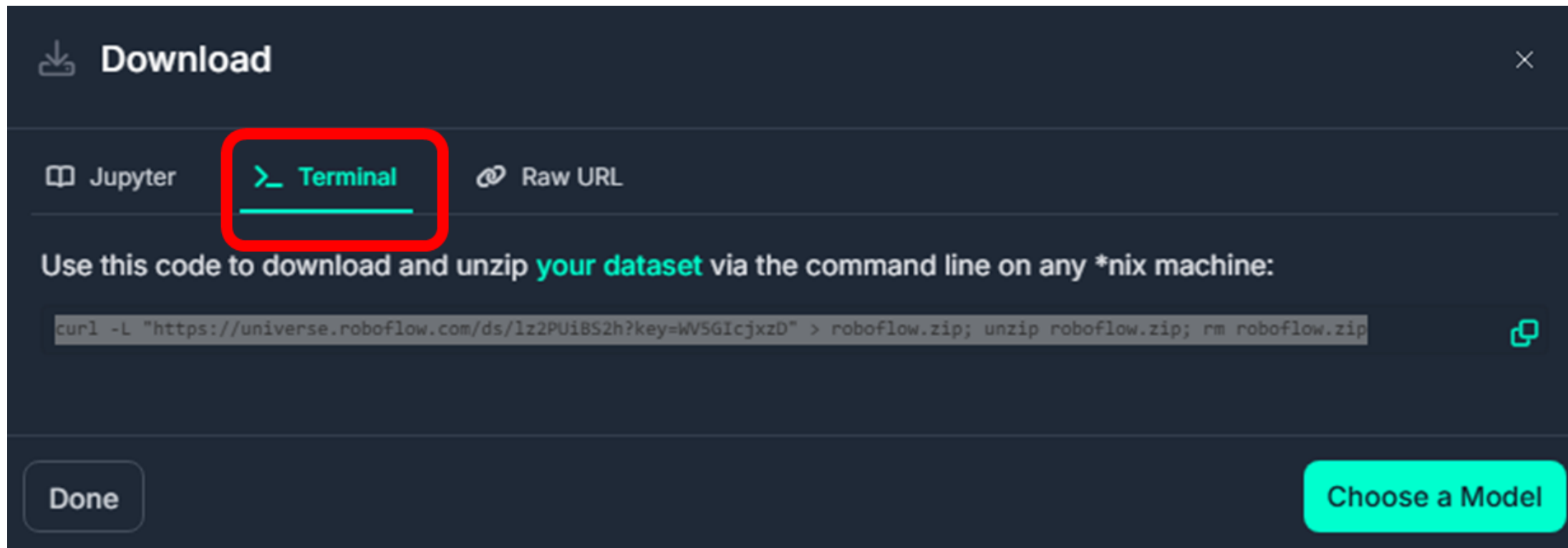
☐ Show download code

☒ Custom train this dataset using the provided code snippet in a notebook.

Cancel Continue

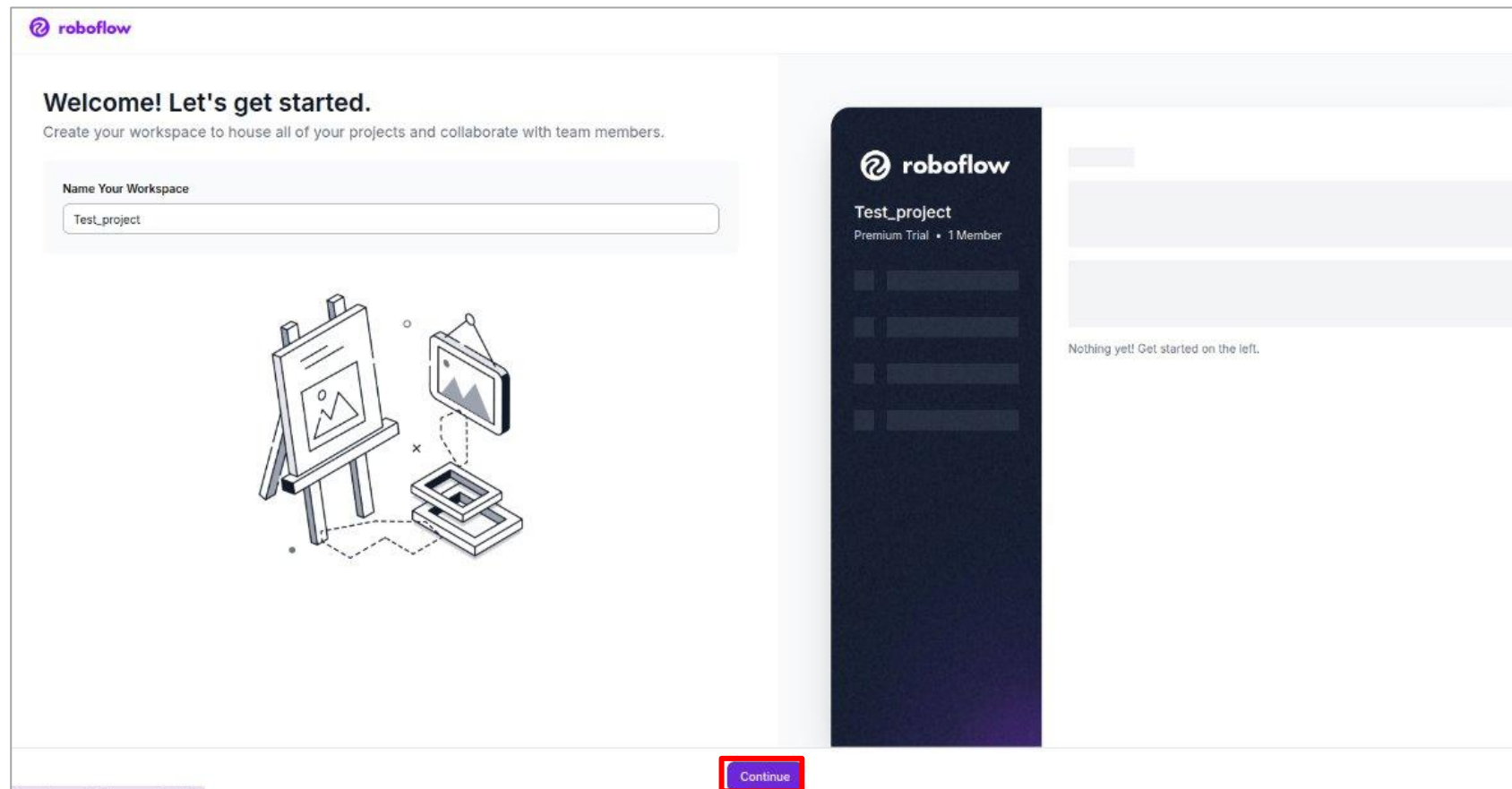
▶ 모델 학습 - 데이터셋 다운로드

- 상단 바에서 Terminal을 선택 후 나온 명령어를 이용하여 데이터 다운로드.
- Tutorial/Ultralytics/data 폴더 생성 후 해당 폴더에서 복사한 명령어 수행
- 다운로드 위치는 반드시 기억해둘 것



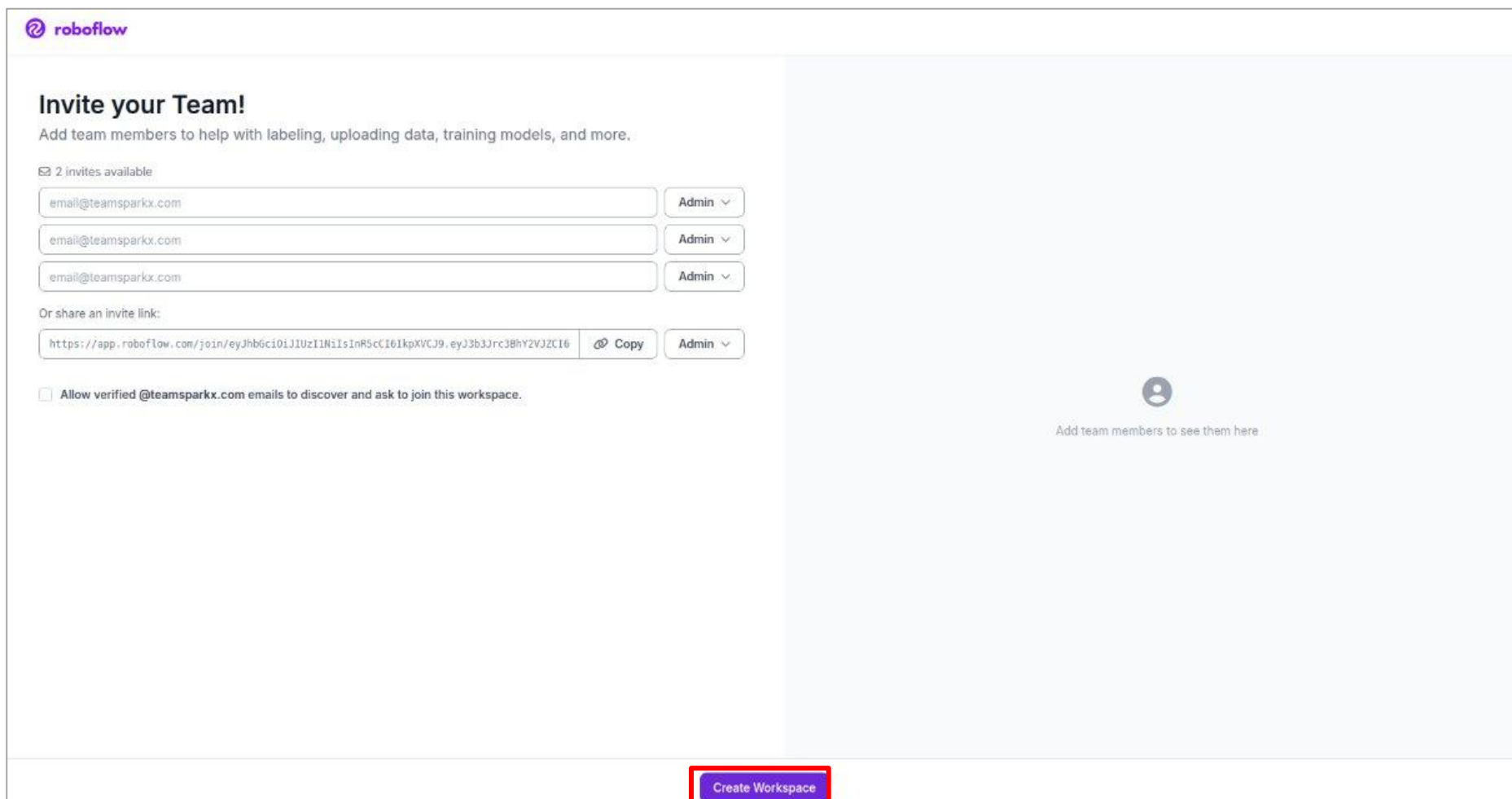
▶ 모델 학습 - 커스텀 데이터셋 생성

- Roboflow 로그인 후 workspace 이름 설정
- 하단 continue 버튼 클릭



▶ 모델 학습 - 커스텀 데이터셋 생성

- 팀원 초대 후 하단 Create Workspace 버튼 클릭



The image shows the Roboflow 'Invite your Team!' interface. It features a form for adding team members with three email input fields, each with an 'Admin' dropdown menu. Below this is a section for sharing an invite link, including a text input with a URL and a 'Copy' button. At the bottom left, there is a checkbox labeled 'Allow verified @teamsparkx.com emails to discover and ask to join this workspace.' On the right side, there is a large light blue area with a person icon and the text 'Add team members to see them here'. At the bottom center, there is a red-outlined button labeled 'Create Workspace'.

roboflow

Invite your Team!

Add team members to help with labeling, uploading data, training models, and more.

✉ 2 invites available

email@teamsparkx.com Admin ▾

email@teamsparkx.com Admin ▾

email@teamsparkx.com Admin ▾

Or share an invite link:

https://app.roboflow.com/join/eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ3b3Jrc3RlY2VJZCI6... Copy Admin ▾

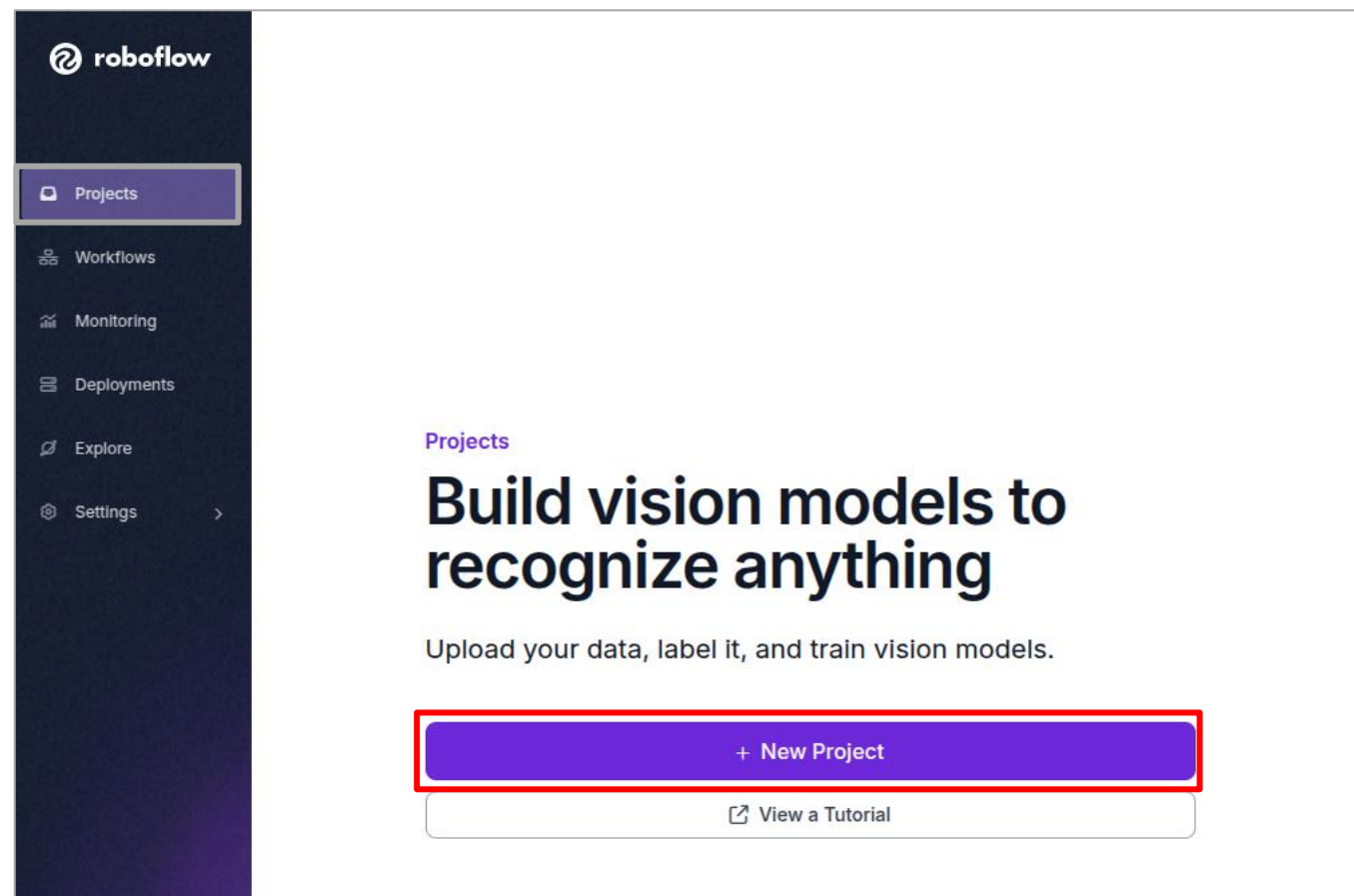
☐ Allow verified @teamsparkx.com emails to discover and ask to join this workspace.

Add team members to see them here

Create Workspace

▶ 모델 학습 - 커스텀 데이터셋 생성

- 왼쪽 상단 Projects 클릭
- New Project 클릭하여 새로운 프로젝트 생성



▶ 모델 학습 - 커스텀 데이터셋 생성

▪ Project Name 설정

Let's create your project.

Testproject > Private Trial My First Project

Project Name: My First Project

Annotation Group: objects

Visibility: Private Public

Project Type

Object Detection Bounding Boxes # Counts Tracking
Identify objects and their positions with bounding boxes.

Classification Image Labels Filtering Content Moderation
Assign labels to the entire image. Single-Label Multi-Label

Instance Segmentation Polygons Measuring Odd Shapes
Detect multiple objects and their actual shape.

Keypoint Detection Skeleton Structure Pose Estimation
Identify keypoints ("skeletons") on subjects.

Multimodal Prompts Visual Question Answering Captions
Describe images using text pairs.

Show More ↓

Cancel Create Private Project



▶ 모델 학습 - 커스텀 데이터셋 생성

- Annotation Group, Object Detection 선택 아래 사진과 같이 되어 있는지 확인

Let's create your project.

Testproject > Private Trial My First Project

Project Name: My First Project

Annotation Group: objects

Visibility: Private Public

Project Type

Object Detection Bounding Boxes Counts Tracking
Identify objects and their positions with bounding boxes.

Classification Image Labels Filtering Content Moderation
Assign labels to the entire image. Single-Label Multi-Label

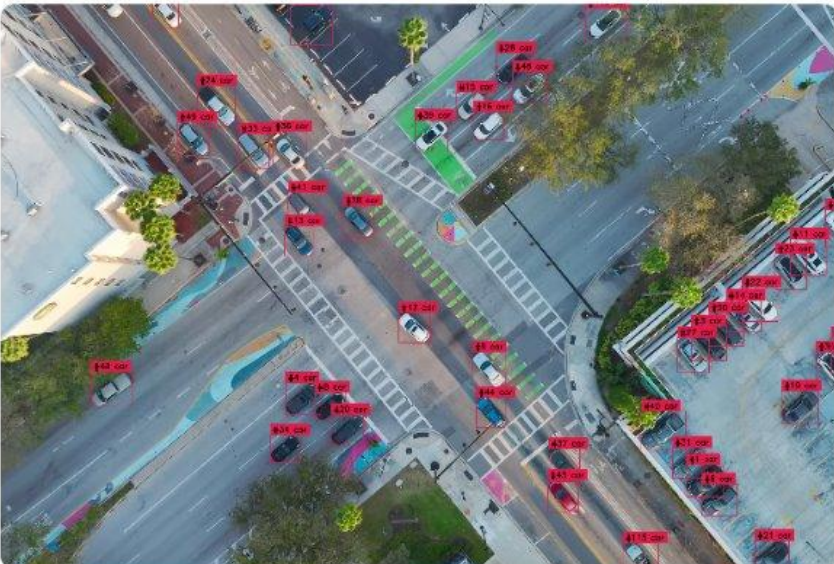
Instance Segmentation Polygons Measuring Odd Shapes
Detect multiple objects and their actual shape.

Keypoint Detection Skeleton Structure Pose Estimation
Identify keypoints ("skeletons") on subjects.

Multimodal Prompts Visual Question Answering Captions
Describe images using text pairs.

Show More ↓

Cancel Create Private Project



▶ 모델 학습 - 커스텀 데이터셋 생성

- Visibility는 Public으로 설정하기!!

Let's create your project.

Testproject > Private Trial My First Project

Project Name: My First Project

Annotation Group: objects

Visibility: ~~Private~~ Public

Project Type

Object Detection ☒ Bounding Boxes # Counts Tracking

Identify objects and their positions with bounding boxes.

Classification ☒ Image Labels Filtering Content Moderation

Assign labels to the entire image.

☐ Single-Label ☐ Multi-Label

Instance Segmentation ☒ Polygons Measuring Odd Shapes

Detect multiple objects and their actual shape.

Keypoint Detection ☒ Skeleton Structure Pose Estimation

Identify keypoints ("skeletons") on subjects.

Multimodal ☒ Prompts Visual Question Answering Captions

Describe images using text pairs.

Show More ↓

Cancel Create Private Project



▶ 모델 학습 - 커스텀 데이터셋 생성

- Create Project 클릭

Let's create your project.

Testproject > Private Trial My First Project

Project Name: My First Project

Annotation Group: objects

Visibility: Private Public

Project Type

Object Detection ☒ Bounding Boxes # Counts Tracking
Identify objects and their positions with bounding boxes.

Classification ☒ Image Labels Filtering Content Moderation
Assign labels to the entire image. ☐ Single-Label ☐ Multi-Label

Instance Segmentation ☒ Polygons Measuring Odd Shapes
Detect multiple objects and their actual shape.

Keypoint Detection ☒ Skeleton Structure Pose Estimation
Identify keypoints ("skeletons") on subjects.

Multimodal ☒ Prompts Visual Question Answering Captions
Describe images using text pairs.

Show More ↓

Cancel **Create Private Project**



▶ 모델 학습 - 커스텀 데이터셋 생성

- 사용할 모든 이미지(Train, Validation, Test)들을 드래그

TESTPROJECT

Test
Object Detection

DATA

↑ Upload Data

Annotate

Dataset 0

Versions Train

Analytics

Classes & Tags

MODELS

Models

Visualize

DEPLOY

Deployments

↑ Upload

Batch Name: Uploaded on 07/25/25 at 11:03 am

Tags: Search or add tags for images...

☒ Create batch instantly

Drag and drop file(s) to upload, or:

Select File(s) Select Folder

Supported Formats

- Images**
.jpg, .png, .bmp, .webp, .avif
- Annotations**
in 26 formats
- Videos**
.mov, .mp4
- PDFs**
.pdf

*Max size of 20MB and 16,400 × 10,900 pixels.

Need images to get started?

Search on Roboflow Universe
Search across 600k datasets and 400 million images in the world's largest platform for computer vision data.

Search for images →

Bulk Upload Images
Upload 1k+ images to Roboflow projects using Python SDK, REST API, and CLI.

Learn More

▶ 모델 학습 - 커스텀 데이터셋 생성

- Label With My Team 클릭

The screenshot displays the Roboflow web interface for a project named 'TESTPROJECT'. On the left sidebar, the 'Annotate' button is highlighted. The main area shows a batch upload status: 'Annotate > Batch', 'Uploaded on 07/25/25 at 11:03 am', and 'Uploaded Jul 25, 2025 (11:39 AM)'. A sample image of a white object is shown with the filename 'KakaoTalk_20250725_1059...'. On the right, a panel titled 'How do you want to label your images?' offers four options: 'Label Myself', 'Label With My Team' (which is highlighted with a red border and a right arrow), 'Auto-Label Entire Batch', and 'Hire Outsourced Labelers' (marked with a 'Trial' badge).

TESTPROJECT

← Annotate > Batch
Uploaded on 07/25/25 at 11:03 am
Uploaded Jul 25, 2025 (11:39 AM)

Upload More Rename

Test
Object Detection

DATA
↑ Upload Data
Annotate
Dataset 0
Versions Train
Analytics
Classes & Tags

MODELS
Models

How do you want to label your images?

Label Myself
Label images with our AI labeling tools.

Label With My Team >
Split up the labeling work across your team.

Auto-Label Entire Batch
Use your own custom model or a zero-shot model to automatically label your entire batch.

Hire Outsourced Labelers Trial
Work with a professional labeling team vetted by Roboflow.

▶ 모델 학습 - 커스텀 데이터셋 생성

- 팀원 초대 후 Start Labeling 클릭

Annotate > Batch

← Uploaded on 07/25/25 at 11:03 am

⌚ Uploaded Jul 25, 2025 (11:58 AM)


KakaoTalk_20250725_1059...

Upload MoreRename

Assign Images to Team Members

Total Images to Assign 1 / 1

☒ Shuffle images when assigning

Add InstructionsAdd Team Members

Search for team members...

정민
ejm9530@teamsparkx.com

1 image

BackStart Labeling

▶ 모델 학습 - 커스텀 데이터셋 생성

- Annotate -> Annotating 클릭

The screenshot shows the 'Annotate' interface of a machine learning platform. On the left is a sidebar with a 'TESTPROJECT' header and a thumbnail image. Below the thumbnail are sections for 'Test' (Object Detection), 'DATA' (with an 'Upload Data' button), and 'MODELS'. The 'Annotate' button in the 'DATA' section is highlighted with a red box. The main area is titled 'Annotate' and features filters for 'Sort By' (Newest), 'Labeler' (Show All), and 'Reviewer' (Show All). It is divided into two panels: 'Unassigned' (0 Batches) and 'Annotating' (1 Job). The 'Annotating' panel is highlighted with a red box and contains details for a job uploaded on 07/25/25 at 11:03... by Labeler '엄정민' and Reviewer '엄정민'. It shows a progress bar and a status summary: 0 Approved, 0 Rejected, 0 Annotated, and 1 Unannotated. A 'Start Annotating' button with a right arrow is at the bottom right of the 'Annotating' panel.

TESTPROJECT

Annotate

Sort By: Newest ▾ Labeler: Show All ▾ Reviewer: Show All ▾

Unassigned
0 Batches
[Upload More Images](#)

Annotating
1 Job

Uploaded on 07/25/25 at 11:03... ▾
Labeler: 엄정민
Reviewer: 엄정민

1 Images
● 0 Approved
● 0 Rejected
○ 0 Annotated
○ 1 Unannotated

[Start Annotating →](#)

▶ 모델 학습 - 커스텀 데이터셋 생성

- Unannotated 된 이미지들을 각 팀원이 클릭

TESTPROJECT



Test
Object Detection

DATA
↑ Upload Data

Annotate

Dataset 0

Versions **Train**

Analytics

Classes & Tags

Uploaded on 07/25/25 at 11:03 am [🔗](#)

Progress

1 Images
0 Approved
0 Rejected
0 Annotated
1 Unannotated

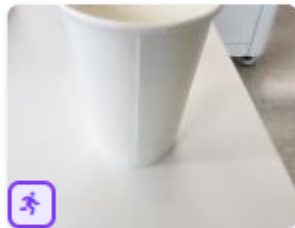
Instructions [🔗 Edit](#)

No specific instructions were added when this job was assigned

Assignment [🔗 Reassign](#)

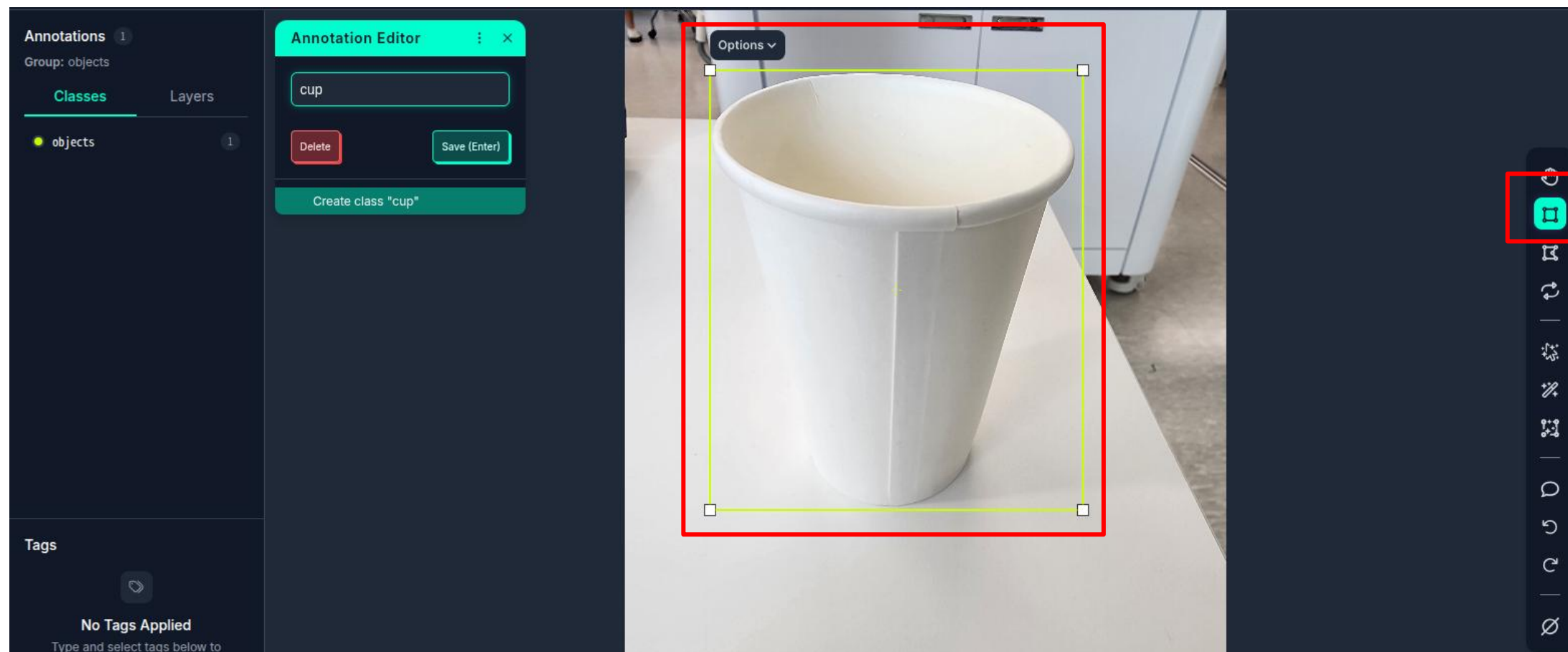
엄정민
Labeler
엄정민
Reviewer

Unannotated 1 **Annotated 0**


KakaoTalk_20250725_1...

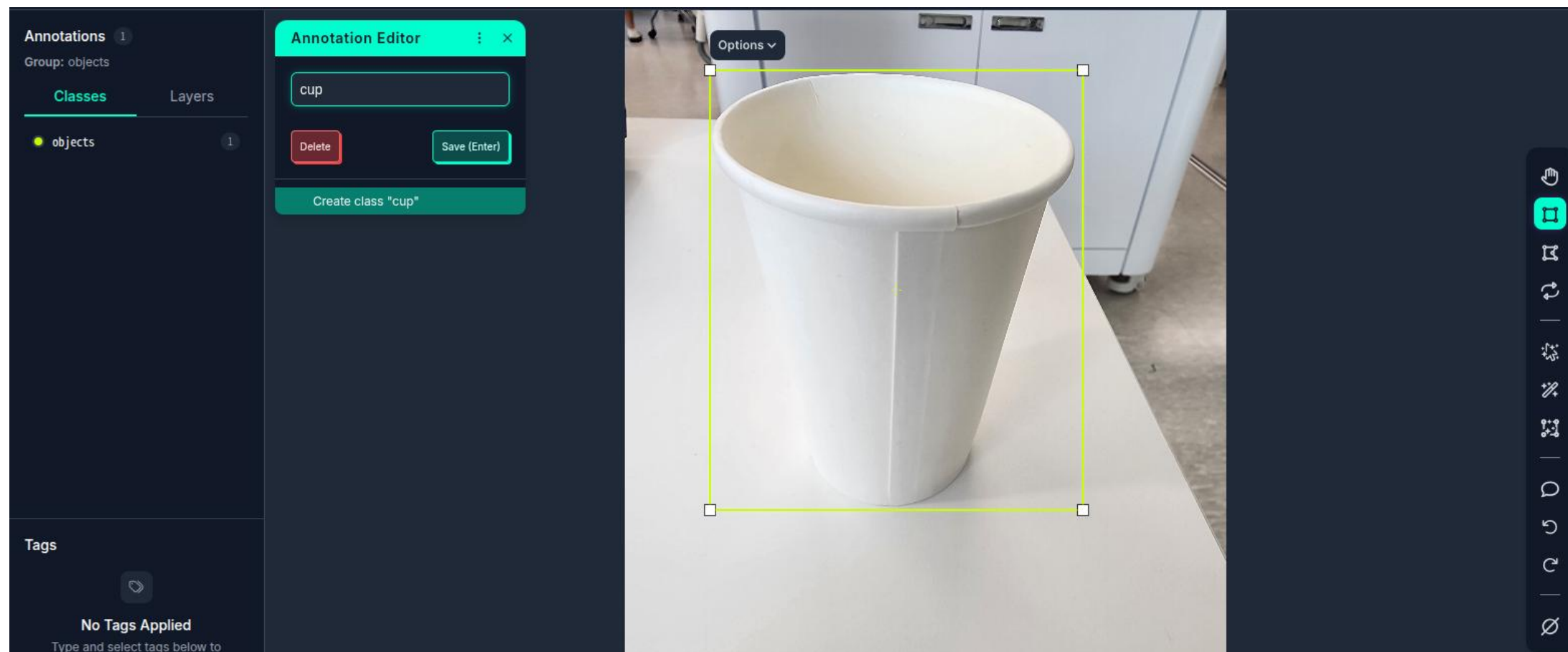
▶ 모델 학습 - 커스텀 데이터셋 생성

- 오른쪽 툴바의 Bounding Box Tool을 클릭한 후 해당 객체에 박스 드래그



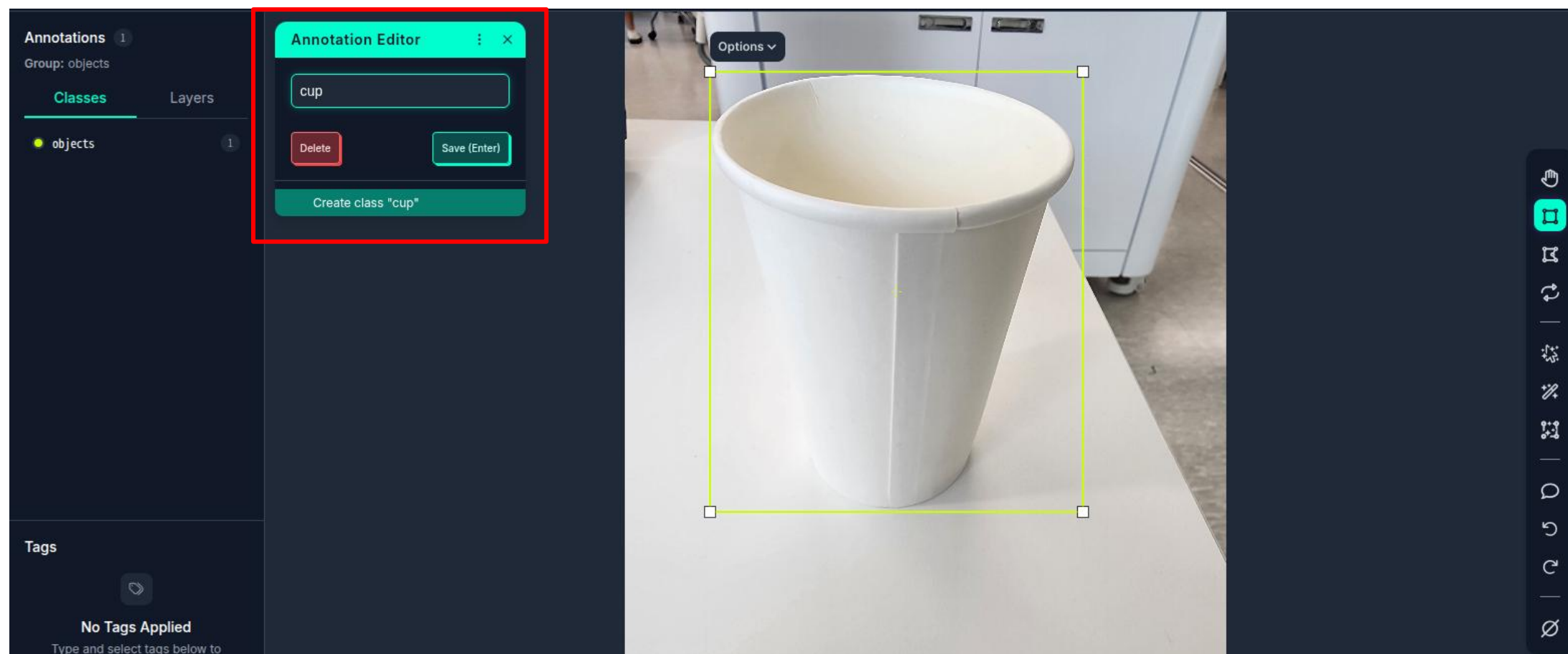
▶ 모델 학습 - 커스텀 데이터셋 생성

- 오른쪽 툴바의 AI Labeling은 credit 비용이 크므로 되도록이면 사용 지양



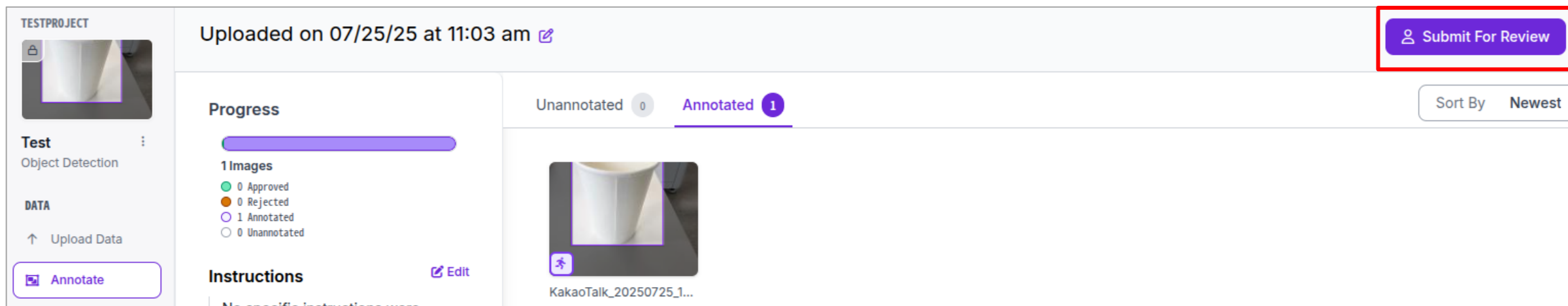
▶ 모델 학습 - 커스텀 데이터셋 생성

- Annotation Editor에서 클래스 이름 작성 후 Save 클릭



▶ 모델 학습 - 커스텀 데이터셋 생성

- 왼쪽 위 이전 화살표 버튼을 누른 후 Submit For Review (선택사항)



▶ 모델 학습 - 커스텀 데이터셋 생성

- Reviewer는 라벨링된 데이터 확인 후 승인하거나 라벨링 문제 있으면 거절 (선택사항)

The screenshot displays the 'TESTPROJECT' interface for object detection training. On the left, a sidebar shows 'Test' (Object Detection) and 'DATA' (Upload Data, Annotate). The main area shows the dataset status: 'Uploaded on 07/25/25 at 11:03 am'. A 'Progress' bar is shown, and a legend indicates 0 Approved, 0 Rejected, 1 Annotated, and 0 Unannotated images. The 'Instructions' section is partially visible. On the right, a list of images is shown, with the first image 'KakaoTalk_20250725_1...' highlighted. A red box highlights the 'Submit For Review' button in the top right corner.

TESTPROJECT

Uploaded on 07/25/25 at 11:03 am

Submit For Review

Progress

1 Images

- 0 Approved
- 0 Rejected
- 1 Annotated
- 0 Unannotated

Instructions

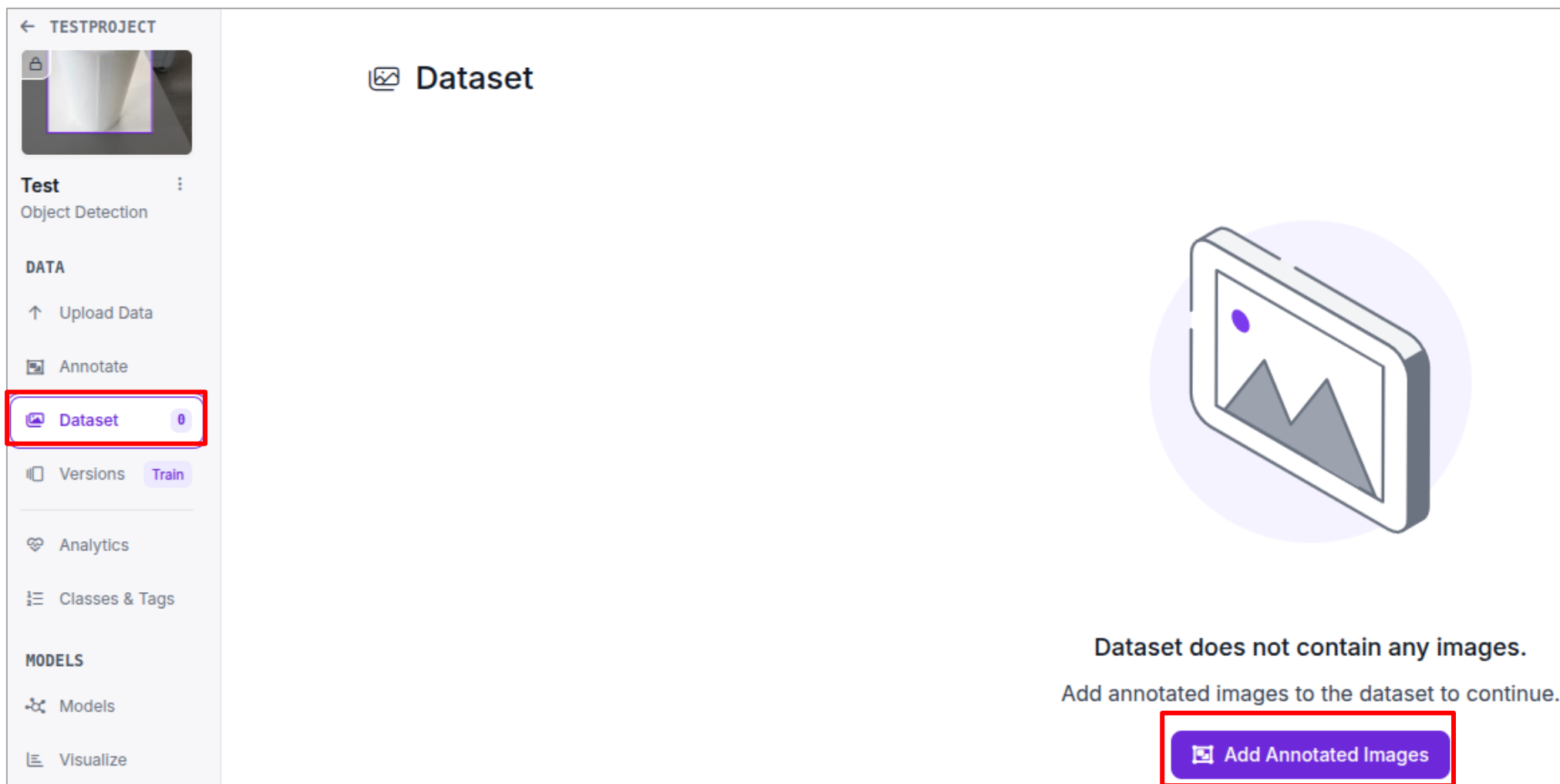
Unannotated 0 Annotated 1

Sort By Newest

KakaoTalk_20250725_1...

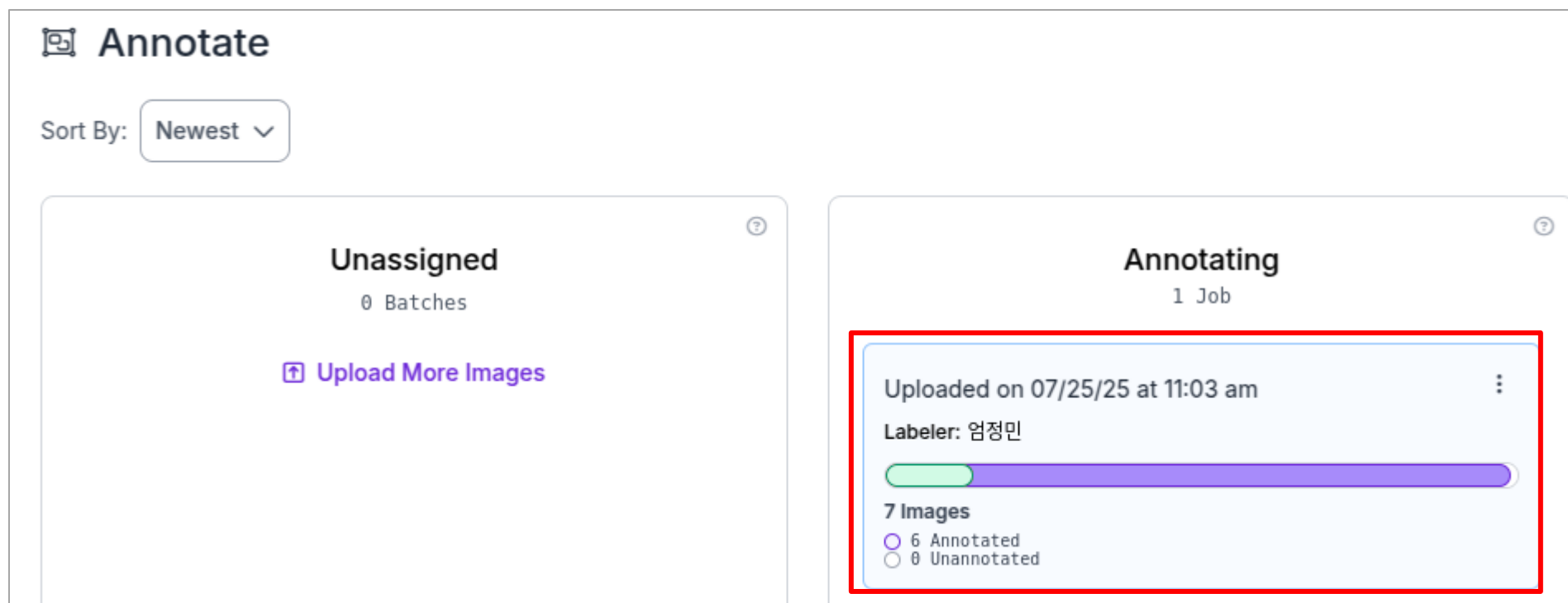
▶ 모델 학습 - 커스텀 데이터셋 생성

- Side-bar의 Dataset 클릭 후 중앙에 Add Annotated Images 클릭



▶ 모델 학습 - 커스텀 데이터셋 생성

- Annotation한 작업 클릭



▶ 모델 학습 - 커스텀 데이터셋 생성

- Add images to Dataset 클릭

Uploaded on 07/25/25 at 11:03 am [🔗](#)

Add 7 images to Dataset

Progress

7 Images

6 Annotated

0 Unannotated

Instructions

[🔗 Edit](#)

No specific instructions were added when this job was assigned

Assignment

[🔗 Reassign](#)

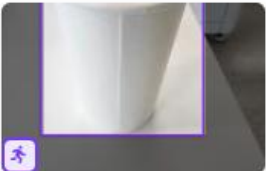
엄정민

Labeler

To Do 0

Approved 1

Annotated 6



[🔗](#)

KakaoTalk_20250725_10590...

▶ 모델 학습 - 커스텀 데이터셋 생성

- Split Images Between Train/Valid/Test를 클릭한 후 원하는 split 비율 설정하여 하단에 Add Images 클릭

The screenshot shows a dialog box titled "Add Images To Dataset". At the top, it indicates "Labeled Images to Add: 7" and "Remaining Batch Images: 0". Below this, a note states: "Your remaining Images will be split into a new batch." The "Method" section has a dropdown menu set to "Split Images Between Train/Valid/Test". The "Distribution" section features a slider with three segments: Train (70%), Valid (20%), and Test (10%). Below the slider, the counts are displayed: "Train: 5 images", "Valid: 1 images", and "Test: 1 images". A red rectangle highlights the Method dropdown, the Distribution slider, and the resulting image counts. At the bottom right, there is a purple button labeled "Add 7 Images".

Method	Train (%)	Valid (%)	Test (%)	Train Count	Valid Count	Test Count
Split Images Between Train/Valid/Test	70%	20%	10%	5 images	1 images	1 images

▶ 모델 학습 - 커스텀 데이터셋 생성

- Side bar에 Versions 클릭
- Preprocessing에서 원하는 전처리 방식 선택
 - Auto-Orient: 이미지 방향과 색상 정보 순서 통일
 - Resize: 이미지 크기 설정
 - 그 외 원하는 전처리는 Add Preprocessing Step을 클릭하여 추가
- Continue 클릭

The screenshot displays the TESTPROJECT interface. On the left sidebar, the 'Versions' button is highlighted with a red box. The main content area shows the 'Versions' section, which currently has no versions created. Below this, the 'Preprocessing' step is highlighted with a red box. It includes a list of preprocessing steps: 'Auto-Orient' and 'Resize' (Stretch to 640x640), each with an 'Edit' button. There is also an 'Add Preprocessing Step' button and a 'Continue' button at the bottom.

TESTPROJECT

Test
Object Detection

DATA

Upload Data

Annotate

Dataset 7

Versions Train

Analytics

Classes & Tags

MODELS

Models

Visualize

DEPLOY

Deployments

Versions

No versions created yet.

Experiment with different configurations to achieve better training results.

Version Name:
2025-08-04 11:41am

Source Images
Images: 7
Classes: 4
Unannotated: 0

Train/Test Split
Training Set: 5 images
Validation Set: 1 images
Testing Set: 1 images

3 Preprocessing

What can preprocessing do?
Decrease training time and increase performance by applying image transformations to all images in this dataset.

Auto-Orient Edit ×

Resize
Stretch to 640×640 Edit ×

+ Add Preprocessing Step

Continue

▶ 모델 학습 - 커스텀 데이터셋 생성

- Augmentation에서 원하는 증강기법 추가 및 수정 후 Continue 클릭

The screenshot displays the Google Cloud AI Platform interface for creating a custom dataset and training a model. The left sidebar shows the navigation menu with options like Test, DATA, Versions, Train, Analytics, Classes & Tags, MODELS, Visualize, and DEPLOY. The main area is titled 'Versions' and shows 'No versions created yet.' The right panel is titled 'Augmentation' and is highlighted with a red border. It contains a list of augmentation steps: Flip (Horizontal), Crop (0% Minimum Zoom, 20% Maximum Zoom), Saturation (Between -25% and +25%), and Noise (Up to 0.22% of pixels). Each step has an 'Edit' button and a close button (X). At the bottom, there is a '+ Add Augmentation Step' button and 'Continue' and 'Clear All' buttons.

TESTPROJECT

Test
Object Detection

DATA

Upload Data

Annotate

Dataset 7

Versions Train

Versions

No versions created yet.

Preprocessing

Auto-Orient: Applied
Resize: Stretch to 640x640

4 Augmentation

What can augmentation do?
Create new training examples for your model to learn from by generating augmented versions of each image in your training set.

Flip
Horizontal Edit X

Crop
0% Minimum Zoom, 20% Maximum Zoom Edit X

Saturation
Between -25% and +25% Edit X

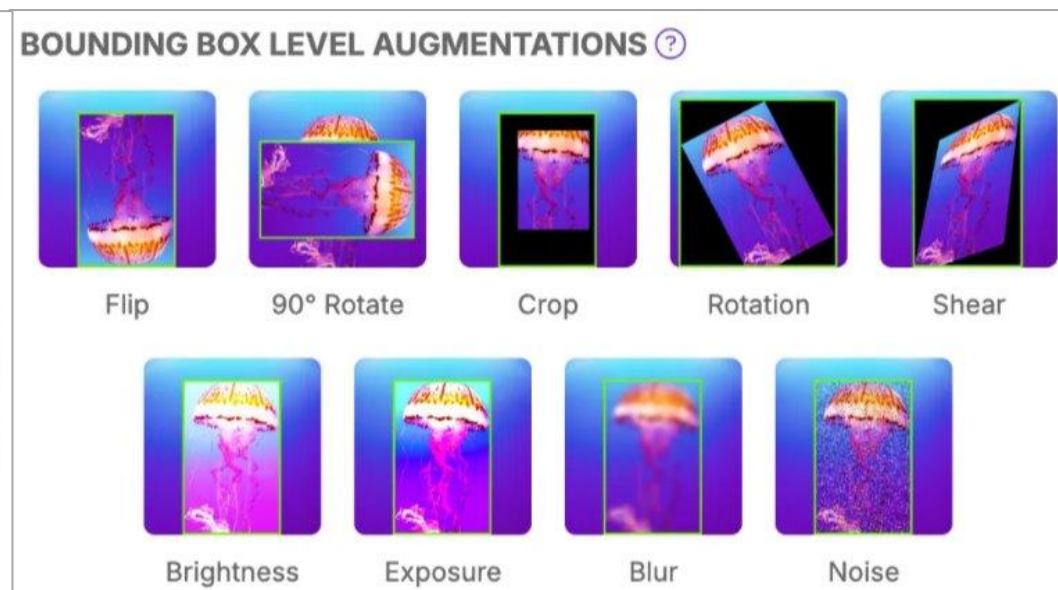
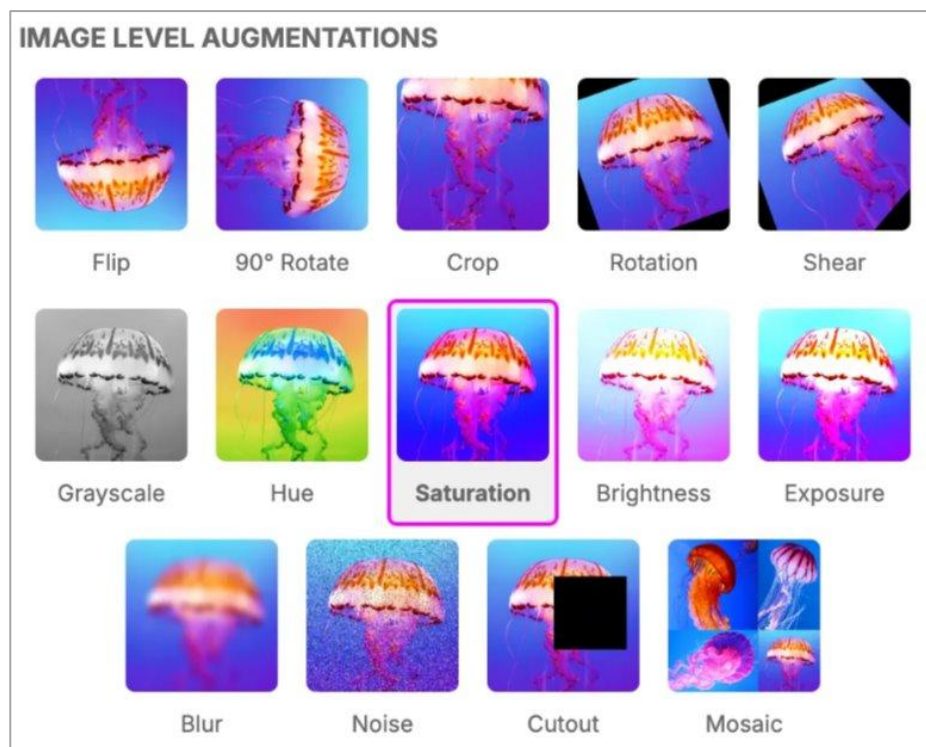
Noise
Up to 0.22% of pixels Edit X

+ Add Augmentation Step

Continue Clear All

▶ 모델 학습 - 커스텀 데이터셋 생성

▪ Augmentation 종류



▶ 모델 학습 - 커스텀 데이터셋 생성

- Maximum Version Size에서 이미지 만들 갯수 결정한 후 아래 Create 클릭

5

Create 

Review your selections and select a version size to create a moment-in-time snapshot of your dataset with the applied transformations.

Larger versions take longer to train but often result in better model performance. [See how this is calculated ↗](#)

Maximum Version Size 🔒 Trial

12 images (2x) ▼

Version Notes:

Add any version notes here...

Create

▶ 모델 학습 - 커스텀 데이터셋 생성

- 완료 후 Download Dataset을 클릭

The screenshot displays the Roboflow 'Dataset Versions' page. On the left, a sidebar shows a list of versions, with the current version 'v1' selected. The main area shows details for version 'v1', including its creation time '2025-03-24 10:36am' and a 'Download Dataset' button, which is highlighted with a red rectangle. Below this, a message states 'This version doesn't have a model.' and provides options to 'Custom Train' or 'How to Upload Custom Weights'. The 'Dataset Split' section shows '0 Images' for the Train Set, '0 Images' for the Valid Set, and '1 Images' for the Test Set. The 'Preprocessing' section indicates 'Auto-Orient: Applied' and 'Resize: Stretch to 640x640'. The 'Augmentations' section shows 'Outputs per training example: 3' and 'Flip: Horizontal, Vertical'.

▶ 모델 학습 - 커스텀 데이터셋 생성

- 원하는 모델 포맷 선택 후 Show download code 선택
- Continue 클릭

Download

Image and Annotation Format

Select a Format

Download Options

☐ Download zip to computer
Downloads all images, annotations, and classes.

☒ Show download code
Custom train this dataset using the provided code snippet in a notebook.

Cancel Continue

▶ 모델 학습 - 커스텀 데이터셋 생성

- Terminal을 선택하여 아래 code를 복사
- 복사한 코드를 터미널에 입력하여 원하는 디렉토리에 데이터셋 다운로드

 Download ×

 Jupyter

 Terminal

 Raw URL

Use this code to download and unzip **your dataset** via the command line on any *nix machine:

```
curl -L "https://app.roboflow.com/ds/4Rn16pbX0N?key=cKiF0WJYGK" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip
```



 Warning: Do not share this snippet beyond your team, it contains a private key that is tied to your Roboflow account. Acceptable use policy applies.

[Copy Snippet and Open Notebook](#)

▶ Ultralytics Predict

▪ YOLO-World

- Open-vocabulary detection: 미리 정해진 사전(클래스)이 아닌, 열려있는 단어(학습하지 않은 클래스도)로 객체를 탐지
- 새로운 객체 탐지를 위한 별도의 데이터 수집, 라벨링, 추가 학습이 불필요 -> 개발 시간 및 비용 절감
- 탐지할 단어 목록(Vocabulary)을 미리 지정 후 사용

구분	기존 YOLO	YOLO-World
탐지대상	정해진 클래스	세상의 모든 것(텍스트로 자유롭게 정의)
학습	필수 (새로운 객체는 추가 학습 필요)	불필요(Zero-Shot) (학습없이 바로 탐지)
입력	이미지 / 영상	이미지 / 영상 + 텍스트
유연성	낮음 (한정적)	높음 (무한한 확장성)

▶ Ultralytics Predict – Zero-shot predict

- YOLO-World을 활용한 zero-shot predict
 - Zero-shot predict: 모델이 학습하지 않은 클래스에 대해서 예측

```
rokey@rokey:~/ros2_ws$ yolo detect predict model=yolov8m-worldv2  
source='/home/rokey/YOLOWorld/fruits2.jpg'
```

- Model: 사용하고자 하는 YOLO-World 모델
 - (참조: <https://docs.ultralytics.com/models/yolo-world/#available-models-supported-tasks-and-operating-modes>)
 - 다양한 모델의 크기로 Zero-shot 예측해보기
- Source: 예측하고자 하는 이미지 (str 형태)
 - 저장되어 있는 이미지 디렉토리
 - 이미지 웹 주소

Object Detection

▶ 모델 학습 - YOLO-World

▪ YOLO-World finetuning

```
from ultralytics import YOLOWorld

# 사전 학습된 YOLOv8s-worldv2 모델 불러오기
model = YOLOWorld("yolov8s-worldv2.pt")

# 모델 finetuning. data에 custom dataset 경로 지정해야 함
model.train(data="coco8.yaml", epochs=100, imgsz=640)

# 학습된 모델로 test 이미지에 inference
results = model("path/to/bus.jpg")
```

Object Detection

▶ Ultralytics Predict – Zero-shot predict

- YOLO-World을 활용한 zero-shot predict (탐지할 클래스 지정 후 predict)

```
from ultralytics import YOLOWorld

# 사전 학습된 YOLOv8s-worldv2 모델 불러오기
model = YOLOWorld("yolov8m-worldv2.pt")

# 탐지하고자 하는 객체 클래스 지정
model.set_classes(["lemon", "watermelon", "pineapple", "orange", "peach", "banana"])

# 지정된 클래스를 바탕으로 모델 저장
model.save("yoloworld_mv2.pt")
```

Python

Object Detection

▶ Ultralytics Predict – Zero-shot predict

- YOLO-World을 활용한 zero-shot predict (탐지할 클래스 지정 후 predict)

```
rokey@rokey:~/ros2_ws$ yolo detect predict model=/home/rokey/YOLOWorld/yoloworld_mv2.pt source='/home/rokey/YOLOWorld/fruits2.jpg'
```

- Model을 저장한 디렉토리로 설정

Object Detection

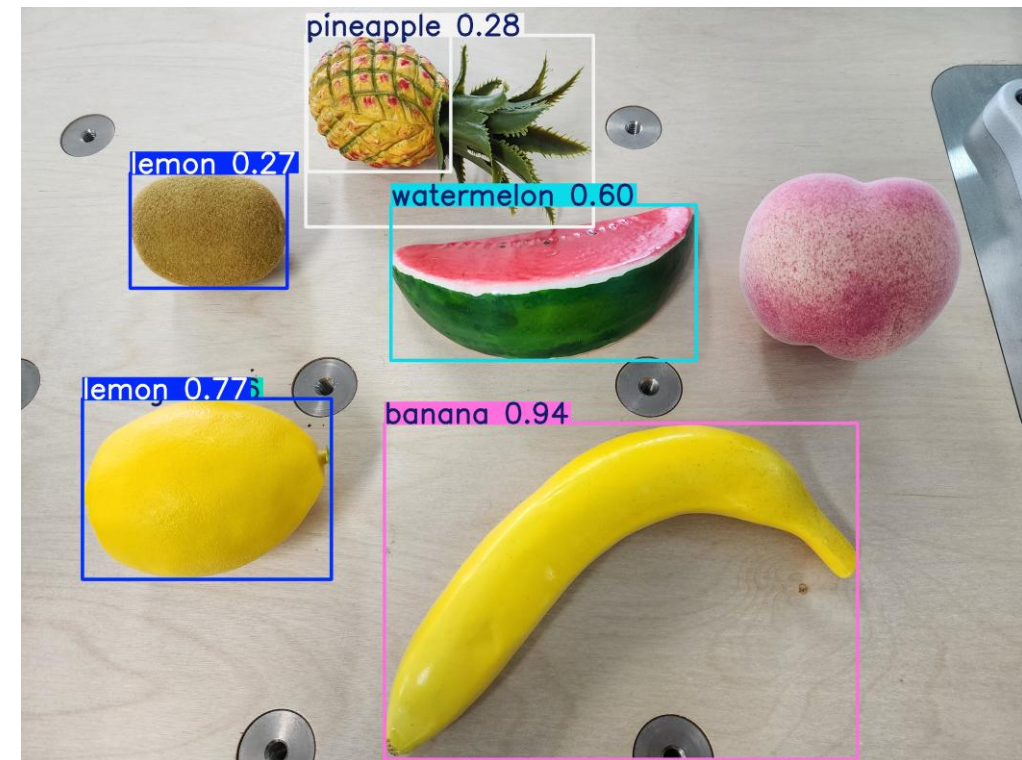
▶ Ultralytics Predict – Zero-shot predict

- 사용한 모델은 [lemon, peach, pineapple, watermelon, kiwi]를 학습하지 않은 상태

- 탐지할 클래스 미지정



- 탐지할 클래스 지정



▶ Ultralytics Predict

▪ YOLOE

- YOLO 계열 객체 탐지 모델의 효율성과 정확성을 극대화하기 위해 Detection Head 구조를 개선한 모델
- Zero-Shot Detection 지원: 모델이 학습하지 않은 객체에 대해서도 텍스트 프롬프트, 시각 프롬프트 등을 통해 탐지

구분	기존 YOLO	YOLOE
탐지 대상	정해진 클래스	텍스트 또는 이미지로 정의된 클래스
학습	필수 (새로운 객체는 추가 학습 필요)	불필요 (Zero-Shot & Prompt Mechanism)
입력	이미지/영상	이미지/영상 + 텍스트/시각 프롬프트
유연성	낮음 (한정적)	매우 높음 (프롬프트로 유연성 극대화)

▶ Ultralytics Predict

- YOLOE 유형: 프롬프트 기반 모델 vs 프롬프트 프리 모델
 - Yoloe-11m-seg.pt
 - 프롬프트 기반 모델
 - 텍스트(class 이름)나 시각적(예시 이미지) 프롬프트를 명시적으로 입력해야 관련 객체를 감지
 - 특정 객체만 찾고자 할 때, 새로운 객체를 찾고자 할 때(제로샷 감지)
 - Yoloe-11m-seg-pf.pt
 - 프롬프트 프리 모델
 - 별도의 프롬프트 없이 자체적으로 학습된 대규모 어휘를 사용하여 객체를 감지
 - 일상적이고 다양한 객체를 자동으로 감지할 때

▶ Ultralytics Predict

- YOLOE를 활용한 예측 모델 다운로드
- 참조: <https://docs.ultralytics.com/ko/models/yoloe/#textvisual-prompt-models>
- Model: 사용하고자 하는 YOLOE 모델
- Source: 예측하고자 하는 이미지 (str 형태): 저장되어 있는 이미지 디렉토리, 이미지 웹 주소, 노트북 카메라 번호

```
from ultralytics import YOLOE
```

```
# 사전 학습된 YOLOE 모델 불러오기
```

```
model = YOLOE("yoloe-v8m-seg-pf.pt") # 모델 종류는 ultralytics 홈페이지 참고
```

```
# 터미널에서 실행
```

```
yolo detect predict model="각자 pt 파일 경로" source="카메라 번호" show=True
```

```
yolo detect predict model="각자 pt 파일 경로" source="각자 파일 경로"
```

```
# 예시
```

```
# 각자 pt 파일 경로: /home/rokey/ros2_ws/src/Tutorial/0D_Tutorial/yoloe-v8m-seg-pf.pt
```

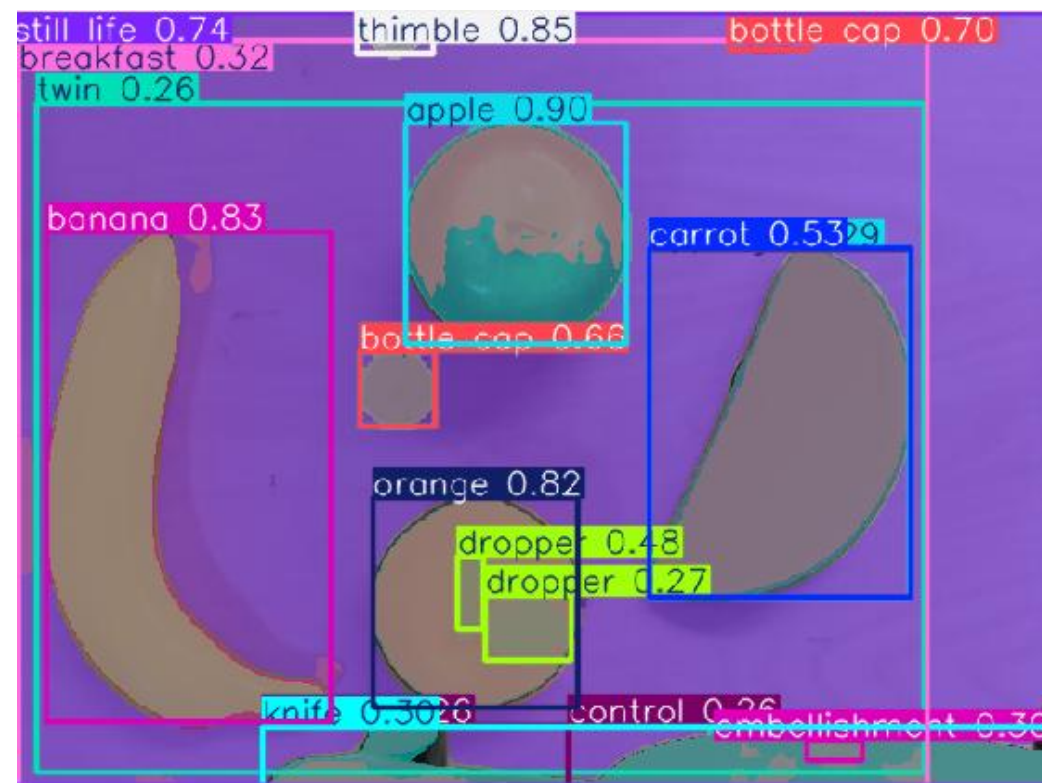
```
# 카메라 번호: 웹캠, Realsense(0~5 등)
```

```
# 각자 파일 경로: /home/rokey/ros2_ws/src/Tutorial/0D_Tutorial/sample.jpg
```

▶ Ultralytics Predict

- 예측: 프롬프트 프리 모델 Yolo-e-11m-seg-pf.pt
- 프롬프트 없이도 내부적으로 학습된 어휘를 사용하여 이미지 속의 바나나, 사과, 오렌지 등을 자동으로 감지하고 라벨링

```
# yolo-e 11m seg에서 pf 모델  
  
from ultralytics import YOLOE  
  
# 1. 프롬프트 기반 모델 로드  
model = YOLOE("yolo-e-11m-seg-pf.pt")  
  
# 2. 예측 실행  
results = model.predict(source='4', show=True)
```



▶ Ultralytics Predict

- 예측: 프롬프트 기반 모델 Yolo-11m-seg.pt / 프롬프트 제공 X 경우
- 텍스트 프롬프트(클래스 이름 리스트)를 제공하지 않고 실행하면 객체를 감지하지 못함

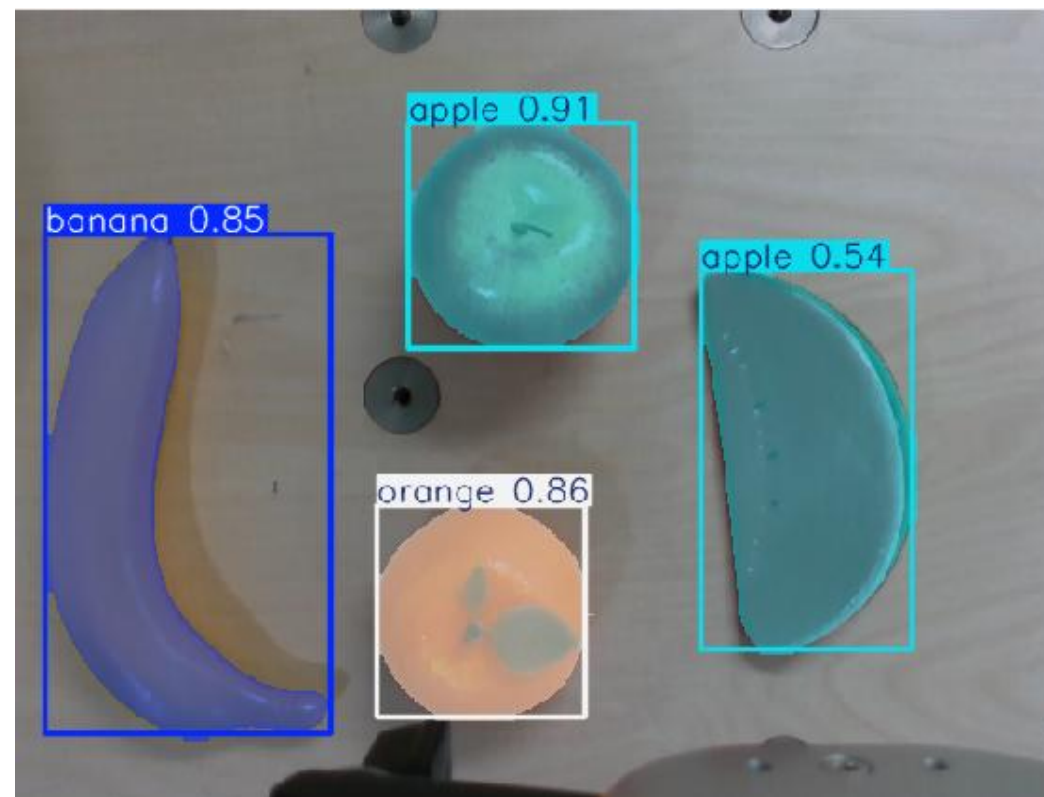
```
# yolo-11m-seg에서 프롬프트 기반 모델(프롬프트 제공 X)  
  
from ultralytics import YOLOE  
  
# 1. 프롬프트 기반 모델 로드  
model = YOLOE("yolo-11m-seg.pt")  
  
# 2. 감지할 클래스 목록(텍스트 프롬프트) 설정  
# model.set_classes(["banana", "apple", "orange"])  
  
# 3. 예측 실행  
results = model.predict(source='4', show=True)
```



▶ Ultralytics Predict

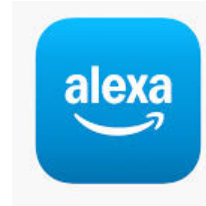
- 예측 : 프롬프트 기반 모델 Yolo-11m-seg.pt/ 프롬프트 제공 ○ 경우
- 찾고자 하는 객체 이름을 텍스트 프롬프트로 제공할 경우 원하는 객체를 감지

```
# yolo-11m-seg에서 프롬프트 기반 모델(프롬프트 제공 0)  
  
from ultralytics import YOLOE  
  
# 1. 프롬프트 기반 모델 로드  
model = YOLOE("yolo-11m-seg.pt")  
  
# 2. 감지할 클래스 목록(텍스트 프롬프트) 설정  
model.set_classes(["banana", "apple", "orange"])  
  
# 3. 예측 실행  
results = model.predict(source='4', show=True)
```



▶ Wakeup Word

- 본격적인 명령 수행이나 대화 입력을 본격적으로 처리하기 앞서 기기의 시동을 걸기 위한 문구
- Apple : 시리야~
- Google : OK Google
- Amazon : Alexa



▶ Wakeup Word Model

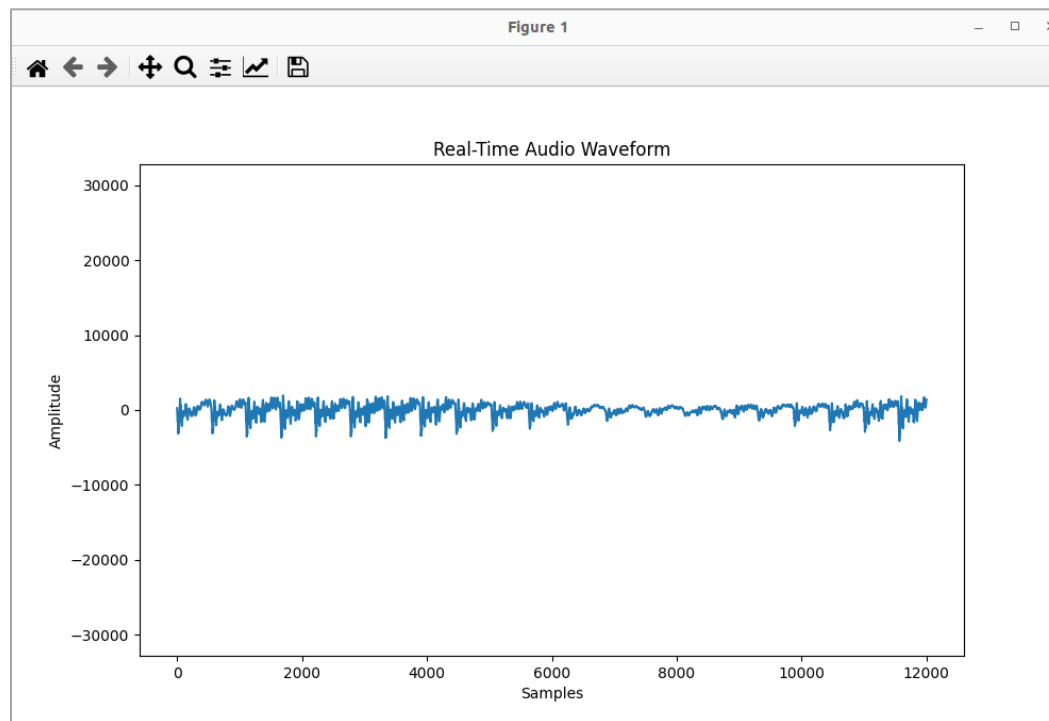
- Wakeup Word가 말로 입력되었는지를 빠르고 정확하게 감지하는 데 특화된 머신러닝 모델

▶ Wakeup Word Model – Tutorial(마이크 테스트)

- Tutorial/VoiceProcessing 폴더 내에 mic_test.py 파일 실행

```
python3 mic_test.py
```

- 그래프가 발생하는 소리에 맞추어 생성되는지 확인



Voice Processing

▶ Wakeup Word Model – Tutorial

- Tutorial/VoiceProcessing 폴더 내에 wakeup_word.py 파일 실행

```
python3 wakeup_word.py
```

- Hello rokey(미리 학습시켜 놓은 고정 키워드) 라고 말하면 인식되는지 확인

```
ALSA lib pcm_oss.c:397:(_snd_pcm_oss_open) Cannot open device /dev/dsp
ALSA lib confmisc.c:160:(snd_config_get_card) Invalid field card
ALSA lib pcm_usb_stream.c:482:(_snd_pcm_usb_stream_open) Invalid card 'card'
ALSA lib confmisc.c:160:(snd_config_get_card) Invalid field card
ALSA lib pcm_usb_stream.c:482:(_snd_pcm_usb_stream_open) Invalid card 'card'
ALSA lib pcm_dmix.c:1032:(snd_pcm_dmix_open) unable to open slave
Cannot connect to server socket err = No such file or directory
Cannot connect to server request channel
jack server is not running or cannot be started
JackShmReadWritePtr::~JackShmReadWritePtr - Init not done for -1, skipping unlock
JackShmReadWritePtr::~JackShmReadWritePtr - Init not done for -1, skipping unlock
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
confidence: 0.0
confidence: 0.0
confidence: 0.0
confidence: 0.0
confidence: 0.0
confidence: 0.00094936916
confidence: 0.0009103874
confidence: 0.0010104163
confidence: 0.0009238564
confidence: 0.0009284327
confidence: 0.0032738843
confidence: 0.008094118
confidence: 0.00090125913
confidence: 0.00089251046
confidence: 0.00089275074
confidence: 0.0008964024
confidence: 0.040820517
confidence: 0.004893717
confidence: 0.0008975874
confidence: 0.0008957061
confidence: 0.000909507
confidence: 0.0009234828
confidence: 0.99286216
Wakeword detected!
user@user:~/Desktop/Tutorial/VoiceProcessing$
```

```
~/Tutorial/VoiceProcessing$ python3
```

```
>>> import numpy as np
>>> np.__version__
'2.2.4'
>>>
```

```
~/Tutorial/VoiceProcessing$ pip install --upgrade --force-reinstall numpy==1.24.4
```

▶ STT – Tutorial

- Tutorial/VoiceProcessing 폴더 내에 STT.py 파일 실행

```
python3 STT.py
```

- Hello rokey(미리 학습시켜 놓은 고정 키워드) 라고 말하면 인식되는지 확인
- Tutorial/VoiceProcessing에 있는 .env 파일내에 OPEN_AI_API_KEY가 올바르게 되어있는지 확인 필요
 - .env 파일이 없으면 직접 생성하여 API 키 추가

▶ Keyword Extraction – Tutorial

- 자연어(e.g :망치 가져와)에서 키워드(e.g.: 망치)를 추출하는 작업
- OpenAI의 GPT-4o (LangChain 기반 LLMChain), 프롬프트 엔지니어링 진행

```
prompt_content = """
당신은 사용자의 문장에서 특정 도구와 목적지를 추출해야 합니다.

<목표>
- 문장에서 다음 리스트에 포함된 도구를 최대한 정확히 추출하세요.
- 문장에 등장하는 도구의 목적지(어디로 옮기라고 했는지)도 함께 추출하세요.

<도구 리스트>
- 망치, 드라이버, 가위, 위치1, 위치2, 위치3

<출력 형식>
- 다음 형식을 반드시 따르세요: [도구1 도구2 ... / 위치1 위치2 ...]
- 도구와 위치는 각각 공백으로 구분
- 도구가 없으면 앞쪽은 공백 없이 비우고, 목적지가 없으면 '/' 뒤는 공백 없이 비웁니다.
- 도구와 목적지의 순서는 등장 순서를 따릅니다.

<특수 규칙>
- 명확한 도구 명칭이 없지만 문맥상 유추 가능한 경우(예: "못 박는 것" → 망치)는 리스트 내 항목
...

<사용자 입력>
"{user_input}"
"""
```

▶ Keyword Extraction – Tutorial

- Tutorial/VoiceProcessing 폴더 내에 keyword_extraction.py 파일에서 output_message 변수 내용을 적절히 수정

```
if __name__ == "__main__":  
    # stt = STT()  
    # output_message = stt.speech2text()  
    output_message = "못 박을수 있는 공구를 위치1에 넣어줘"  
    extract_keyword = ExtractKeyword(use_embedding=False)  
    keyword = extract_keyword.extract_keyword(output_message)
```

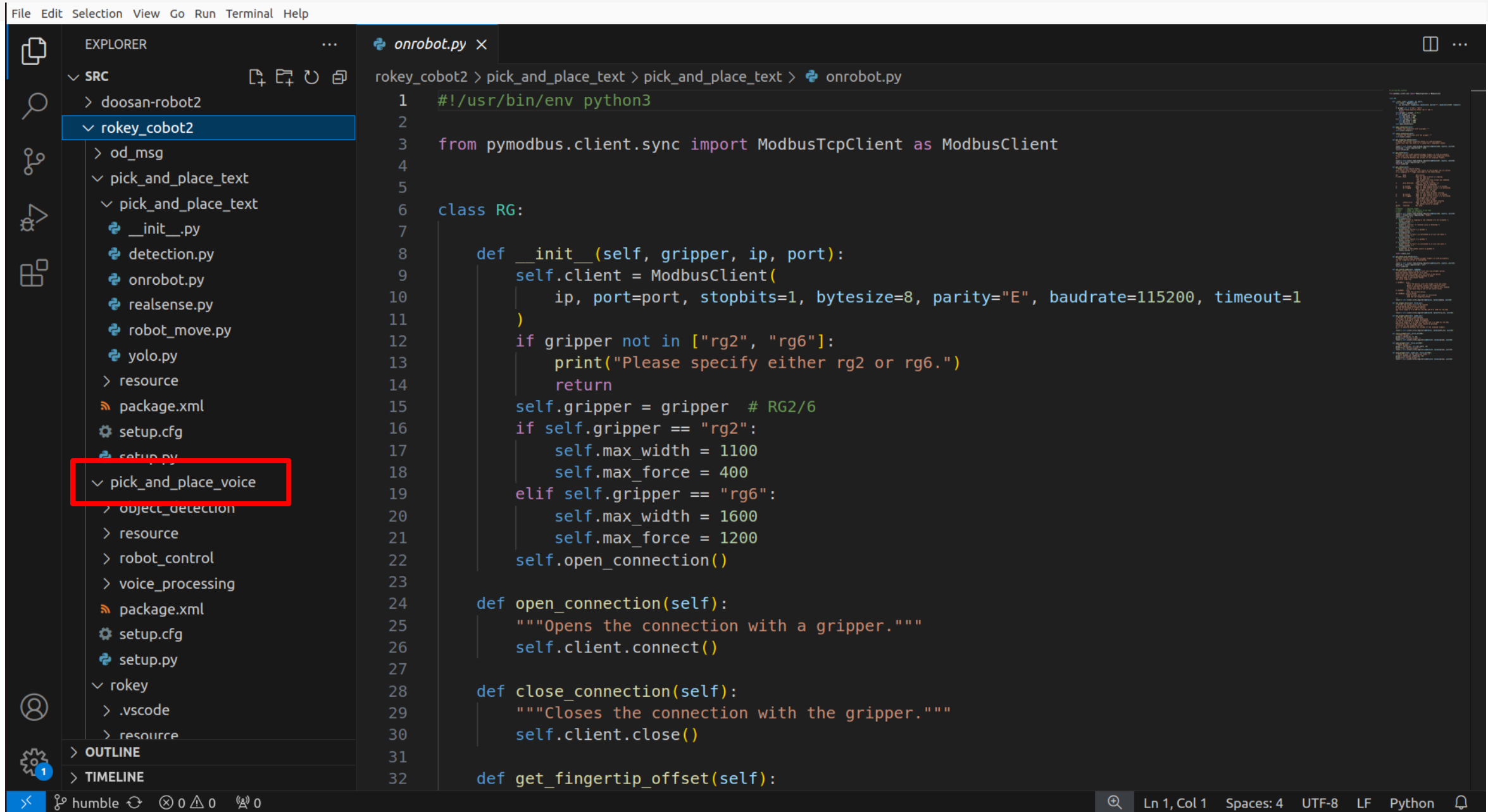
- get_keword.py 파일 실행

```
python3 keyword_extraction.py
```

Pick and Place Voice

▶ 초기 세팅

- `~/cobot_ws/src/DoosanBootcamp3rd/dsr_rokey/pick_and_place_text/resource`에 있는 model pt파일, json 파일, .env 파일 등을 `~/cobot_ws/src/DoosanBootcamp3rd/dsr_rokey/pick_and_place_voice/resource` 폴더로 복사하기
- 그 후 `cobot_ws`에서 `colcon build` 수행



The screenshot shows a VS Code editor interface. On the left, the Explorer sidebar displays a project structure for 'rokey_cobot2'. The 'pick_and_place_voice' directory is highlighted with a red rectangle. The main editor area shows the 'onrobot.py' file, which contains Python code for a gripper class. The code includes imports, a class definition, and several methods for initializing the gripper, opening and closing connections, and getting the fingertip offset.

```
File Edit Selection View Go Run Terminal Help

EXPLORER
  SRC
    > doosan-robot2
    > rokey_cobot2
      > od_msg
      > pick_and_place_text
      > pick_and_place_text
        > __init__.py
        > detection.py
        > onrobot.py
        > realsense.py
        > robot_move.py
        > yolo.py
        > resource
        > package.xml
        > setup.cfg
        > setup.py
        > pick_and_place_voice
          > object_detection
          > resource
          > robot_control
          > voice_processing
          > package.xml
          > setup.cfg
          > setup.py
        > rokey
          > .vscode
          > resource
      > OUTLINE
      > TIMELINE

rokey_cobot2 > pick_and_place_text > pick_and_place_text > onrobot.py

1  #!/usr/bin/env python3
2
3  from pymodbus.client.sync import ModbusTcpClient as ModbusClient
4
5
6  class RG:
7
8      def __init__(self, gripper, ip, port):
9          self.client = ModbusClient(
10             ip, port=port, stopbits=1, bytesize=8, parity="E", baudrate=115200, timeout=1
11         )
12         if gripper not in ["rg2", "rg6"]:
13             print("Please specify either rg2 or rg6.")
14             return
15         self.gripper = gripper # RG2/6
16         if self.gripper == "rg2":
17             self.max_width = 1100
18             self.max_force = 400
19         elif self.gripper == "rg6":
20             self.max_width = 1600
21             self.max_force = 1200
22         self.open_connection()
23
24     def open_connection(self):
25         """Opens the connection with a gripper."""
26         self.client.connect()
27
28     def close_connection(self):
29         """Closes the connection with the gripper."""
30         self.client.close()
31
32     def get_fingertip_offset(self):
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF Python

▶ 실습 진행

- 총 5개의 터미널에서 별도 실행(터미널은 cobot_ws 폴더에서 실행)

- 터미널 1

```
ros2 launch realsense2_camera rs_align_depth_launch.py depth_module.depth_profile:=848x480x30 rgb_camera.color_profile:=1280x720x30  
initial_reset:=true align_depth.enable:=true enable_rgbd:=true pointcloud.enable:=true
```

- 터미널 2

```
Colcon build  
Source install/setup.bash  
ros2 launch dsr_bringup2 dsr_bringup2_rviz.launch.py mode:=real host:=192.168.1.100 port:=12345 model:=m0609
```

Pick and Place Voice

▶ 실습 진행

- 총 5개의 터미널에서 별도 실행(터미널은 cobot_ws 디렉토리에서 실행)

- 터미널 3

```
source ~/cobot_ws/install/setup.bash
export PYTHONPATH=$PYTHONPATH:~/cobot_ws/install/dsr_common2/lib/dsr_common2/imp
ros2 run pick_and_place_voice robot_control
```

- 터미널 4

```
source ~/cobot_ws/install/setup.bash
ros2 run pick_and_place_voice get_keyword
```

- 터미널 5

```
source ~/cobot_ws/install/setup.bash
ros2 run pick_and_place_voice object_detection
```

Pick and Place Voice

▶ 실습 진행

- 터미널4

→ Confidence값이 출력되기 시작하면(0 초과 값) 마이크가 열린 상태

→ 이 때 "Hello rokey"라는 키워드를 마이크에 입력하면 confidence score가 임계값 이상이 되어 5초 동안 녹음이 시작됨

```
[INFO] [1747371701.272583972] [get_keyword_node]: MicRecorderNode initialized.  
[INFO] [1747371701.272813649] [get_keyword_node]: wait for client's request...  
open stream  
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.  
confidence: 0.0  
confidence: 0.0  
confidence: 0.0  
confidence: 0.0  
confidence: 0.0  
confidence: 0.0010433239  
confidence: 0.00095051585  
confidence: 0.00092410075  
confidence: 0.9956314  
Wakeword detected!  
음성 녹음을 시작합니다.  
5초 동안 말해주세요...
```

Pick and Place Voice

▶ 실습 진행

- 터미널4

→ 녹음이 완료되면 STT결과인 "망치 가져와"와 keyword extraction 결과인 "hammer"가 나옴

```
음성 녹음을 시작합니다.  
5초 동안 말해주세요...  
녹음 완료. whisper에 전송 중...  
STT 결과: 망치 가져와  
llm's response: ['hammer']  
object: ['hammer']  
target: []
```

▶ 실습 진행

▪ 터미널5

→ 터미널 4에서 hammer라는 키워드를 받으면 object detection에서는 현재 모델이 추론할 수 있는 class들을 보여준 후 사용자가 요청한 객체가 몇 번 Class 인지 label_id를 보여줌

```
[INFO] [1747371702.603884257] [img_node]: Waiting for client's call...
[INFO] [1747371702.689642549] [object_detection_node]: Retry getting camera intrinsics.
[INFO] [1747371702.690535478] [object_detection_node]: Retry getting camera intrinsics.
[INFO] [1747371702.690857686] [object_detection_node]: Retry getting camera intrinsics.
[INFO] [1747371702.692190960] [object_detection_node]: ObjectDetectionNode initialized.
[INFO] [1747371731.965707576] [object_detection_node]: Received request: od_msg.srv.SrvDepthPosition_
Request(target='hammer')
%d frames captured 33
classes:
{0: 'drill', 1: 'hammer', 2: 'pliers', 3: 'screwdriver', 4: 'wrench'}
label_id: 1
detections: [{'box': [331.4661948464134, 132.53612079042378, 462.4263980749882, 446.49151148940575],
'score': 0.8793405691782633, 'label': 1}, {'box': [523.0604932380445, 36.12169577858665, 639.8974294
951468, 320.8521044182055], 'score': 0.8410644170009729, 'label': 4}, {'box': [171.74977574203953, 12
8.07435746626422, 242.3395381118312, 390.1575160171046], 'score': 0.7588703885222926, 'label': 3}]
[INFO] [1747371735.135351497] [object_detection_node]: Detection: box=[331.4661948464134, 132.5361207
9042378, 462.4263980749882, 446.49151148940575], score=0.8793405691782633
```

▶ 실습 진행

▪ 터미널5

→ 그 후 object detection 결과(box와 detection된 class_id)를 보여주고 이 중 사용자가 원하는 객체의 좌표를 출력해줌

```
[INFO] [1747371702.603884257] [img_node]: Waiting for client's call...
[INFO] [1747371702.689642549] [object_detection_node]: Retry getting camera intrinsics.
[INFO] [1747371702.690535478] [object_detection_node]: Retry getting camera intrinsics.
[INFO] [1747371702.690857686] [object_detection_node]: Retry getting camera intrinsics.
[INFO] [1747371702.692190960] [object_detection_node]: ObjectDetectionNode initialized.
[INFO] [1747371731.965707576] [object_detection_node]: Received request: od_msg.srv.SrvDepthPosition_
Request(target='hammer')
%d frames captured 33
classes:
{0: 'drill', 1: 'hammer', 2: 'pliers', 3: 'screwdriver', 4: 'wrench'}
label_id: 1
detections: [{'box': [331.4661948464134, 132.53612079042378, 462.4263980749882, 446.49151148940575],
'score': 0.8793405691782633, 'label': 1}, {'box': [523.0604932380445, 36.12169577858665, 639.8974294
951468, 320.8521044182055], 'score': 0.8410644170009729, 'label': 4}, {'box': [171.74977574203953, 12
8.07435746626422, 242.3395381118312, 390.1575160171046], 'score': 0.7588703885222926, 'label': 3}]
[INFO] [1747371735.135351497] [object_detection_node]: Detection: box=[331.4661948464134, 132.5361207
9042378, 462.4263980749882, 446.49151148940575], score=0.8793405691782633
```

▶ 실습 진행

- 결과
 - 최종적으로 Wakeup Word인식 → 음성인식 → Object Detection → Pick and Place 동작하는지 확인

ROKEY BOOT CAMP

수고하셨습니다.